PONDICHERRY UNIVERSITY
RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES
DEPARTMENT OF STATISTICS



M.Sc. QUANTITATIVE FINANCE
SEMESTER III
LECTURE NOTES ON

Machine Learning in Quantitative Finance

PREPARED BY
NAME: N Rohit Vedhanandh
REG. NO.: 25MSQUFPY0002

NOVEMBER 2026

MSQF 536: MACHINE LEARNING TECHNIQUES CREDITS: 4

Course Objectives:

- *To provide knowledge of machine learning techniques.*
- *This course familiarizes in supervised and unsupervised learning.*

Course Outcomes (COs):

CO1: Learn the basic concepts of machine learning
CO2: Understand the various supervised learning methods
CO3: Knowledge of unsupervised learning algorithms
CO4: Application of dimensionality reduction methods
CO5: Study the fundamental concepts of Artificial Neural Network

Unit I: Data Pre-processing

Data Pre-processing — Different types of variables — Preparing training and test datasets — Missing Value Imputation — Imbalance in categorical data.

Unit II: Supervised Learning

Supervised Learning — Framework of Regression — Model — Loss Function — Logistic Regression — K Nearest Neighbour (KNN) — Naïve Bayes — Tree based models — Decision Tree — Random Forest — Support Vector Machine.

Unit III: Unsupervised Learning

Unsupervised Learning — Cluster Analysis — Similarity and Dissimilarity measures — Proximity measures — Types of clustering — Hierarchical clustering: Agglomerative methods and Divisive methods — Non-hierarchical clustering: k-means clustering algorithm — Density based clustering — DBSCAN.

Unit IV: Dimensionality Reduction & Ensemble Methods

Dimensionality Reduction — Principal Component Analysis — Singular Value Decomposition (SVD) — Reinforcement Learning — Recurrent Reinforcement Learning — Bagging, Boosting, XGBOOST.

Unit V: Artificial Neural Networks

Artificial Neural networks — Fundamental concepts on neural networks — Activation Function — Multi-layer Feedforward Networks — Convolutional Neural Network — Recurrent Neural Network.

Books for Reference:

1. Hao Ni, Xin Dong, Jinsons Zheng, Guangxi Yu (2021), *An Introduction to Machine Learning in Quantitative Finance*, World Scientific.
2. Mathew F Dixon, Igor Halperin, Paul Bilokon (2020), *Machine Learning in Finance: From Theory to Practice*, Springer.
3. Saurav Singla (2020), *Machine Learning For Finance*, BPB Publications.

COs	PO1	PO2	PO3	PO4	PO5
CO1	X	XXX	XXX	XX	X
CO2	XX	XXX	XXX	XX	X
CO3	XX	XXX	XXX	XX	X
CO4	XX	XX	XXX	XXX	X
CO5	XX	XX	XX	XXX	X

Contents

1 Machine Learning Foundations & Data Preprocessing **1**
- 1.1 Challenges in Machine Learning . . . 1
- 1.2 Types of Variables . . . 1
- 1.3 Data Preprocessing Pipeline . . . 1
  - 1.3.1 Scaling & Normalization . . . 2
- 1.4 Train-Test-Validation Splitting . . . 3
  - 1.4.1 Common Partitioning Ratios . . . 3
  - 1.4.2 K-Fold Cross-Validation . . . 3
  - 1.4.3 Data Leakage Prevention . . . 3
  - 1.4.4 Splitting Financial Time-Series Data . . . 3
- 1.5 Missing Data Mechanisms & Imputation . . . 4
  - 1.5.1 Missingness Mechanisms . . . 4
  - 1.5.2 Imputation Methods . . . 4
  - 1.5.3 Decision Framework for Selecting Imputation Methods . . . 4
- 1.6 Imbalanced Categorical Data . . . 4
  - 1.6.1 Consequences of Class Imbalance . . . 5
  - 1.6.2 Remedial Solutions . . . 5
- 1.7 Model Evaluation Metrics . . . 5
- 1.8 Mistakes to be Avoided in ML Pipelines . . . 6
- 1.9 Key Takeaways . . . 6

2 K-Nearest Neighbors (KNN) **8**
- 2.1 Introduction to KNN . . . 8
  - 2.1.1 Core Algorithmic Characteristics . . . 8
  - 2.1.2 Algorithmic Workflow . . . 8
- 2.2 Distance Metrics . . . 8
  - 2.2.1 Euclidean Distance . . . 9
  - 2.2.2 Manhattan Distance . . . 9
  - 2.2.3 Minkowski Distance . . . 9
- 2.3 Worked Numerical Examples . . . 9
  - 2.3.1 Fruit Classification via Euclidean Distance . . . 9
  - 2.3.2 Fintech Stock Recommendation with Standardized Indicators . . . 11
  - 2.3.3 Bank Loan Approval via Manhattan Distance . . . 12
  - 2.3.4 Real Estate House Price Prediction (KNN Regression) . . . 13
- 2.4 Parameter Selection: Small k vs. Large k . . . 14

3 Naive Bayes Classification **15**
- 3.1 Probability Foundations & Notations . . . 15
  - 3.1.1 Key Probability Identities . . . 15
  - 3.1.2 Conditional Probability & Multiplication Rule . . . 15
  - 3.1.3 Statistical Independence . . . 16

3.2 Bayes' Theorem . . . 16
3.3 Worked Numerical Example: Credit Risk Classification . . . 16
3.4 The Bayes Classifier Framework . . . 17
3.4.1 Marginal Evidence Across K Classes . . . 17
3.4.2 Optimal Bayes Decision Rule . . . 17
3.5 The Naive Bayes Classifier . . . 17
3.5.1 The Challenge of High-Dimensional Likelihoods . . . 17
3.5.2 The Naive Conditional Independence Assumption . . . 18
3.5.3 Naive Bayes Classification Rule . . . 18
3.5.4 Algorithmic Step-by-Step Sequence . . . 18

4 Logistic Regression **20**
4.1 Introduction and Core Motivation . . . 20
4.1.1 Why Not Standard Linear Regression? . . . 20
4.2 The Sigmoid (Logistic) Function and Probability Modeling . . . 20
4.2.1 Algebraic Equivalence of the Sigmoid Function . . . 21
4.2.2 Properties of the Sigmoid Curve and Decision Threshold . . . 21
4.3 Key Terms and Matrix Notation . . . 22
4.3.1 Matrix Representation . . . 22
4.4 Odds, Log-Odds (Logit), and Parameter Interpretation . . . 22
4.4.1 Definition of Odds . . . 22
4.4.2 Log-Odds (Logit Function) . . . 23
4.4.3 Interpreting Coefficients and Odds Ratios . . . 23
4.5 Model Training via Maximum Likelihood Estimation (MLE) . . . 24
4.6 Assumptions of Logistic Regression . . . 24
4.7 Variants, Metrics, and Trade-Offs . . . 24
4.7.1 Types of Logistic Regression . . . 24
4.7.2 Model Evaluation Metrics . . . 25
4.7.3 Advantages and Disadvantages . . . 25
4.8 End-to-End Workflow and Generalized Linear Model (GLM) Classification . . . 25
4.8.1 Five-Step Implementation Pipeline . . . 25
4.8.2 Membership in Generalized Linear Models (GLM) . . . 26

5 Decision Trees **27**
5.1 Overview and Core Concepts . . . 27
5.1.1 Structural Components of a Decision Tree . . . 27
5.1.2 General Recursive Splitting Algorithm . . . 27
5.2 Impurity Measures and Splitting Criteria . . . 28
5.2.1 Standard Tree Induction Algorithms . . . 28
5.2.2 Entropy and Information Gain . . . 28
5.2.3 Gain Ratio (C4.5) . . . 29
5.2.4 Gini Impurity (CART) . . . 29
5.2.5 Comparison: Gini Impurity vs. Entropy . . . 29
5.2.6 Advantages and Disadvantages of Decision Trees . . . 29
5.3 Solved Problem 5.1: Play Tennis Decision Tree . . . 30
5.4 Solved Problem 5.2: Credit Loan Approval Decision Tree . . . 34
5.5 Solved Problem 5.3: Candidate Job Offer Decision Tree . . . 38

6 Support Vector Machines **42**
6.1 Core Principles and Mathematical Foundations . . . 42
6.1.1 Fundamental Definitions and Geometry . . . 42
6.1.2 Functional Margin and Hard-Margin Condition . . . 43
6.2 Boundary Identification and Analytical Solution Techniques . . . 43
6.2.1 Horizontal Separation ($x_2 = c$) . . . 43
6.2.2 Vertical Separation ($x_1 = c$) . . . 43
6.2.3 Diagonal Separation ($x_2 - x_1 = c$ or $x_2 + x_1 = c$) . . . 44
6.3 Solved Problem 6.1: Horizontal Decision Boundary . . . 44
6.4 Solved Problem 6.2: Vertical Decision Boundary . . . 46
6.5 Solved Problem 6.3: Diagonal Decision Boundary . . . 48

7 Unsupervised Learning & Cluster Analysis **51**
7.1 Foundations of Unsupervised Learning . . . 51
7.1.1 Conceptual Framework and Objectives . . . 51
7.1.2 Supervised versus Unsupervised Learning . . . 51
7.1.3 The Standard Clustering Pipeline . . . 51
7.1.4 The Data Matrix and Variable Typology . . . 52
7.2 Proximity, Distance Metrics, and Metric Axioms . . . 52
7.2.1 Similarity, Dissimilarity, and Conversion . . . 52
7.2.2 Formal Metric Axioms for Distance Functions . . . 54
7.2.3 Standard Continuous Distance Metrics . . . 55
7.2.4 Proximity for Binary and Mixed Data . . . 55
7.3 Taxonomy of Clustering Methodologies . . . 56
7.4 Hierarchical Clustering . . . 56
7.4.1 Dendrogram Architecture and Horizontal Cuts . . . 56
7.4.2 Agglomerative versus Divisive Paradigms . . . 57
7.4.3 The Agglomerative Induction Algorithm . . . 57
7.4.4 Inter-Cluster Linkage Criteria . . . 57
7.4.5 Divisive Splitting Strategies: Monothetic versus Polythetic . . . 57
7.4.6 Strengths, Limitations, and Empirical Heuristics . . . 57
7.5 Non-Hierarchical Partitioning: The K-Means Algorithm . . . 59
7.5.1 Foundational Principles and Procedure . . . 59
7.6 Density-Based Clustering: DBSCAN, HDBSCAN, and OPTICS . . . 60
7.6.1 Core Principles of Density Clustering . . . 60
7.6.2 Local Point Typology . . . 60
7.6.3 Advanced Density Algorithms: HDBSCAN and OPTICS . . . 60
7.6.4 Practical Considerations in Unsupervised Pipelines . . . 61

8 Dimensionality Reduction: PCA & SVD **62**
8.1 The Curse of Dimensionality and Feature Selection . . . 62
8.1.1 Motivation and Problem Formulation . . . 62
8.1.2 Taxonomy: Feature Selection versus Feature Extraction . . . 62
8.2 Principal Component Analysis (PCA) . . . 63
8.2.1 Foundations and Orthogonal Projections . . . 63
8.2.2 The Algorithmic Workflow of PCA . . . 64
8.2.3 Solved Numerical Example: PCA on a Bivariate Dataset . . . 65
8.3 Singular Value Decomposition (SVD) . . . 67
8.3.1 Mathematical Formulation . . . 67
8.3.2 Connection Between Singular Values and Eigenvalues . . . 68
8.3.3 Truncated Low-Rank Approximation (Eckart-Young Theorem) . . . 68
8.3.4 Solved Numerical Example: SVD of a 2×2 Matrix . . . 68

8.3.5 Comparative Synthesis: PCA versus SVD 69

9 Reinforcement Learning & Financial Trading 71
9.1 Principles of Reinforcement Learning 71
9.1.1 The Trial-and-Error Paradigm 71
9.1.2 The Agent–Environment Interaction Cycle 71
9.1.3 Discounted Returns and Core Formal Terminology 72
9.1.4 Prominent Real-World Application Domains 72
9.2 Recurrent Reinforcement Learning (RRL) in Financial Trading 72
9.2.1 Direct Policy Optimization versus Value-Function Methods 72
9.2.2 The Recurrent Trading Policy Model 74
9.2.3 Objective Utilities and Gradient Ascent Optimization 74
9.3 Action Selection: Exploration versus Exploitation 75
9.3.1 The ϵ-Greedy Action Selection Policy 75
9.4 Q-Learning: Tabular Temporal-Difference Control 75
9.4.1 The Q-Table and the Bellman Update Rule 75
9.5 Comparative Synthesis Across Learning Paradigms 76

10 Ensemble Learning: Bagging, Boosting, & XGBoost 78
10.1 Principles of Ensemble Learning 78
10.1.1 The Wisdom of the Crowd 78
10.1.2 Bias–Variance Error Decomposition and Ensemble Strategies 78
10.2 Bootstrap Aggregating (Bagging) 78
10.2.1 Bootstrap Resampling and Out-of-Bag (OOB) Data 78
10.2.2 The Bagging Procedural Protocol 79
10.2.3 Random Forests: De-Correlating Ensembles via Feature Subsampling . . 80
10.3 Boosting Paradigms: AdaBoost and Gradient Boosting 80
10.3.1 The Sequential Error-Focusing Philosophy 80
10.3.2 Adaptive Boosting (AdaBoost) 80
10.3.3 Gradient Boosting 81
10.3.4 Comprehensive Comparison: Bagging versus Boosting 81
10.4 XGBoost (Extreme Gradient Boosting) 81
10.4.1 Algorithmic Innovations and Formal Objective 81
10.4.2 Core Technical Features and Quantitative Benefits 82

Comprehensive Formula Reference Sheet 84
Chapter 1: Machine Learning Foundations and Data Preprocessing 84
Chapter 2: K-Nearest Neighbors (KNN) 85
Chapter 3: Naive Bayes Classification 86
Chapter 4: Logistic Regression 87
Chapter 5: Decision Trees 89
Chapter 6: Support Vector Machines (SVM) 90
Chapter 7: Unsupervised Learning and Cluster Analysis 91
Chapter 8: Dimensionality Reduction: PCA and SVD 93
Chapter 9: Reinforcement Learning and Financial Trading 94
Chapter 10: Ensemble Learning: Bagging, Boosting, and XGBoost 95

Chapter 1

Machine Learning Foundations & Data Preprocessing

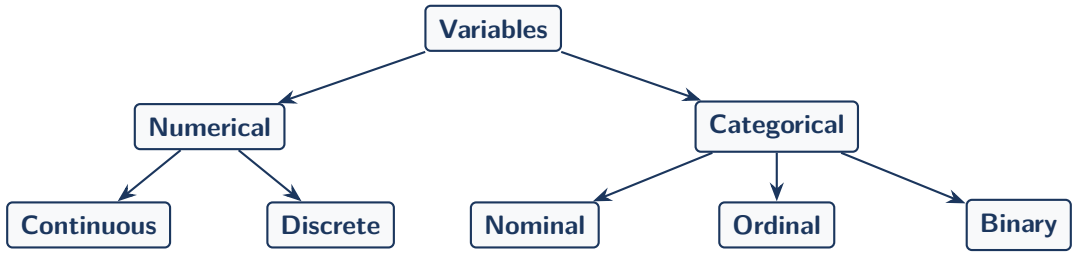
1.1 Challenges in Machine Learning

Machine learning applications in financial engineering face several structural and regulatory constraints:

- **Requires Large, Clean Data**: Models demand substantial volumes of high-quality, cleansed historical records to generalize effectively.
- **Risk of Overfitting**: Complex models risk fitting noise and idiosyncratic market movements rather than genuine underlying signals.
- **Interpretability ("Black Box")**: Advanced algorithms often lack transparent mechanistic explanations, creating auditing hurdles.
- **Regulatory & Ethical Concerns in Finance**: Strict regulatory compliance mandates model explainability, fairness, and accountability in financial decision-making.

1.2 Types of Variables

Variables encountered in machine learning datasets are broadly categorized into numerical and categorical types:



1.3 Data Preprocessing Pipeline

The standard end-to-end data preparation workflow transforms raw financial inputs into model-ready datasets:

- **Cleaning**: Removes duplicate records, corrects erroneous entries, and handles missing observations.

Table 1.1: Taxonomy of Data Types and Financial Examples

Type	Subtype	Meaning	Finance Example
Numerical	Continuous	Any value in a continuous range	Stock price, portfolio return, volatility
Numerical	Discrete	Countable whole numbers	Number of trades per day, number of defaults
Categorical	Nominal	No natural ordering among categories	Sector: Banking, IT, Pharma
Categorical	Ordinal	Natural hierarchical order exists	Credit Rating: AAA > AA > A > BBB
Categorical	Binary	Exactly two distinct categories	Loan Default: Yes / No

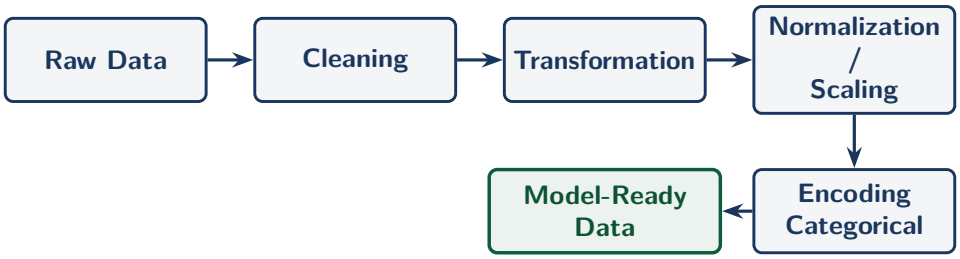


Figure 1.1: End-to-End Data Preprocessing Pipeline from Raw Inputs to Model-Ready Data.

- **Transformation**: Converts raw figures into mathematically suitable modeling formats (e.g., calculating log returns from sequential stock prices).
- **Normalization / Scaling**: Brings numerical features across heterogeneous magnitudes into a common comparable scale.
- **Encoding**: Converts non-numeric categorical attributes into numerical indicator representations.

1.3.1 Scaling & Normalization

Features measured in disparate units must be rescaled to ensure distance-based and gradient-based algorithms treat features proportionately:

Min-Max Scaling

Rescales feature values into the bounded closed interval $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1.1}$$

Z-Score Standardization

Standardizes observations using empirical sample mean and sample standard deviation:

$$Z = \frac{x - \mu}{\sigma} \tag{1.2}$$

1.4 Train-Test-Validation Splitting

Datasets are partitioned into distinct subsets to evaluate model performance on unseen data, verifying whether the algorithm generalizes rather than memorizing training samples:

- **Training Set**: Used to fit model parameters and learn patterns.
- **Validation Set**: Used during model development to tune hyperparameters and guide model selection.
- **Testing Set**: Used strictly for final unbiased model performance evaluation.

1.4.1 Common Partitioning Ratios

- **Train-Test Splits**: $80\% - 20\%$ or $70\% - 30\%$.
- **Train-Validation-Test Splits**: $70\% - 15\% - 15\%$.

1.4.2 K-Fold Cross-Validation

In k-fold cross-validation:

1. The training dataset is partitioned into k equal parts ("folds").
2. The model is trained on $k-1$ folds and validated on the remaining fold.
3. This procedure is repeated k times, rotating the held-out validation fold sequentially across all splits.

Benefits & Common Choices

- Provides a significantly more robust estimate of true out-of-sample model performance.
- Reduces vulnerability to an anomalous single lucky or unlucky split.
- Common configurations: $k = 5$ or $k = 10$.

1.4.3 Data Leakage Prevention

Data Leakage Warning

Never scale or normalize the entire dataset prior to splitting.
If sample mean μ and standard deviation σ are calculated over the aggregate dataset, information from the test set leaks directly into the training stage.
Mandatory Pipeline Sequence:

1. Split the raw dataset first.
2. Fit the scaler/imputer exclusively on training data.
3. Transform validation and test data using the saved training parameters.

1.4.4 Splitting Financial Time-Series Data

For time-series data like stock prices, a purely random split is fundamentally problematic because it mixes past and future observations. Sequential financial records must strictly use **chronological splits**.

1.5 Missing Data Mechanisms & Imputation

1.5.1 Missingness Mechanisms

Table 1.2: Mechanisms of Missing Data and Practical Financial Examples

Type	Meaning	Finance Example
MCAR	**Missing Completely at Random**: Missingness is entirely independent of observed and unobserved variables.	Trade entry terminal intermittently experiences random network packet drops logging trades.
MAR	**Missing at Random**: Missingness is statistically related to other observed features in the dataset.	Younger retail banking customers are systematically more likely to leave the optional income field blank.
MNAR	**Missing Not at Random**: Missingness is directly correlated with the actual missing value itself.	Highly indebted distressed borrowers deliberately omit outstanding secondary credit obligations.

1.5.2 Imputation Methods

Common techniques for replacing missing data include: • Mean / Median / Mode, • K-Nearest Neighbors (KNN) Imputation, • Forward Fill, • Backward Fill, and • Interpolation.

Table 1.3: Advantages and Disadvantages of Imputation Methods

Method	Advantages	Disadvantages
Mean / Median	Simple, fast to compute	Distorts sample variance; ignores inter-feature relationships.
Mode	Directly applicable to categorical data	Can artificially over-represent the most common category.
KNN	Exploits multivariate relationships between features	Computationally expensive on large financial datasets.
Forward / Backward Fill	Well-suited for time-series sequences	Assumes most recent observed value remains representative.
Interpolation	Produces smooth estimates capturing trends	Unsuitable for non-sequential, unordered data.

1.5.3 Decision Framework for Selecting Imputation Methods

1.6 Imbalanced Categorical Data

Class imbalance arises when one category of the target variable appears far more frequently than other categories:

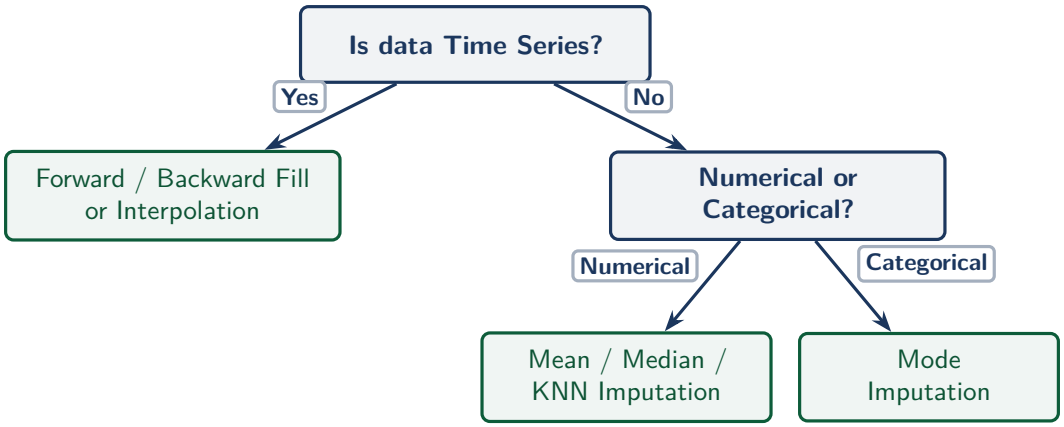


Figure 1.2: Decision Framework for Selecting Missing Value Imputation Methods.

Financial Example

In credit card fraud detection:

- Genuine Transactions: 99.9%
- Fraudulent Transactions: 0.1%

A naive model that blindly predicts "no fraud" every single time achieves 99.9% raw accuracy, yet is entirely useless. Thus, **accuracy alone is fundamentally misleading**.

1.6.1 Consequences of Class Imbalance

- Biases model parameters heavily toward majority class patterns.
- Renders overall accuracy metric uninformative for decision-makers.

1.6.2 Remedial Solutions

- **Oversampling**: Duplicating or synthetically generating minority class instances.
- **Undersampling**: Down-sampling majority class observations.
- **SMOTE**: Synthetic Minority Over-sampling Technique, interpolating new instances along minority feature lines.

1.7 Model Evaluation Metrics

For binary classification problems, predictions and actual outcomes are summarized in a confusion matrix:

Table 1.4: Binary Classification Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Precision

Of all instances predicted as positive, how many were genuinely correct?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{1.3}$$

Recall

Of all actual positive instances in the population, how many did the model identify?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{1.4}$$

F_1-Score

Harmonic mean of Precision and Recall, balancing false alarms against missed detections:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{1.5}$$

1.8 Mistakes to be Avoided in ML Pipelines

1. **Global Scaling Before Splitting**: Scaling or normalizing the entire dataset prior to train/test partitioning causes data leakage.
2. **Blind Mean Imputation on MNAR Data**: Imputing missing values with unconditional means when data is Missing Not at Random injects substantial estimation bias.
3. **Ignoring Class Imbalance**: Relying exclusively on overall classification accuracy in skewed datasets conceals poor minority detection.
4. **Unchecked One-Hot Encoding on High-Cardinality Variables**: Creating dummy variables for high-cardinality categorical features leads to severe dimensionality inflation.
5. **Failing to Shuffle Non-Time-Series Data**: Neglecting random shuffling before partitioning standard cross-sectional data risks clustering-induced bias.

1.9 Key Takeaways

Core Principles of ML Data Preparation

i) **Pre-processing dominates**: Data preprocessing accounts for $60\% - 80\%$ of total machine learning workflow effort.

ii) **Always split first**: Partition data into training, validation, and test sets before performing any scaling or modeling.

iii) **Diagnose missingness**: Determine whether missing values are MCAR, MAR, or MNAR before selecting an imputation strategy.

iv) **Address class imbalance**: Use SMOTE, oversampling, or undersampling; accuracy alone is never sufficient.

v) **Respect temporal order**: For financial and time-series data, strictly employ chrono-

logical splits and forward-looking imputation to prevent look-ahead bias.

Chapter 2

K-Nearest Neighbors (KNN)

2.1 Introduction to KNN

Definition 2.1.1: K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a non-parametric, distance-based supervised machine learning algorithm used for both classification and regression tasks.

2.1.1 Core Algorithmic Characteristics

- **Supervised Learning**: Operates on labeled training datasets to predict continuous targets or discrete classes.
- **Dual Capability**: Utilized for classification (via majority voting) as well as regression (via local averaging).
- **Distance-Based Metric Method**: Assigns predictions purely based on geometric proximity in the feature space.
- **Lazy Learning / Instance-Based**: Does not construct an explicit internal model or parameter set during training; retains the entire training dataset and defers computation until inference time.

2.1.2 Algorithmic Workflow

The operational execution pipeline of KNN proceeds through four sequential stages:

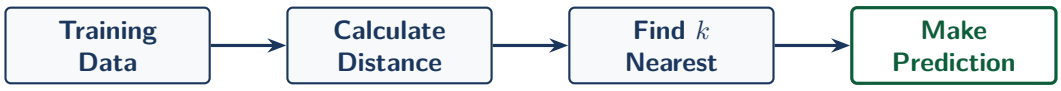


Figure 2.1: Sequential Execution Pipeline of the k-Nearest Neighbors Algorithm.

2.2 Distance Metrics

Geometric closeness between query observations and stored instances is computed using formal metric distance functions.

2.2.1 Euclidean Distance

For two observations in a 2-dimensional feature space, $\mathbf{X} = (x_1, x_2)$ and $\mathbf{Y} = (y_1, y_2)$:

$$d(\mathbf{X}, \mathbf{Y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2} \tag{2.1}$$

For an arbitrary p-dimensional feature vector (or n features):

$$d(\mathbf{X}, \mathbf{Y}) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2} = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2} \tag{2.2}$$

2.2.2 Manhattan Distance

The Manhattan distance (L_1 norm) measures rectilinear city-block distance across coordinates:

$$d(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{n} |x_i - y_i| \tag{2.3}$$

- **For 2 Dimensions**: $d = |x_1 - x_2| + |y_1 - y_2|$
- **For 3 Dimensions**: $d = |x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|$

2.2.3 Minkowski Distance

The generalized Minkowski metric of order $p \geq 1$ encompasses both Manhattan and Euclidean distances:

$$d(\mathbf{X}, \mathbf{Y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p} \tag{2.4}$$

Table 2.1: Minkowski Distance Orders and Resulting Metrics

Order p	**Distance Metric**	**Mathematical Formula**
$p = 1$	Manhattan distance	$\sum \lvert x_i - y_i \rvert$
$p = 2$	Euclidean distance	$\sqrt{\sum (x_i - y_i)^2}$
$p \geq 1$	Minkowski distance	$\left(\sum \lvert x_i - y_i \rvert^p \right)^{1/p}$

Where:

- n = number of dimensions (features)
- p = positive real number (the "order" of the distance metric)
- x_i, y_i = the i-th feature coordinates of the two compared observations

2.3 Worked Numerical Examples

2.3.1 Fruit Classification via Euclidean Distance

Example 2.3.1: Fruit Classification (Apple vs. Lemon)

A dataset contains observations evaluated on two physical attributes: *Sweetness* and *Crunchiness*.

Fruit Training Dataset

Point	Sweetness	Crunchiness	Class
A	7	7	Apple
B	8	6	Apple
C	3	8	Lemon
D	2	7	Lemon
E	6	8	Apple
F	1	9	Lemon

Task: Classify a new query observation $\mathbf{X} = (5, 7)$ using $k = 3$ and Euclidean distance.

Solution:

Step 1: Compute Euclidean distances from query point $\mathbf{X}(5, 7)$:

$$\text{To } A(7,7): \quad d = \sqrt{(5-7)^2 + (7-7)^2} = \sqrt{4+0} = 2.00$$

$$\text{To } B(8,6): \quad d = \sqrt{(5-8)^2 + (7-6)^2} = \sqrt{9+1} = \sqrt{10} \approx 3.16$$

$$\text{To } C(3,8): \quad d = \sqrt{(5-3)^2 + (7-8)^2} = \sqrt{4+1} = \sqrt{5} \approx 2.23$$

$$\text{To } D(2,7): \quad d = \sqrt{(5-2)^2 + (7-7)^2} = \sqrt{9+0} = 3.00$$

$$\text{To } E(6,8): \quad d = \sqrt{(5-6)^2 + (7-8)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

$$\text{To } F(1,9): \quad d = \sqrt{(5-1)^2 + (7-9)^2} = \sqrt{16+4} = \sqrt{20} \approx 4.04$$

Step 2: Sort distances in ascending order:

Point	Distance	Class
E	1.41	Apple
A	2.00	Apple
C	2.23	Lemon
D	3.00	Lemon
B	3.16	Apple
F	4.04	Lemon

Step 3: Evaluate majority class for $k = 3$:

$$\{k = 3\} = \{E, A, C\}$$

- Apple: 2 votes (E, A)
- Lemon: 1 vote (C)

$\therefore$ $\mathbf{X}$ is to be classified as **Apple**.

2.3.2 Fintech Stock Recommendation with Standardized Indicators

Example 2.3.2: Fintech Stock Recommendation (Buy, Hold, Avoid)

A fintech company builds a KNN system to recommend whether a stock is a **Buy**, **Hold**, or **Avoid** based on three financial indicators: Price-to-Earnings (P/E), Return on Equity (ROE), and Debt-to-Equity (D/E).
The training samples are provided after standardizing each indicator via $Z = \frac{x-\mu}{\sigma}$:

Standardized Financial Indicator Training Dataset

Stock	$P/E\,(Z)$	$ROE\,(Z)$	$D/E\,(Z)$	**Class**
S_1	-1.351	1.542	-1.184	Buy
S_2	-0.787	0.806	-0.858	Buy
S_3	-0.129	-0.299	-0.204	Hold
S_4	0.341	-0.668	0.286	Hold
S_5	1.750	-1.404	1.757	Avoid
S_6	1.093	-1.036	1.267	Avoid
S_7	-1.069	1.174	-1.021	Buy
S_8	0.153	-0.115	-0.041	Hold

Task: Predict the recommendation class for a new stock with standardized features:

$$\mathbf{X} = (-0.559,\ 0.437,\ -0.531)$$

using $k = 3$ and Euclidean distance.

Solution:
Step 1: Compute Euclidean distance for each stock:

$$d(\mathbf{X}, S_i) = \sqrt{(z_{1,X} - z_{1,S_i})^2 + (z_{2,X} - z_{2,S_i})^2 + (z_{3,X} - z_{3,S_i})^2}$$

Stock	$P/E\,(Z)$	$ROE\,(Z)$	$D/E\,(Z)$	**Distance**	**Class**
S_1	-1.351	1.542	-1.184	1.487	Buy
S_2	-0.787	0.806	-0.858	0.527	Buy
S_3	-0.129	-0.299	-0.204	0.939	Hold
S_4	0.341	-0.668	0.286	1.266	Hold
S_5	1.750	-1.404	1.757	3.760	Avoid
S_6	1.093	-1.036	1.267	2.874	Avoid
S_7	-1.069	1.174	-1.021	1.002	Buy
S_8	0.153	-0.115	-0.041	1.053	Hold

Step 2: Sorted Euclidean distance table:

Stock	Distance	Class
S_2	0.527	Buy
S_3	0.939	Hold
S_7	1.002	Buy
S_8	1.053	Hold
S_4	1.266	Hold
S_1	1.487	Buy
S_6	2.874	Avoid
S_5	3.760	Avoid

Step 3: Majority voting for $k = 3$:

$$\{k = 3\} = \{S_2, S_3, S_7\}$$

- Buy: 2 votes (S_2, S_7)
- Hold: 1 vote (S_3)
- Avoid: 0 votes

$\therefore$ The new point is to be classified as **Buy** for $k = 3$.

2.3.3 Bank Loan Approval via Manhattan Distance

Example 2.3.3: Bank Loan Approval Decision

A bank utilizes KNN to decide whether to **Approve** or **Reject** a loan application based on two features:

- Credit score (scaled by dividing by 100, such that $750 \implies 7.5$)
- Annual Income (in Rs. Lakhs)

Loan Applicant Training Dataset

Applicant	Credit Score	Annual Income (Rs. Lakhs)	Decision
A_1	7.5	8.0	Approve
A_2	6.8	6.0	Approve
A_3	6.0	4.0	Reject
A_4	5.5	3.0	Reject
A_5	7.2	7.0	Approve
A_6	5.8	3.5	Reject

Task: Predict the decision for a new applicant with Credit Score = 6.5 and Annual Income = 5.5 Lakhs, i.e., $\mathbf{X} = (6.5, 5.5)$, using $k = 3$ and Manhattan distance.

Solution:

Step 1: Compute Manhattan distances from query point $\mathbf{X}(6.5, 5.5)$:

$$\begin{aligned}
\text{To } A_1(7.5, 8.0): &\quad d = |6.5 - 7.5| + |5.5 - 8.0| = 1.0 + 2.5 = 3.5 \\
\text{To } A_2(6.8, 6.0): &\quad d = |6.5 - 6.8| + |5.5 - 6.0| = 0.3 + 0.5 = 0.8 \\
\text{To } A_3(6.0, 4.0): &\quad d = |6.5 - 6.0| + |5.5 - 4.0| = 0.5 + 1.5 = 2.0 \\
\text{To } A_4(5.5, 3.0): &\quad d = |6.5 - 5.5| + |5.5 - 3.0| = 1.0 + 2.5 = 3.5 \\
\text{To } A_5(7.2, 7.0): &\quad d = |6.5 - 7.2| + |5.5 - 7.0| = 0.7 + 1.5 = 2.2 \\
\text{To } A_6(5.8, 3.5): &\quad d = |6.5 - 5.8| + |5.5 - 3.5| = 0.7 + 2.0 = 2.7
\end{aligned}$$

Step 2: Sorted Manhattan distance ranking:

Applicant	Distance	Decision
A_2	0.8	Approve
A_3	2.0	Reject
A_5	2.2	Approve
A_6	2.7	Reject
A_4	3.5	Reject
A_1	3.5	Approve

Step 3: Majority vote for $k = 3$:

$$\{k = 3\} = \{A_2, A_3, A_5\}$$

- Approve: 2 votes (A_2, A_5)
- Reject: 1 vote (A_3)

$\therefore$ The new applicant $\mathbf{X} = (6.5, 5.5)$ is to be concluded as **Approve** when $k = 3$.

2.3.4 Real Estate House Price Prediction (KNN Regression)

Example 2.3.4: Real Estate Price Estimation (KNN Regression)

A real estate agency uses KNN regression to estimate house prices based on:

- Area (in hundreds of sq. ft., e.g., 1400 sq. ft. $\implies$ 14)
- Age (in years)

Real Estate Housing Dataset

House	Area (hundred sq. ft.)	Age (years)	Price (Rs. Lakhs)
H_1	12	5	50
H_2	15	3	65
H_3	8	10	30
H_4	20	2	90
H_5	10	8	38
H_6	18	6	75

Task: Predict the price of a new house with Area = 14 (1400 sq. ft.) and Age = 4 years, i.e., $\mathbf{X} = (14, 4)$, using $k = 3$ and Manhattan distance.

Solution:

Step 1: Compute Manhattan distances from query house $\mathbf{X}(14, 4)$:

$$\begin{aligned}
\text{To } H_1(12,5): &\quad d = |14-12| + |4-5| = 2+1 = 3 \\
\text{To } H_2(15,3): &\quad d = |14-15| + |4-3| = 1+1 = 2 \\
\text{To } H_3(8,10): &\quad d = |14-8| + |4-10| = 6+6 = 12 \\
\text{To } H_4(20,2): &\quad d = |14-20| + |4-2| = 6+2 = 8 \\
\text{To } H_5(10,8): &\quad d = |14-10| + |4-8| = 4+4 = 8 \\
\text{To } H_6(18,6): &\quad d = |14-18| + |4-6| = 4+2 = 6
\end{aligned}$$

Step 2: Sorted Manhattan distance ranking:

House	Distance	Price (Rs. Lakhs)
H_2	2	65
H_1	3	50
H_6	6	75
H_4	8	90
H_5	8	38
H_3	12	30

Step 3: Average prices across $k = 3$ nearest neighbors:

$$\{k = 3\} = \{H_2, H_1, H_6\}$$

$$\hat{y} = \frac{65 + 50 + 75}{3} = \frac{190}{3} \approx 63.33 \text{ Lakhs}$$

$\therefore$ The new point $\mathbf{X} = (14, 4)$ is predicted to have a price of **Rs. 63.33 Lakhs** for $k = 3$.

2.4 Parameter Selection: Small k vs. Large k

Selecting the hyperparameter k balances local sensitivity against global regularization:

Table 2.2: Comparison of Small k versus Large k Characteristics

Aspect	**Small k**	**Large k**
Neighbourhood	Local	Broad
Sensitivity to Noise	High	Lower
Variance	Higher	Lower
Bias	Lower	Higher
Decision Boundary	Irregular / complex	Smoother
Risk	Overfitting	Underfitting

Guiding Principle for Choosing k

Choose k to balance bias and variance.

Chapter 3

Naive Bayes Classification

3.1 Probability Foundations & Notations

The Naive Bayes classifier is grounded in classical axiomatic probability theory and conditional distributions.

Table 3.1: Probability Notations and Terminology

Notation	Meaning
$P(A)$	Probability of event A
$P(A^c)$	Probability of complement of A
$P(A \cap B)$	Joint probability of A and B occurring simultaneously
$P(A \cup B)$	Probability of either event A or event B occurring
$P(A \mid B)$	Conditional probability of event A given event B has occurred
$P(A, B)$	Joint probability of events A and B (alternative standard notation)

3.1.1 Key Probability Identities

- **Complement Rule**:

$$P(A^c) = 1 - P(A) \tag{3.1}$$

- **Addition Rule (Union)**:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \tag{3.2}$$

- **Joint Probability Equivalence**:

$$P(A \cap B) = P(A, B) \tag{3.3}$$

3.1.2 Conditional Probability & Multiplication Rule

Definition 3.1.1: Conditional Probability

For two events A and B with $P(B) > 0$:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \tag{3.4}$$

Re-arranging yields the general **Multiplication Rule**:

$$P(A, B) = P(A \mid B)P(B) \tag{3.5}$$
$$P(A, B) = P(B \mid A)P(A) \tag{3.6}$$

3.1.3 Statistical Independence

If events A and B are statistically independent:

$$P(A \cap B) = P(A) \cdot P(B) \tag{3.7}$$

and equivalently:

$$P(A \mid B) = P(A) \tag{3.8}$$

3.2 Bayes' Theorem

Suppose an analyst wishes to determine the probability that an observation belongs to class C_k after observing feature evidence vector $\mathbf{X}$:

Theorem 3.2.1: Bayes' Theorem

$$P(C_k \mid \mathbf{X}) = \frac{P(\mathbf{X} \mid C_k)P(C_k)}{P(\mathbf{X})} \tag{3.9}$$

In conceptual terms:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}} \tag{3.10}$$

3.3 Worked Numerical Example: Credit Risk Classification

Example 3.3.1: Credit Risk Classification

A retail lending institution assesses credit risk with two target states:

- C_1: Good Credit
- C_2: Default

Historical portfolio priors and likelihoods for a financial indicator vector $\mathbf{X}$ are given by:

$$P(C_1) = 0.80, \quad P(C_2) = 0.20$$
$$P(\mathbf{X} \mid C_1) = 0.10, \quad P(\mathbf{X} \mid C_2) = 0.60$$

Task: Compute the evidence $P(\mathbf{X})$, determine the posterior probabilities $P(C_1 \mid \mathbf{X})$ and $P(C_2 \mid \mathbf{X})$, and classify the borrower.

Solution:
Step 1: Compute Total Evidence $P(\mathbf{X})$ via Law of Total Probability:

$$\begin{aligned} P(\mathbf{X}) &= P(\mathbf{X} \mid C_1)P(C_1) + P(\mathbf{X} \mid C_2)P(C_2) \\ &= (0.10 \times 0.80) + (0.60 \times 0.20) \\ &= 0.08 + 0.12 \\ &= 0.20 \end{aligned}$$

Step 2: Calculate Posterior Probabilities:

$$P(C_1 \mid \mathbf{X}) = \frac{P(\mathbf{X} \mid C_1)P(C_1)}{P(\mathbf{X})} = \frac{0.10 \times 0.80}{0.20} = \frac{0.08}{0.20} = 0.40$$

$$P(C_2 \mid \mathbf{X}) = \frac{P(\mathbf{X} \mid C_2)P(C_2)}{P(\mathbf{X})} = \frac{0.60 \times 0.20}{0.20} = \frac{0.12}{0.20} = 0.60$$

Step 3: Comparison and Decision:

$$P(C_2 \mid \mathbf{X}) > P(C_1 \mid \mathbf{X}) \quad (0.60 > 0.40)$$

$$\therefore \text{Classify } \mathbf{X} \text{ as } \textbf{Default } (C_2).$$

3.4 The Bayes Classifier Framework

3.4.1 Marginal Evidence Across K Classes

For a classification problem with K discrete classes $C_1, C_2, \ldots, C_K$, the overall marginal probability of evidence $\mathbf{X}$ is:

$$P(\mathbf{X}) = \sum_{k=1}^{K} P(\mathbf{X} \mid C_k)P(C_k) \tag{3.11}$$

3.4.2 Optimal Bayes Decision Rule

The Bayes classifier assigns an observation $\mathbf{X}$ to the class maximizing the posterior probability:

$$\hat{c} = \arg\max_{C_k} P(C_k \mid \mathbf{X}) = \arg\max_{C_k} \frac{P(\mathbf{X} \mid C_k)P(C_k)}{P(\mathbf{X})} \tag{3.12}$$

Because the evidence denominator $P(\mathbf{X})$ is identical across all classes C_k, the decision rule simplifies to:

$$\hat{c} = \arg\max_{C_k} P(\mathbf{X} \mid C_k)P(C_k) \tag{3.13}$$

3.5 The Naive Bayes Classifier

3.5.1 The Challenge of High-Dimensional Likelihoods

When an observation is characterized by p features $\mathbf{X} = (x_1, x_2, \ldots, x_p)$:

$$P(C_k \mid x_1, x_2, \ldots, x_p) = \frac{P(x_1, x_2, \ldots, x_p \mid C_k)P(C_k)}{P(x_1, x_2, \ldots, x_p)} \tag{3.14}$$

Directly estimating the joint multivariate conditional distribution $P(x_1, x_2, \ldots, x_p \mid C_k)$ from finite sample sizes is computationally and statistically challenging due to exponential data sparsity.

3.5.2 The Naive Conditional Independence Assumption

Definition 3.5.1: The Naive Assumption

All input features $x_1, x_2, \ldots, x_p$ are mutually conditionally independent given the class label C_k:

$$P(x_1, x_2, \ldots, x_p \mid C_k) = \prod_{j=1}^{p} P(x_j \mid C_k) \tag{3.15}$$

that is:

$$P(x_1, x_2, \ldots, x_p \mid C_k) = P(x_1 \mid C_k) \cdot P(x_2 \mid C_k) \cdots P(x_p \mid C_k) \tag{3.16}$$

3.5.3 Naive Bayes Classification Rule

Applying the independence assumption produces the core operational classification formula:

$$\hat{c} = \arg\max_{C_k} P(C_k) \prod_{j=1}^{p} P(x_j \mid C_k) \tag{3.17}$$

Why Is It Termed "Naive"?

The assumption is termed "naive" because conditional independence among predictors is rarely strictly true in real-world scenarios.
Financial Example: In credit scoring, variables such as *Income*, *Debt*, and *Employment Status* clearly exhibit mutual dependencies. However, in practice, the naive approximation dramatically simplifies computation while delivering strong empirical classification performance.

3.5.4 Algorithmic Step-by-Step Sequence

Naive Bayes Execution Procedure

i) **Start with Training Data**: Assemble feature vectors $\mathbf{X}_i$ and class labels y_i.

ii) **Calculate Prior Probabilities**: Compute class base rates $P(C_k) = \frac{N_{C_k}}{N}$.

iii) **Calculate Univariate Likelihoods**: Estimate 1-D conditional probabilities $P(x_j \mid C_k)$ for each feature given each class.

iv) **Apply Naive Assumption**: Decompose joint likelihood into the product:

$$P(x_1, \ldots, x_p \mid C_k) = \prod_{j=1}^{p} P(x_j \mid C_k)$$

v) **Calculate Class Score**: Evaluate the unnormalized joint score for each candidate class:

$$\text{Score}(C_k) = P(C_k) \prod_{j=1}^{p} P(x_j \mid C_k)$$

vi) **Compare Scores Across All Classes**: Evaluate $\text{Score}(C_k)$ across $k = 1, \ldots, K$.

vii) **Predict Output Class**:

$$\hat{c} = \arg\max_{C_k} \text{Score}(C_k)$$

Chapter 4

Logistic Regression

4.1 Introduction and Core Motivation

Logistic regression is a fundamental supervised learning algorithm utilized for classification problems. Unlike regression models that predict a continuous numerical quantity, logistic regression models the conditional probability that a given observation belongs to a particular class.

4.1.1 Why Not Standard Linear Regression?

In ordinary linear regression, the relationship between features and the target is modeled as:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon \tag{4.1}$$

Under this linear formulation, the predicted value $\hat{Y}$ can take any real value across the entire unbounded interval:

$$\hat{Y} \in (-\infty, +\infty) \tag{4.2}$$

However, probabilities are mathematically constrained to lie strictly within the unit interval:

$$P(Y = 1 \mid X) \in [0, 1] \tag{4.3}$$

If ordinary linear regression were used for classification, predictions could yield nonsensical probabilities less than 0 or greater than 1. Consequently, a transformation is needed that "squashes" the unbounded linear output $(-\infty, +\infty)$ into the valid probability range $[0, 1]$. In logistic regression, this transformation is provided by the **sigmoid (logistic) function**.

4.2 The Sigmoid (Logistic) Function and Probability Modeling

Logistic regression first computes a linear combination of the input features:

$$Z = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p \tag{4.4}$$

The real-valued score Z is then passed through the sigmoid (logistic) activation function to yield the predicted probability of the positive class ($Y = 1$):

$$P(Y = 1) = \frac{1}{1 + e^{-Z}} \tag{4.5}$$

4.2.1 Algebraic Equivalence of the Sigmoid Function

Multiplying both the numerator and the denominator by e^Z (since $\frac{e^Z}{e^Z} = 1$) provides the alternate standard representation:

$$P(Y=1) = \frac{1 \cdot e^Z}{(1+e^{-Z}) \cdot e^Z} \tag{4.6}$$

$$= \frac{e^Z}{e^Z + e^{-Z} \cdot e^Z} \tag{4.7}$$

$$= \frac{e^Z}{e^Z + e^{Z-Z}} \tag{4.8}$$

$$= \frac{e^Z}{e^Z + e^0} \tag{4.9}$$

$$= \frac{e^Z}{e^Z + 1} = \frac{e^Z}{1+e^Z} \tag{4.10}$$

Substituting the linear combination $Z = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$ directly yields:

$$P(Y=1) = \frac{e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}{1+e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}} \tag{4.11}$$

4.2.2 Properties of the Sigmoid Curve and Decision Threshold

The sigmoid curve possesses distinctive geometric characteristics:

- **S-Shaped Geometry:** The curve exhibits a smooth, monotonic S-shape.
- **Asymptotic Bounds:** As $Z \to -\infty$, $P(Y=1) \to 0$. As $Z \to +\infty$, $P(Y=1) \to 1$.
- **Symmetric Midpoint:** At $Z=0$, $e^0 = 1$, yielding exactly $P(Y=1) = \frac{1}{1+1} = 0.5$.

To convert continuous probability estimates into discrete class labels, a **decision threshold** is established (conventionally set to 0.5):

$$\hat{Y} = \begin{cases} 1, & \text{if } P(Y=1) \geq 0.5 \quad (Z \geq 0) \\ 0, & \text{if } P(Y=1) < 0.5 \quad (Z < 0) \end{cases} \tag{4.12}$$

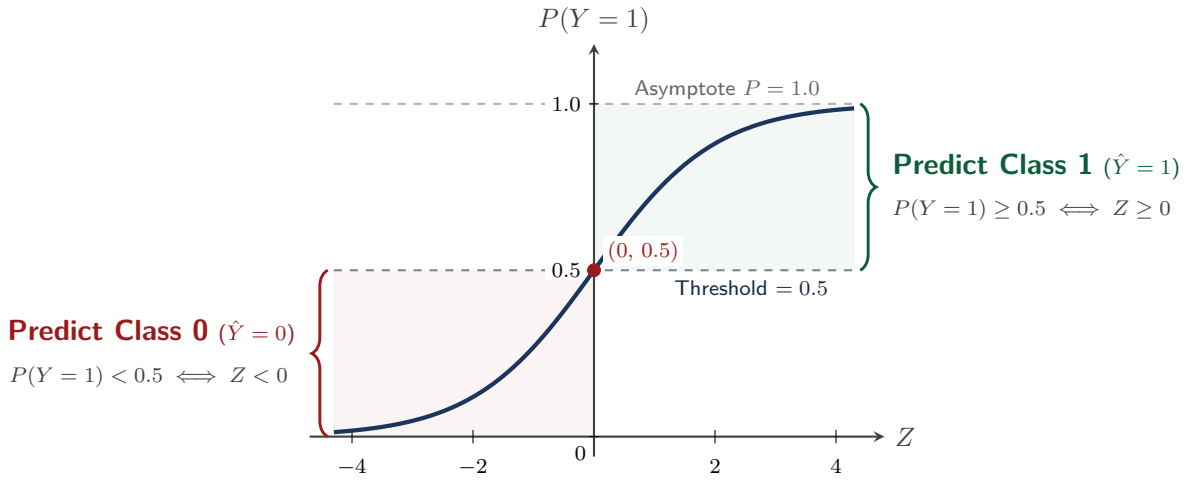


Figure 4.1: The Sigmoid activation function squashing unbounded linear values Z into valid class probabilities $[0,1]$ with a decision boundary at $Z=0$ ($P=0.5$).

4.3 Key Terms and Matrix Notation

Definition 4.3.1: Core Variables of Logistic Regression

- **Independent Variables (X):** The set of input features or predictor variables $X_1, X_2, \ldots, X_p$.
- **Dependent Variable (Y):** The categorical target outcome, typically binary $Y \in \{0, 1\}$.
- **Logistic Function (Sigmoid Function):** The nonlinear squashing function $\sigma(Z) = \frac{1}{1+e^{-Z}}$ mapping real scores to $(0, 1)$.
- **Coefficient (β_j):** The parameter estimated by the model for predictor X_j, indicating how strongly X_j affects the dependent variable on the log-odds scale.
- **Intercept (β_0):** The constant baseline term representing the log-odds when all independent variables equal zero ($X_1 = X_2 = \cdots = X_p = 0$).

4.3.1 Matrix Representation

In compact linear algebra notation, the dataset of n observations and p features is represented as an $n \times p$ feature matrix X:

$$X = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1p} \\ x_{21} & x_{22} & \ldots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \ldots & x_{np} \end{bmatrix} \tag{4.13}$$

where:

- n is the total number of observations (samples).
- p is the total number of features (predictors).

The observed class labels form a binary response vector $y \in \{0, 1\}^n$:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad y_i \in \{0, 1\} \tag{4.14}$$

4.4 Odds, Log-Odds (Logit), and Parameter Interpretation

4.4.1 Definition of Odds

The **odds** of an event represent the ratio of the probability that the event occurs to the probability that the event does not occur:

$$\text{Odds} = \frac{P(Y = 1)}{1 - P(Y = 1)} \tag{4.15}$$

Example 4.4.1: Calculating Probabilities and Odds

Consider a basket containing 3 apples and 5 oranges (total items $= 3 + 5 = 8$).

1. **Probability of selecting an Orange:**

$$P(\text{Orange}) = \frac{5}{8} = 0.625 \tag{4.16}$$

2. **Odds of selecting an Orange:**

$$\text{Odds(Orange)} = \frac{\text{Number of Oranges}}{\text{Number of Not Oranges (Apples)}} = \frac{5}{3} \approx 1.667 \tag{4.17}$$

3. **Odds of selecting an Apple:**

$$\text{Odds(Apple)} = \frac{\text{Number of Apples}}{\text{Number of Not Apples (Oranges)}} = \frac{3}{5} = 0.600 \tag{4.18}$$

4.4.2 Log-Odds (Logit Function)

Taking the natural logarithm of the odds produces the **log-odds** (or **logit**):

$$\ln(\text{Odds}) = \ln\left(\frac{P(Y=1)}{1-P(Y=1)}\right) \tag{4.19}$$

Substituting $P(Y=1) = \frac{e^Z}{1+e^Z}$ and $1 - P(Y=1) = \frac{1}{1+e^Z}$:

$$\frac{P(Y=1)}{1-P(Y=1)} = \frac{\frac{e^Z}{1+e^Z}}{\frac{1}{1+e^Z}} = e^Z \tag{4.20}$$

Taking the natural logarithm:

$$\ln\left(\frac{P(Y=1)}{1-P(Y=1)}\right) = \ln(e^Z) = Z = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p \tag{4.21}$$

Thus, **logistic regression is a linear model for the log-odds**.

4.4.3 Interpreting Coefficients and Odds Ratios

For a one-unit increase in predictor X_j (holding all other variables constant), the log-odds changes by β_j:

$$\Delta \ln(\text{Odds}) = \beta_j \tag{4.22}$$

Exponentiating both sides yields the **Odds Ratio (OR)**:

$$\text{Odds Ratio} = e^{\beta_j} \tag{4.23}$$

The directional impact of β_j on the odds is summarized as follows:

- $\beta_j > 0 \implies e^{\beta_j} > 1$: The odds of the event increase.
- $\beta_j < 0 \implies e^{\beta_j} < 1$: The odds of the event decrease.
- $\beta_j = 0 \implies e^{\beta_j} = 1$: Predictor X_j has no effect on the odds.

Example 4.4.2: Odds Ratio Interpretation with $\beta_j = 0.7$

Suppose the estimated coefficient for a predictor X_j is $\beta_j = 0.7$.

$$\text{Odds Ratio} = e^{\beta_j} = e^{0.7} \approx 2.0137 \approx 2 \tag{4.24}$$

Interpretation: A one-unit increase in X_j roughly doubles the odds of the outcome event occurring ($Y = 1$).

4.5 Model Training via Maximum Likelihood Estimation (MLE)

Logistic regression estimates its parameters $\beta_0, \beta_1, \ldots, \beta_p$ using **Maximum Likelihood Estimation (MLE)**.

- **Objective of MLE:** Find the parameter vector β that maximizes the joint likelihood of observing the given sample data.
- **Lack of Closed-Form Solution:** Unlike ordinary least squares (OLS) regression where $\hat{\beta} = (X^T X)^{-1} X^T y$, there is **no closed-form analytical solution** for the MLE of logistic regression parameters.
- **Numerical Optimization:** Parameter estimates are solved numerically using iterative optimization algorithms (e.g., Newton-Raphson, Iteratively Reweighted Least Squares, or Gradient Ascent).

4.6 Assumptions of Logistic Regression

Valid inference and stable performance in logistic regression rely on six foundational assumptions:

1. **Independent Observations:** Sample observations must be independent of one another (no repeated measures or auto-correlation).
2. **Binary Dependent Variable:** The response variable Y is binary (for standard binomial logistic regression).
3. **Linearity in Log-Odds:** Independent variables exhibit a strictly linear relationship with the log-odds $\ln\left(\frac{P}{1-P}\right)$.
4. **Absence of Extreme Outliers:** The predictor space should not contain extreme leverage points or anomalous outliers.
5. **Adequately Large Sample Size:** Because MLE relies on large-sample asymptotic properties, sufficient sample size is required to prevent biased parameter estimates.
6. **Low Multicollinearity:** Predictor variables should exhibit little to no multicollinearity. High correlation among predictors inflates parameter standard errors.

4.7 Variants, Metrics, and Trade-Offs

4.7.1 Types of Logistic Regression

Depending on the nature of the target variable, logistic regression is categorized into three types:

- **Binomial Logistic Regression:** The target has exactly two classes (e.g., Yes / No, Default / Non-Default).
- **Multinomial Logistic Regression:** The target has three or more unordered nominal classes (e.g., Cat, Dog, Sheep).
- **Ordinal Logistic Regression:** The target has three or more ordered categories (e.g., Low, Medium, High).

4.7.2 Model Evaluation Metrics

Model performance for logistic regression is assessed using standard classification metrics:

- **Accuracy:** Ratio of correctly classified instances to total instances.
- **Precision:** Ratio of true positives to all predicted positives ($\frac{\text{TP}}{\text{TP+FP}}$).
- **Recall (Sensitivity):** Ratio of true positives to all actual positives ($\frac{\text{TP}}{\text{TP+FN}}$).
- **F1-Score:** Harmonic mean of precision and recall ($2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision+Recall}}$).
- **AUC-ROC:** Area Under the Receiver Operating Characteristic curve, measuring discrimination across all possible classification thresholds.

4.7.3 Advantages and Disadvantages

Advantages (Pros)	Disadvantages (Cons)
• Outputs well-calibrated probabilities, not just hard class labels. • Highly interpretable parameters via odds ratios (e^{β_j}). • Directly extends to multiclass classification problems.	• Interaction terms between features must be constructed manually. • Interpretations are multiplicative on odds, not linear on probabilities. • Complete separation prevents convergence (coefficients diverge to $\pm\infty$). • Requires regularization (e.g., L1/L2 penalties) or informative priors to handle separation.

4.8 End-to-End Workflow and Generalized Linear Model (GLM) Classification

4.8.1 Five-Step Implementation Pipeline

1. **Prepare Data:** Encode categorical predictors (e.g., one-hot or dummy encoding) and scale continuous features if regularization is utilized.
2. **Fit Model:** Estimate parameter vector β using Maximum Likelihood Estimation (MLE) via numerical optimization.
3. **Interpret Coefficients:** Exponentiate parameters to interpret predictor effects as odds ratios (e^{β_j}).
4. **Predict and Threshold:** Compute predicted probabilities $P(Y = 1) = \sigma(X\beta)$ and assign class labels based on a selected threshold (typically 0.5).
5. **Evaluate Model:** Measure performance using Accuracy, Precision, Recall, F1-Score, and AUC-ROC.

4.8.2 Membership in Generalized Linear Models (GLM)

Logistic regression is a member of the broader **Generalized Linear Models (GLM)** family. In GLM theory, a model is defined by three components:

1. **Random Component:** The dependent variable Y follows a Bernoulli (or Binomial) distribution: $Y \sim \text{Bernoulli}(p)$.
2. **Systematic Component:** A linear combination of predictors $Z = \beta_0 + \sum_{j=1}^{p} \beta_j X_j$.
3. **Link Function:** The **logit link function** $g(p) = \ln\left(\frac{p}{1-p}\right) = Z$, which maps the expected probability parameter from $[0, 1]$ to the unbounded real line $(-\infty, +\infty)$.

Chapter 5

Decision Trees

5.1 Overview and Core Concepts

A **Decision Tree** is a supervised machine learning model that predicts a target outcome by recursively splitting the data into smaller, progressively more homogeneous (pure) subsets based on feature values.

5.1.1 Structural Components of a Decision Tree

- **Root Node:** The topmost node of the tree, containing all training samples prior to any partitioning.
- **Decision Node (Internal Node):** A node that tests a specific feature condition to split data into child branches.
- **Branch:** A directed link representing the outcome of a decision test leading to a subsequent node.
- **Leaf Node (Terminal Node):** An end node carrying the final prediction (class label in classification or mean value in regression); it undergoes no further splitting.
- **Parent Node:** A node immediately preceding a split.
- **Child Node:** A node generated immediately as the outcome of a parent node split.
- **Tree Depth:** The maximum number of levels (edges) from the root node to the deepest leaf node.

5.1.2 General Recursive Splitting Algorithm

1. **Initialization:** Place all training observations at the root node.
2. **Check Termination:** If a stopping condition is met (e.g., node is pure, maximum depth reached, or insufficient samples), convert the node into a terminal leaf.
3. **Evaluate Candidate Splits:** For every available feature and potential split point, compute the impurity reduction across the subsets.
4. **Select Optimal Split:** Select the feature and threshold that yields the best split criterion:
    - Maximum Information Gain (ID3 algorithm),
    - Maximum Gain Ratio (C4.5 algorithm),
    - Maximum Gini Gain / Minimum Weighted Gini Impurity (CART algorithm), or

- Maximum Variance Reduction (for continuous regression trees).

5. **Recursive Partitioning:** Partition the sample observations into disjoint child subsets and recurse on each child node.

6. **Stopping Criteria:** Terminate tree growth when:
   - The node is completely pure (all samples share the same class),
   - The predefined maximum depth has been reached,
   - The minimum sample size required for a split is breached, or
   - No available candidate split yields a positive impurity improvement.

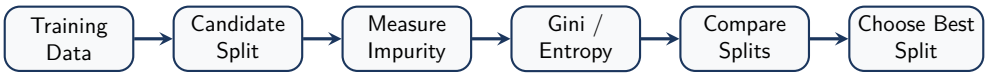


Figure 5.1: End-to-end split evaluation pipeline for constructing decision tree nodes.

5.2 Impurity Measures and Splitting Criteria

5.2.1 Standard Tree Induction Algorithms

Different decision tree induction algorithms utilize distinct impurity metrics and optimization objectives:

Algorithm	Impurity Measure	Split Criterion
ID3	Entropy	Information Gain
C4.5	Entropy	Gain Ratio
CART	Gini Impurity	Gini Gain / Minimum Weighted Gini

5.2.2 Entropy and Information Gain

Entropy quantifies the amount of uncertainty or disorder in a dataset S containing c classes:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i) \tag{5.1}$$

where $p_i = \frac{\text{Number of samples in class } i}{\text{Total observations in node}}$ is the empirical class probability.

For a two-class problem ($c = 2$ with probabilities p_1 and p_2):

$$H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2) \tag{5.2}$$

- A node containing only one class is completely pure: $H(S) = 0$.
- Maximum entropy occurs under equal class balance ($p_1 = p_2 = 0.5 \implies H(S) = 1.0$).

Information Gain (IG) measures the reduction in entropy achieved by partitioning a parent dataset S using attribute A:

$$IG(S, A) = H(\text{parent}) - H(\text{children}) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \tag{5.3}$$

where $H(\text{children}) = \sum_i \frac{n_i}{n} H_i$ represents the weighted average entropy across all resulting child partitions S_v of size n_i.

5.2.3 Gain Ratio (C4.5)

To overcome ID3's bias toward attributes with many distinct values, the **C4.5** algorithm normalizes Information Gain by the split information:

$$\text{Gain Ratio}(S, A) = \frac{IG(S, A)}{\text{Split Info}(S, A)} \tag{5.4}$$

where $\text{Split Info}(S, A) = -\sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$.

5.2.4 Gini Impurity (CART)

Gini Impurity measures the probability of misclassifying a randomly chosen element from the set if it were randomly labeled according to the class distribution:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2 \tag{5.5}$$

For a binary classification setting ($c = 2$):

$$\text{Gini}(S) = 1 - p_1^2 - p_2^2 \tag{5.6}$$

- **Smaller Gini:** Indicates a purer node.
- **Larger Gini:** Indicates a more mixed, impure node.
- $\text{Gini} = 0 \implies$ The node is completely pure (all samples belong to a single class).
- For binary classification, maximum Gini is 0.5 when $p_1 = p_2 = 0.5$.

5.2.5 Comparison: Gini Impurity vs. Entropy

Property	**Gini Impurity**	**Entropy**
Measures	Impurity	Uncertainty
Minimum Value	0	0
Pure Node Value	0	0
Binary Maximum	0.5 (at $p = 0.5$)	1.0 (at $p = 0.5$)
Good Split Condition	Lower Weighted Gini	Higher Information Gain
Computation	Simpler (algebraic squares)	Heavy (evaluates logarithms)
Common Usage	CART	ID3 / C4.5 algorithms

5.2.6 Advantages and Disadvantages of Decision Trees

Advantages	**Disadvantages**
• Highly intuitive and easy to understand. • Easy to visualize graphically. • Easy to explain to non-technical audiences. • Captures complex non-linear relationships. • Directly applicable to classification and regression.	• Highly susceptible to overfitting on training data. • Deep trees become unwieldy and overly complex. • High variance: small data perturbations alter the tree. • A single tree has limited predictive stability.

5.3 Solved Problem 5.1: Play Tennis Decision Tree

Example 5.3.1: ID3 Induction on Play Tennis Dataset

Construct an optimal decision tree using the ID3 algorithm (Entropy and Information Gain) on the classic 15-sample Play Tennis dataset.

Day	Outlook	Temp	Humidity	Wind	Tennis?
1	Sunny (S)	Hot (H)	High (H)	Weak (W)	No (N)
2	Sunny (S)	Hot (H)	High (H)	Strong (S)	No (N)
3	Overcast (O)	Hot (H)	High (H)	Weak (W)	Yes (Y)
4	Rain (R)	Mild (M)	High (H)	Weak (W)	Yes (Y)
5	Rain (R)	Cool (C)	Normal (N)	Weak (W)	Yes (Y)
6	Rain (R)	Cool (C)	Normal (N)	Strong (S)	No (N)
7	Overcast (O)	Cool (C)	Normal (N)	Strong (S)	Yes (Y)
8	Sunny (S)	Mild (M)	High (H)	Weak (W)	No (N)
9	Sunny (S)	Cool (C)	Normal (N)	Weak (W)	Yes (Y)
10	Rain (R)	Mild (M)	Normal (N)	Weak (W)	Yes (Y)
11	Sunny (S)	Mild (M)	Normal (N)	Strong (S)	Yes (Y)
12	Overcast (O)	Mild (M)	High (H)	Strong (S)	Yes (Y)
13	Overcast (O)	Hot (H)	Normal (N)	Weak (W)	Yes (Y)
14	Rain (R)	Mild (M)	High (H)	Strong (S)	No (N)
15	Sunny (S)	Hot (H)	High (H)	Weak (W)	No (N)

Overall Dataset Statistics and Entropy

The target variable contains 15 total observations: 9 Yes and 6 No:

$$p_1 = P(Y) = \frac{9}{15} = 0.60, \quad p_2 = P(N) = \frac{6}{15} = 0.40 \tag{5.7}$$

$$H(S) = -0.6\log_2(0.6) - 0.4\log_2(0.4) = 0.97095 \approx 0.9709 \tag{5.8}$$

Root Attribute Evaluation

We calculate the weighted entropy $H(\text{children}) = \sum \frac{n_i}{n} H_i$ for each candidate predictor:

1. Outlook ($n = 15$):

Value	Count (n_i)	Yes (Y)	No (N)	Entropy (H_i)	Weighted Entropy
Sunny (S)	6	2	4	0.9182	$\frac{6}{15} \times 0.9182 = 0.3673$
Overcast (O)	4	4	0	0.0000	$\frac{4}{15} \times 0 = 0.0000$
Rain (R)	5	3	2	0.9709	$\frac{5}{15} \times 0.9709 = 0.3236$
Total	15	9	6		**0.6909**

2. Temperature ($n = 15$):

Value	Count (n_i)	Yes (Y)	No (N)	Entropy (H_i)	Weighted Entropy
Hot (H)	5	2	3	0.9709	$\frac{5}{15} \times 0.9709 = 0.3236$
Mild (M)	6	4	2	0.9182	$\frac{6}{15} \times 0.9182 = 0.3673$
Cool (C)	4	3	1	0.8112	$\frac{4}{15} \times 0.8112 = 0.2163$
Total	15	9	6		**0.9072**

3. Humidity ($n = 15$):

Value	Count (n_i)	Yes (Y)	No (N)	Entropy (H_i)	Weighted Entropy
High (H)	8	3	5	0.9544	$\frac{8}{15} \times 0.9544 = 0.5090$
Normal (N)	7	6	1	0.5916	$\frac{7}{15} \times 0.5916 = 0.2761$
Total	15	9	6		**0.7851**

4. Wind ($n = 15$):

Value	Count (n_i)	Yes (Y)	No (N)	Entropy (H_i)	Weighted Entropy
Weak (W)	9	6	3	0.9182	$\frac{9}{15} \times 0.9182 = 0.5509$
Strong (S)	6	3	3	1.0000	$\frac{6}{15} \times 1.0000 = 0.4000$
Total	15	9	6		**0.7672**

Root Information Gain Comparison

Attribute	Information Gain Computation	Gain Value
Outlook	$0.9709 - 0.69088$	**0.28002**
Wind	$1.0000 - 0.76720$ (or $0.9709 - 0.7672 = 0.2037$)	0.23280
Humidity	$0.9544 - 0.78500$ (or $0.9709 - 0.7850 = 0.1859$)	0.16940
Temperature	$0.9709 - 0.90720$	0.06370

Ranking: Outlook > Wind > Humidity > Temp. Therefore, **Outlook** is selected as the **Root Node**.

- When Outlook = Overcast: Count = 4, all 4 Yes $\implies$ **Pure Leaf: Predict Yes**.

Partitioning the Rain Branch ($n = 5$)

The Rain subset contains 5 observations (Days 4, 5, 6, 10, 14) with 3 Yes and 2 No ($H(S_{\text{Rain}}) = 0.9709$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Temperature					
Mild	3	2	1	0.9182	$\frac{3}{5} \times 0.9182 = 0.5509$
Cool	2	1	1	1.0000	$\frac{2}{5} \times 1.0000 = 0.4000$
Weighted Sum					**0.9509**
Humidity					
High	2	1	1	1.0000	$\frac{2}{5} \times 1.0000 = 0.4000$
Normal	3	2	1	0.9182	$\frac{3}{5} \times 0.9182 = 0.55092$
Weighted Sum					**0.95092**
Wind					
Weak	3	3	0	0	$\frac{3}{5} \times 0 = 0$
Strong	2	0	2	0	$\frac{2}{5} \times 0 = 0$
Weighted Sum					**0**

Information Gain under Rain:

$$IG(\text{Temperature}) = 0.9709 - 0.9509 = 0.0200$$
$$IG(\text{Humidity}) = 0.9709 - 0.95092 = 0.01998$$
$$IG(\text{Wind}) = 0.9709 - 0.0000 = \mathbf{0.9709}$$

Ranking: Wind > Temp > Humidity. Under Rain, we split on **Wind**:

- Wind = Weak $\implies$ **Yes**
- Wind = Strong $\implies$ **No**

Partitioning the Sunny Branch ($n = 6$)

The Sunny subset contains 6 observations (Days 1, 2, 8, 9, 11, 15) with 2 Yes and 4 No ($H(S_{\text{Sunny}}) = 0.9182$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Temperature					
Hot	3	0	3	0	$\frac{3}{6} \times 0 = 0$
Mild	2	1	1	1.0000	$\frac{2}{6} \times 1.0000 = 0.3333$
Cool	1	1	0	0	$\frac{1}{6} \times 0 = 0$
Weighted Sum					**0.3333**
Humidity					
High	4	0	4	0	$\frac{4}{6} \times 0 = 0$
Normal	2	2	0	0	$\frac{2}{6} \times 0 = 0$
Weighted Sum					**0**
Wind					
Weak	4	1	3	0 (or 0.811)	$\frac{4}{6} \times 0 = 0$
Strong	2	1	1	1.0000	$\frac{2}{6} \times 1.0000 = 0.3333$
Weighted Sum					**0.3333**

Information Gain under Sunny:

$$IG(\text{Temperature}) = 0.9182 - 0.3333 = 0.5849$$
$$IG(\text{Humidity}) = 0.9182 - 0.0000 = \mathbf{0.9182}$$
$$IG(\text{Wind}) = 0.9182 - 0.3333 = 0.5849$$

Ranking: Humidity > Temp = Wind. Under Sunny, we split on **Humidity**:

- Humidity = High $\implies$ **No**
- Humidity = Normal $\implies$ **Yes**

Final Play Tennis Decision Tree

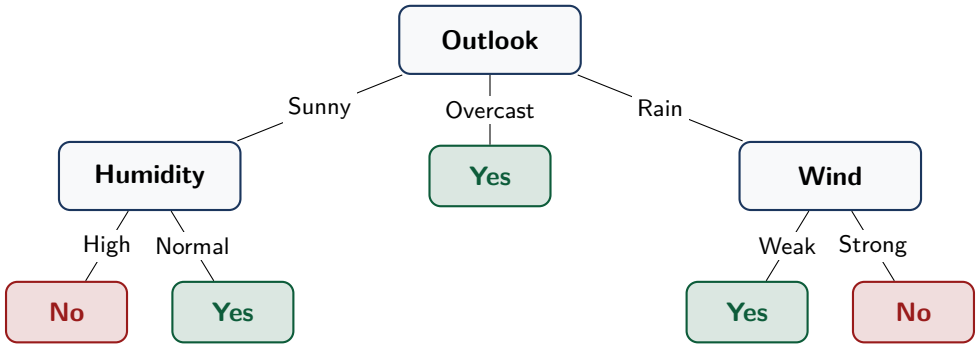


Figure 5.2: Final Decision Tree Structure for Play Tennis Classification.

5.4 Solved Problem 5.2: Credit Loan Approval Decision Tree

Example 5.4.1: Decision Tree Induction for Loan Approval

Given a dataset of 15 loan applicants across features Location, Income, Credit Score, and Employment, build the optimal decision tree using Entropy and Information Gain.

App	Location	Income	Credit Score	Employment	Approved?
1	Urban (U)	High (H)	Poor (P)	Part-time (PT)	No (N)
2	Urban (U)	High (H)	Poor (P)	Full-time (FT)	No (N)
3	Suburban (S)	High (H)	Poor (P)	Part-time (PT)	Yes (Y)
4	Rural (R)	Medium (M)	Poor (P)	Part-time (PT)	Yes (Y)
5	Rural (R)	Low (L)	Good (G)	Part-time (PT)	Yes (Y)
6	Rural (R)	Low (L)	Good (G)	Full-time (FT)	No (N)
7	Suburban (S)	Low (L)	Good (G)	Full-time (FT)	Yes (Y)
8	Urban (U)	Medium (M)	Poor (P)	Part-time (PT)	No (N)
9	Urban (U)	Low (L)	Good (G)	Part-time (PT)	Yes (Y)
10	Rural (R)	Medium (M)	Good (G)	Part-time (PT)	Yes (Y)
11	Urban (U)	Medium (M)	Good (G)	Full-time (FT)	Yes (Y)
12	Suburban (S)	Medium (M)	Poor (P)	Full-time (FT)	Yes (Y)
13	Suburban (S)	High (H)	Good (G)	Part-time (PT)	Yes (Y)
14	Rural (R)	Medium (M)	Poor (P)	Full-time (FT)	No (N)
15	Urban (U)	High (H)	Poor (P)	Part-time (PT)	No (N)

Overall Dataset Statistics and Entropy

The target variable Approved? contains 9 Yes and 6 No out of 15 applications:

$$P_Y = \frac{9}{15}, \quad P_N = \frac{6}{15} \tag{5.9}$$

$$H_{\text{Approved}} = -\left(\frac{9}{15}\right)\log_2\left(\frac{9}{15}\right) - \left(\frac{6}{15}\right)\log_2\left(\frac{6}{15}\right) = 0.9709 \tag{5.10}$$

Root Attribute Evaluation

1. Location ($n = 15$):

Location	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Urban	6	2	4	0.9182	$\frac{6}{15} \times 0.9182 = 0.36728$
Suburban	4	4	0	0	$\frac{4}{15} \times 0 = 0$
Rural	5	3	2	0.9709	$\frac{5}{15} \times 0.9709 = 0.32363$
Total	15	9	6		**0.6909**

2. Income ($n = 15$):

Income	Count	Yes	No	Entropy (H_i)	Weighted Entropy
High	5	2	3	0.9709	$\frac{5}{15} \times 0.9709 = 0.32363$
Medium	6	4	2	0.9182	$\frac{6}{15} \times 0.9182 = 0.36728$
Low	4	3	1	0.8112	$\frac{4}{15} \times 0.8112 = 0.21632$
Total	15	9	6		**0.9072**

3. Credit Score ($n = 15$):

Credit Score	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Good	7	6	1	0.5916	$\frac{7}{15} \times 0.5916 = 0.27608$
Poor	8	3	5	0.9544	$\frac{8}{15} \times 0.9544 = 0.50901$
Total	15	9	6		**0.7850**

4. Employment ($n = 15$):

Employment	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Full-time	6	3	3	1.0000	$\frac{6}{15} \times 1.0000 = 0.4000$
Part-time	9	6	3	0.9182	$\frac{9}{15} \times 0.9182 = 0.5509$
Total	15	9	6		**0.9509**

Root Information Gain Comparison

Root Attribute	Information Gain	Gain Value
Location	$0.9709 - 0.6909$	**0.2800**
Credit Score	$0.9709 - 0.7850$	0.1859
Income	$0.9709 - 0.9072$	0.0637
Employment	$0.9709 - 0.9509$	0.0200

Ranking: Location > Credit Score > Income > Employment. Thus, **Location** is selected as the **Root Node**.

- When Location = Suburban: Count = 4, all 4 Yes $\implies$ **Pure Leaf: Predict Yes**.

Partitioning the Urban Branch ($n = 6$)

The Urban subset contains 6 observations (Apps 1, 2, 8, 9, 11, 15) with 2 Yes and 4 No ($H(S_{\text{Urban}}) = 0.9182$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Income					
High	3	0	3	0	$\frac{3}{6} \times 0 = 0$
Medium	2	1	1	1.0000	$\frac{2}{6} \times 1 = 0.3333$
Low	1	1	0	0	$\frac{1}{6} \times 0 = 0$
Weighted Sum					**0.3333**
Credit Score					
Good	2	2	0	0	$\frac{2}{6} \times 0 = 0$
Poor	4	0	4	0	$\frac{4}{6} \times 0 = 0$
Weighted Sum					**0**
Employment					
Full-time	2	1	1	1.0000	$\frac{2}{6} \times 1 = 0.3333$
Part-time	4	1	3	0.8112	$\frac{4}{6} \times 0.8112 = 0.5408$
Weighted Sum					**0.8741**

Information Gain under Urban:

$$IG(\text{Income}) = 0.9182 - 0.3333 = 0.5849$$
$$IG(\text{Credit Score}) = 0.9182 - 0.0000 = \mathbf{0.9182}$$
$$IG(\text{Employment}) = 0.9182 - 0.8741 = 0.0441$$

Ranking: Credit Score > Income > Employment. Under Urban, we split on **Credit Score**:

- Credit Score = Good $\implies$ **Yes**
- Credit Score = Poor $\implies$ **No**

Partitioning the Rural Branch ($n = 5$)

The Rural subset contains 5 observations (Apps 4, 5, 6, 10, 14) with 3 Yes and 2 No ($H(S_{\text{Rural}}) = 0.9709$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Income					
Medium	3	2	1	0.9182	$\frac{3}{5} \times 0.9182 = 0.55092$
Low	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Weighted Sum					**0.95092**
Credit Score					
Good	3	2	1	0.8899	$\frac{3}{5} \times 0.8899 = 0.53394$
Poor	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Weighted Sum					**0.93394**
Employment					
Full-time	2	0	2	0	$\frac{2}{5} \times 0 = 0$
Part-time	3	3	0	0	$\frac{3}{5} \times 0 = 0$
Weighted Sum					**0**

Information Gain under Rural:

$$IG(\text{Income}) = 0.9709 - 0.95092 = 0.01998$$
$$IG(\text{Credit Score}) = 0.9709 - 0.93394 = 0.03696$$
$$IG(\text{Employment}) = 0.9709 - 0.0000 = \mathbf{0.9709}$$

Ranking: Employment > Credit Score > Income. Under Rural, we split on **Employment**:

- Employment = Full-time $\implies$ **No**
- Employment = Part-time $\implies$ **Yes**

Final Loan Approval Decision Tree

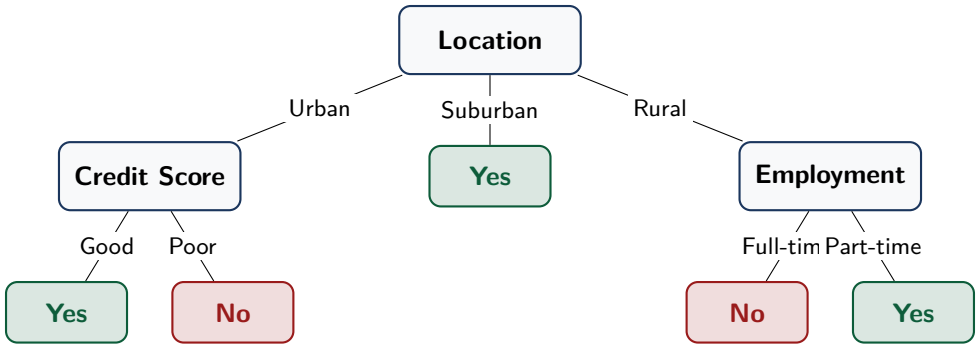


Figure 5.3: Final Decision Tree Structure for Credit Loan Approval.

5.5 Solved Problem 5.3: Candidate Job Offer Decision Tree

Example 5.5.1: Decision Tree Induction for Candidate Job Offers

A human resources department assesses 14 candidates across Experience, Salary Requirement, Certification Status, and Interview Rating to predict whether a Job Offer is extended.

Candidate	Experience	Salary	Certified	Rating	Job Offer
1	Junior (J)	High (H)	No (N)	Fair (F)	No (N)
2	Junior (J)	High (H)	No (N)	Excellent (E)	No (N)
3	Mid (M)	High (H)	No (N)	Fair (F)	Yes (Y)
4	Senior (S)	Medium (M)	No (N)	Fair (F)	Yes (Y)
5	Senior (S)	Low (L)	Yes (Y)	Fair (F)	Yes (Y)
6	Senior (S)	Low (L)	Yes (Y)	Excellent (E)	No (N)
7	Mid (M)	Low (L)	Yes (Y)	Excellent (E)	Yes (Y)
8	Junior (J)	Medium (M)	No (N)	Fair (F)	No (N)
9	Junior (J)	Low (L)	Yes (Y)	Fair (F)	Yes (Y)
10	Senior (S)	Medium (M)	Yes (Y)	Fair (F)	Yes (Y)
11	Junior (J)	Medium (M)	Yes (Y)	Excellent (E)	Yes (Y)
12	Mid (M)	Medium (M)	No (N)	Excellent (E)	Yes (Y)
13	Mid (M)	High (H)	Yes (Y)	Fair (F)	Yes (Y)
14	Senior (S)	Medium (M)	No (N)	Excellent (E)	No (N)

Overall Dataset Statistics and Entropy

The target variable Job Offer contains 9 Yes and 5 No out of 14 candidates:

$$P_{\text{Yes}} = \frac{9}{14}, \quad P_{\text{No}} = \frac{5}{14} \tag{5.11}$$

$$H_S = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right) = 0.9402 \tag{5.12}$$

Root Attribute Evaluation

1. Experience ($n = 14$):

Experience	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Junior (J)	5	2	3	0.9709	$\frac{5}{14} \times 0.9709 = 0.34675$
Mid (M)	4	4	0	0	$\frac{4}{14} \times 0 = 0$
Senior (S)	5	3	2	0.9709	$\frac{5}{14} \times 0.9709 = 0.34675$
Total	14	9	5		**0.6935**

2. Salary ($n = 14$):

Salary	Count	Yes	No	Entropy (H_i)	Weighted Entropy
High (H)	4	2	2	1.0000	$\frac{4}{14} \times 1 = 0.2857$
Medium (M)	6	4	2	0.9182	$\frac{6}{14} \times 0.9182 = 0.3935$
Low (L)	4	3	1	0.8112	$\frac{4}{14} \times 0.8112 = 0.2318$
Total	14	9	5		**0.9110**

3. Certified ($n = 14$):

Certified	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Yes (Y)	7	6	1	0.5916	$\frac{7}{14} \times 0.5916 = 0.2958$
No (N)	7	3	4	0.9852	$\frac{7}{14} \times 0.9852 = 0.4926$
Total	14	9	5		**0.7884**

4. Rating ($n = 14$):

Rating	Count	Yes	No	Entropy (H_i)	Weighted Entropy
Fair (F)	8	6	2	0.8112	$\frac{8}{14} \times 0.8112 = 0.4635$
Excellent (E)	6	3	3	1.0000	$\frac{6}{14} \times 1 = 0.4286$
Total	14	9	5		**0.8921**

Root Information Gain Comparison

Root Attribute	Information Gain	Gain Value
Experience	$0.9402 - 0.6935$	**0.2467**
Certified	$0.9402 - 0.7884$	0.1518
Rating	$0.9402 - 0.8921$	0.0481
Salary	$0.9402 - 0.9110$	0.0292

Ranking: Experience > Certified > Rating > Salary. Thus, **Experience** is chosen as the **Root Node**.

- When Experience = Mid: Count = 4, all 4 Yes $\implies$ **Pure Leaf: Predict Yes**.

Partitioning the Junior Branch ($n = 5$)

The Junior subset contains 5 observations (Candidates 1, 2, 8, 9, 11) with 2 Yes and 3 No ($H(S_{\text{Junior}}) = 0.9709$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Certified					
Yes	2	2	0	0	$\frac{2}{5} \times 0 = 0$
No	3	0	3	0	$\frac{3}{5} \times 0 = 0$
Weighted Sum					**0**
Salary					
High	2	0	2	0	$\frac{2}{5} \times 0 = 0$
Medium	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Low	1	1	0	0	$\frac{1}{5} \times 0 = 0$
Weighted Sum					**0.4000**
Rating					
Fair	3	1	2	0.9182	$\frac{3}{5} \times 0.9182 = 0.55092$
Excellent	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Weighted Sum					**0.95092**

Information Gain under Junior:

$$IG(\text{Certified}) = 0.9709 - 0.0000 = \mathbf{0.9709}$$

$$IG(\text{Salary}) = 0.9709 - 0.4000 = 0.5709$$

$$IG(\text{Rating}) = 0.9709 - 0.95092 = 0.01998$$

Ranking: Certified > Salary > Rating. Under Junior, we split on **Certified**:

- Certified = Yes $\implies$ **Yes**
- Certified = No $\implies$ **No**

Partitioning the Senior Branch ($n = 5$)

The Senior subset contains 5 observations (Candidates 4, 5, 6, 10, 14) with 3 Yes and 2 No ($H(S_{\text{Senior}}) = 0.9709$):

Attribute & Value	Count	Yes	No	Entropy	Weighted Entropy
Certified					
Yes	3	2	1	0.9182	$\frac{3}{5} \times 0.9182 = 0.55092$
No	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Weighted Sum					**0.95092**
Salary					
Medium	3	2	1	0.9182	$\frac{3}{5} \times 0.9182 = 0.55092$
Low	2	1	1	1.0000	$\frac{2}{5} \times 1 = 0.4000$
Weighted Sum					**0.95092**
Rating					
Fair	3	3	0	0	$\frac{3}{5} \times 0 = 0$
Excellent	2	0	2	0	$\frac{2}{5} \times 0 = 0$
Weighted Sum					**0**

Information Gain under Senior:

$$IG(\text{Certified}) = 0.9709 - 0.95092 = 0.01998$$
$$IG(\text{Salary}) = 0.9709 - 0.95092 = 0.01998$$
$$IG(\text{Rating}) = 0.9709 - 0.0000 = \mathbf{0.9709}$$

Ranking: Rating > Certified = Salary. Under Senior, we split on **Rating**:

- Rating = Fair $\implies$ **Yes**
- Rating = Excellent $\implies$ **No**

Final Candidate Job Offer Decision Tree

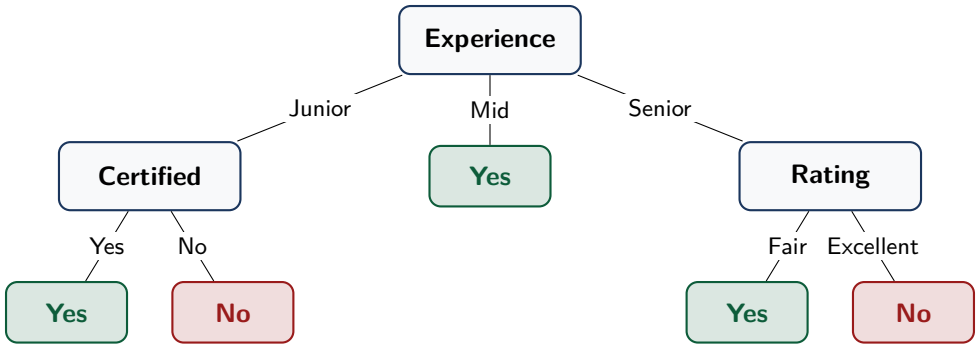


Figure 5.4: Final Decision Tree Structure for Candidate Job Offer Acceptance.

Chapter 6

Support Vector Machines

6.1 Core Principles and Mathematical Foundations

Support Vector Machine (SVM) is a powerful supervised learning algorithm primarily utilized for classification tasks, though it also extends to regression problems via Support Vector Regression (SVR). The objective of an SVM is to identify the optimal separating hyperplane that maximizes the geometric margin between distinct classes.

6.1.1 Fundamental Definitions and Geometry

- **Optimal Hyperplane:** A linear decision boundary that separates classes while maximizing the perpendicular distance to the closest training observations.
- **Support Vectors:** The subset of training data points that lie directly on the margin boundaries. These critical points alone determine the exact position and orientation of the separating hyperplane; all other training points can be removed or relocated without altering the solution.
- **Decision Boundary:** The separating hyperplane where the decision function evaluates to zero:
$$w^T x + b = 0 \tag{6.1}$$
- **Margin Boundaries:** The parallel hyperplanes that delimit the empty slab separating the two classes:
$$\text{Positive Margin Boundary:} \quad w^T x + b = +1 \tag{6.2}$$
$$\text{Negative Margin Boundary:} \quad w^T x + b = -1 \tag{6.3}$$
- **Margin Width (M):** The total geometric distance between the positive and negative margin boundaries:
$$M = \frac{2}{\|w\|} \tag{6.4}$$
where $\|w\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_p^2}$ is the Euclidean norm of the weight vector.
- **Perpendicular Distance of a Point:** The geometric distance of any arbitrary observation x_i to the decision boundary:
$$d_i = \frac{|w^T x_i + b|}{\|w\|} = \frac{|f(x_i)|}{\|w\|} \tag{6.5}$$

6.1.2 Functional Margin and Hard-Margin Condition

The **functional margin** of an observation (x_i, y_i) with label $y_i \in \{-1, +1\}$ is defined as:

$$\hat{\gamma}_i = y_i(w^T x_i + b) \tag{6.6}$$

For a correctly scaled canonical hard-margin SVM, all training observations must satisfy the strict separability constraint:

$$y_i(w^T x_i + b) \geq 1, \quad \forall i = 1, 2, \ldots, n \tag{6.7}$$

where equality $y_i(w^T x_i + b) = 1$ holds if and only if x_i is a support vector.

6.2 Boundary Identification and Analytical Solution Techniques

To solve linear SVM problems analytically for two-dimensional data, we identify whether the classes are separated by a horizontal, vertical, or diagonal boundary:

6.2.1 Horizontal Separation ($x_2 = c$)

A dataset exhibits horizontal separation if the second coordinate (x_2) of all points in one class is strictly greater than the second coordinate of all points in the opposite class:

$$\min\{x_2 \in \text{Class } +1\} > \max\{x_2 \in \text{Class } -1\} \tag{6.8}$$

- Let $p = \min\{x_2 \in \text{Class } +1\}$ denote the smallest x_2 value on the positive side.
- Let $n = \max\{x_2 \in \text{Class } -1\}$ denote the largest x_2 value on the negative side.
- The optimal maximum margin boundary passes through the midpoint:

$$c = \frac{p+n}{2} \implies x_2 = c \tag{6.9}$$

- Setting $w = [0, w_2]^T$, we solve the canonical equations using the closest boundary points:

$$w_2 p + b = 1 \tag{6.10}$$

$$w_2 n + b = -1 \tag{6.11}$$

  Subtracting the equations yields:

$$w_2(p - n) = 2 \implies w_2 = \frac{2}{p-n} \tag{6.12}$$

  Substituting w_2 into either equation determines the intercept b.

6.2.2 Vertical Separation ($x_1 = c$)

A dataset exhibits vertical separation if the first coordinate (x_1) of all points in one class is strictly greater than the first coordinate of all points in the opposite class:

$$\min\{x_1 \in \text{Class } +1\} > \max\{x_1 \in \text{Class } -1\} \tag{6.13}$$

- Let $p = \min\{x_1 \in \text{Class } +1\}$ and $n = \max\{x_1 \in \text{Class } -1\}$.
- The maximum margin boundary is given by:

$$c = \frac{p+n}{2} \implies x_1 = c \tag{6.14}$$

- Setting $w = [w_1, 0]^T$, we solve:

$$w_1 p + b = 1, \quad w_1 n + b = -1 \implies w_1 = \frac{2}{p-n} \tag{6.15}$$

6.2.3 Diagonal Separation ($x_2 - x_1 = c$ or $x_2 + x_1 = c$)

When neither coordinate alone separates the classes, evaluate linear combinations such as $x_2 - x_1$ or $x_2 + x_1$. If the resulting combined score is strictly positive for one class and strictly negative for the other, the optimal decision boundary is diagonal:

$$x_2 - x_1 = c \quad \text{or} \quad x_2 + x_1 = c \tag{6.16}$$

6.3 Solved Problem 6.1: Horizontal Decision Boundary

Example 6.3.1: SVM Induction for Horizontal Separation

Given a 12-point dataset partitioned into Class $+1$ and Class -1:

- **Class $+1$:** $(1,7)$, $(2,8)$, $(3,7)$, $(4,9)$, $(5,8)$, $(6,7)$
- **Class -1:** $(1,3)$, $(2,2)$, $(3,3)$, $(4,1)$, $(5,2)$, $(6,3)$

Determine the maximum margin decision boundary, compute the correctly scaled parameters w and b, identify the support vectors, verify the hard-margin condition, compute the margin width and sample distances, and classify a new query observation $(3,6)$.

Step 1: Identify Separating Orientation and Midpoint

Inspecting the x_2 coordinates:

$$p = \min\{x_2 \in \text{Class } +1\} = 7, \quad n = \max\{x_2 \in \text{Class } -1\} = 3 \tag{6.17}$$

Since all Class $+1$ points have $x_2 \geq 7$ and all Class -1 points have $x_2 \leq 3$, the classes are separated horizontally:

$$7 > c > 3 \tag{6.18}$$

The maximum margin decision boundary lies at the midpoint:

$$c = \frac{p+n}{2} = \frac{7+3}{2} = 5 \implies x_2 = 5 \tag{6.19}$$

Step 2: Solve Correctly Scaled w and b

Since the boundary is horizontal, x_1 has no effect, so $w = [0, w_2]^T$. The decision function is $f(x) = w_2 x_2 + b$. Setting up the margin boundary equations:

$$w_2(7) + b = 1 \tag{6.20}$$
$$w_2(3) + b = -1 \tag{6.21}$$

Subtracting Equation (6.21) from (6.20):

$$4w_2 = 2 \implies w_2 = 0.5 \tag{6.22}$$

Substituting $w_2 = 0.5$ into Equation (6.20):

$$0.5(7) + b = 1 \implies 3.5 + b = 1 \implies b = -2.5 \tag{6.23}$$

Thus, the canonical parameters are:

$$w = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}, \quad b = -2.5 \tag{6.24}$$

and the decision function is:

$$f(x) = 0 \cdot x_1 + 0.5x_2 - 2.5 = 0.5x_2 - 2.5 \tag{6.25}$$

Step 3: Identify Support Vectors

The support vectors are the data points satisfying $f(x) = \pm 1$:

- **Positive Support Vectors ($x_2 = 7$):** $(1, 7)$, $(3, 7)$, $(6, 7)$
- **Negative Support Vectors ($x_2 = 3$):** $(1, 3)$, $(3, 3)$, $(6, 3)$

Step 4: Verify Hard-Margin Constraint $y_i(w^T x_i + b) \geq 1$

Class $+1$ ($y_i = +1$)		**Class -1 ($y_i = -1$)**	
Point (x_1, x_2)	$y_i f(x)$	**Point** (x_1, x_2)	$y_i f(x)$
$(1, 7)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(1, 3)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$
$(2, 8)$	$+1 \cdot (0.5(8) - 2.5) = 1.5$	$(2, 2)$	$-1 \cdot (0.5(2) - 2.5) = 1.5$
$(3, 7)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(3, 3)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$
$(4, 9)$	$+1 \cdot (0.5(9) - 2.5) = 2.0$	$(4, 1)$	$-1 \cdot (0.5(1) - 2.5) = 2.0$
$(5, 8)$	$+1 \cdot (0.5(8) - 2.5) = 1.5$	$(5, 2)$	$-1 \cdot (0.5(2) - 2.5) = 1.5$
$(6, 7)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(6, 3)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$

All points satisfy $y_i(w^T x_i + b) \geq 1$ strictly, confirming hard-margin validity.

Step 5: Margin Width and Geometric Distances

$$\|w\| = \sqrt{0^2 + 0.5^2} = 0.5 \tag{6.26}$$

$$M = \frac{2}{\|w\|} = \frac{2}{0.5} = 4 \tag{6.27}$$

The perpendicular distance of each point to the boundary $x_2 = 5$ is $d_i = \frac{|0.5x_2 - 2.5|}{0.5} = |x_2 - 5|$:

Class $+1$		**Class -1**	
Point	**Distance** d_i	**Point**	**Distance** d_i
$(1, 7)$	2	$(1, 3)$	2
$(2, 8)$	3	$(2, 2)$	3
$(3, 7)$	2	$(3, 3)$	2
$(4, 9)$	4	$(4, 1)$	4
$(5, 8)$	3	$(5, 2)$	3
$(6, 7)$	2	$(6, 3)$	2

Step 6: Classify Query Observation $(3, 6)$

Evaluating the decision function at $x = (3, 6)$:

$$f(3, 6) = 0.5(6) - 2.5 = 3.0 - 2.5 = 0.5 \tag{6.28}$$

Since $f(3, 6) = 0.5 \geq 0$, the query point is assigned to **Class** $+1$:

$$\hat{y} = +1 \tag{6.29}$$

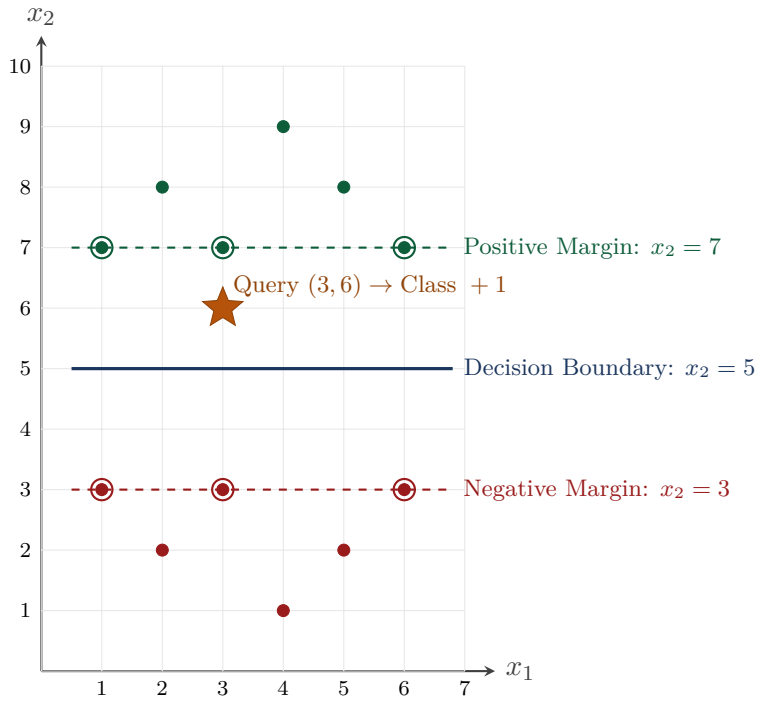


Figure 6.1: Horizontal SVM Maximum Margin Boundary at $x_2 = 5$ with Support Vectors and Query Observation $(3, 6)$.

6.4 Solved Problem 6.2: Vertical Decision Boundary

Example 6.4.1: SVM Induction for Vertical Separation

Given the 12-point dataset:

- **Class** $+1$**:** $(7, 1)$, $(8, 2)$, $(7, 3)$, $(9, 2)$, $(8, 1)$, $(7, 2)$
- **Class** -1**:** $(3, 1)$, $(2, 2)$, $(3, 3)$, $(1, 2)$, $(2, 1)$, $(3, 2)$

Determine the maximum margin decision boundary, solve for w and b, identify the support vectors, verify the hard-margin condition, compute the margin and distances, and classify the query point $(5, 2)$.

Step 1: Identify Separating Orientation and Midpoint

Inspecting the x_1 coordinates:

$$p = \min\{x_1 \in \text{Class } +1\} = 7, \quad n = \max\{x_1 \in \text{Class } -1\} = 3 \tag{6.30}$$

Since all Class $+1$ points have $x_1 \geq 7$ and all Class -1 points have $x_1 \leq 3$, the classes are separated vertically:

$$7 > c > 3 \implies c = \frac{7+3}{2} = 5 \implies x_1 = 5 \tag{6.31}$$

Step 2: Solve Correctly Scaled w and b

Since the boundary is vertical, $w = [w_1, 0]^T$ and $f(x) = w_1 x_1 + b$:

$$w_1(7) + b = 1 \tag{6.32}$$
$$w_1(3) + b = -1 \tag{6.33}$$

Subtracting yields $4w_1 = 2 \implies w_1 = 0.5$, and substituting gives $0.5(7) + b = 1 \implies b = -2.5$.

$$w = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix}, \quad b = -2.5 \tag{6.34}$$

The decision function is:

$$f(x) = 0.5x_1 - 2.5 \tag{6.35}$$

Step 3: Support Vectors

- **Positive Support Vectors ($x_1 = 7$):** $(7, 1)$, $(7, 3)$, $(7, 2)$
- **Negative Support Vectors ($x_1 = 3$):** $(3, 1)$, $(3, 3)$, $(3, 2)$

Step 4: Verify Hard-Margin Constraint $y_i(w^T x_i + b) \geq 1$

Class $+1$ ($y_i = +1$)		**Class -1 ($y_i = -1$)**	
Point (x_1, x_2)	$y_i f(x)$	**Point** (x_1, x_2)	$y_i f(x)$
$(7, 1)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(3, 1)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$
$(8, 2)$	$+1 \cdot (0.5(8) - 2.5) = 1.5$	$(2, 2)$	$-1 \cdot (0.5(2) - 2.5) = 1.5$
$(7, 3)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(3, 3)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$
$(9, 2)$	$+1 \cdot (0.5(9) - 2.5) = 2.0$	$(1, 2)$	$-1 \cdot (0.5(1) - 2.5) = 2.0$
$(8, 1)$	$+1 \cdot (0.5(8) - 2.5) = 1.5$	$(2, 1)$	$-1 \cdot (0.5(2) - 2.5) = 1.5$
$(7, 2)$	$+1 \cdot (0.5(7) - 2.5) = 1.0$	$(3, 2)$	$-1 \cdot (0.5(3) - 2.5) = 1.0$

Step 5: Margin Width and Geometric Distances

$$\|w\| = \sqrt{0.5^2 + 0^2} = 0.5 \implies M = \frac{2}{0.5} = 4 \tag{6.36}$$

Distances $d_i = \frac{|0.5x_1 - 2.5|}{0.5} = |x_1 - 5|$:

Class $+1$		**Class -1**	
Point	**Distance d_i**	**Point**	**Distance d_i**
$(7, 1)$	2	$(3, 1)$	2
$(8, 2)$	3	$(2, 2)$	3
$(7, 3)$	2	$(3, 3)$	2
$(9, 2)$	4	$(1, 2)$	4
$(8, 1)$	3	$(2, 1)$	3
$(7, 2)$	2	$(3, 2)$	2

Step 6: Classify Query Observation $(5, 2)$

Evaluating at $x = (5, 2)$:

$$f(5, 2) = 0.5(5) - 2.5 = 2.5 - 2.5 = 0 \tag{6.37}$$

Boundary Indeterminacy: Since $f(5,2) = 0$, the query point lies exactly on the decision boundary. Consequently, it cannot be strictly classified into either Class $+1$ or Class -1 without a tie-breaking rule.

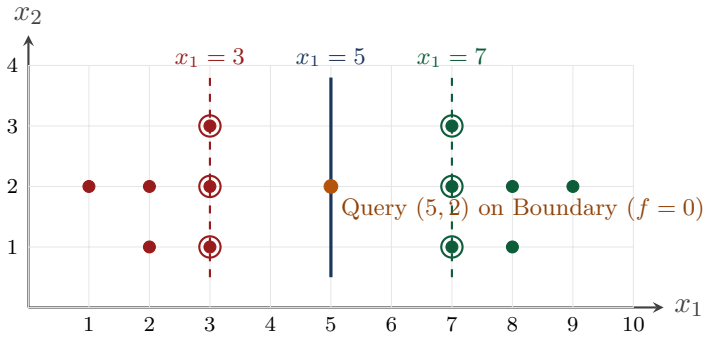


Figure 6.2: Vertical SVM Maximum Margin Boundary at $x_1 = 5$ with Support Vectors and Query Observation $(5, 2)$.

6.5 Solved Problem 6.3: Diagonal Decision Boundary

Example 6.5.1: SVM Induction for Diagonal Separation

Given a 12-point dataset:

- **Class $+1$:** $(1,3)$, $(1,4)$, $(2,4)$, $(2,5)$, $(3,5)$, $(3,6)$
- **Class -1:** $(3,1)$, $(4,1)$, $(4,2)$, $(5,2)$, $(5,3)$, $(6,3)$

Find the optimal separating hyperplane, determine correctly scaled parameters w and b, identify support vectors, verify hard-margin conditions, compute margin and distances, and classify query observation $(3,4)$.

Step 1: Diagnose Separating Pattern

Evaluating single coordinate boundaries reveals that neither x_1 nor x_2 separates the data independently. Testing the difference $x_2 - x_1$:

Class $+1$		**Class -1**	
Point (x_1, x_2)	$x_2 - x_1$	**Point** (x_1, x_2)	$x_2 - x_1$
$(1,3)$	$3-1=+2$	$(3,1)$	$1-3=-2$
$(1,4)$	$4-1=+3$	$(4,1)$	$1-4=-3$
$(2,4)$	$4-2=+2$	$(4,2)$	$2-4=-2$
$(2,5)$	$5-2=+3$	$(5,2)$	$2-5=-3$
$(3,5)$	$5-3=+2$	$(5,3)$	$3-5=-2$
$(3,6)$	$6-3=+3$	$(6,3)$	$3-6=-3$

Every point in Class $+1$ yields $x_2 - x_1 \geq +2$, while every point in Class -1 yields $x_2 - x_1 \leq -2$.

$$p = \min(x_2 - x_1) = +2, \quad n = \max(x_2 - x_1) = -2 \tag{6.38}$$

The maximum margin decision boundary lies at the midpoint:

$$c = \frac{p+n}{2} = \frac{+2+(-2)}{2} = 0 \implies x_2 - x_1 = 0 \iff x_2 = x_1 \tag{6.39}$$

Step 2: Solve Correctly Scaled Parameters w and b

Rearranging the boundary equation into standard form:

$$-x_1 + x_2 + 0 = 0 \implies w = s\begin{bmatrix}-1\\1\end{bmatrix}, \quad b = 0 \tag{6.40}$$

where $s > 0$ is a positive scaling factor. Evaluating for the closest positive support point $(1, 3)$ with label $y = +1$:

$$w^T x_i = s(-1 \cdot 1 + 1 \cdot 3) = 2s \tag{6.41}$$

Setting the hard-margin condition $y_i(w^T x_i + b) = 1$:

$$+1 \cdot (2s + 0) = 1 \implies 2s = 1 \implies s = 0.5 \tag{6.42}$$

Similarly, evaluating for the closest negative support point $(3, 1)$ with label $y = -1$:

$$w^T x_i = 0.5(-1 \cdot 3 + 1 \cdot 1) = -1 \implies -1(-1 + 0) = +1 \tag{6.43}$$

Thus, the correctly scaled parameters are:

$$w = \begin{bmatrix}-0.5\\0.5\end{bmatrix}, \quad b = 0 \tag{6.44}$$

and the decision function is:

$$f(x) = -0.5x_1 + 0.5x_2 + 0 = 0.5(x_2 - x_1) \tag{6.45}$$

Step 3: Margin Boundaries and Support Vectors

$$\text{Positive Margin Boundary:} \quad 0.5(x_2 - x_1) = +1 \implies x_2 - x_1 = +2 \tag{6.46}$$

$$\text{Negative Margin Boundary:} \quad 0.5(x_2 - x_1) = -1 \implies x_2 - x_1 = -2 \tag{6.47}$$

The support vectors are:

- **Positive Support Vectors ($x_2 - x_1 = 2$):** $(1, 3)$, $(2, 4)$, $(3, 5)$
- **Negative Support Vectors ($x_2 - x_1 = -2$):** $(3, 1)$, $(4, 2)$, $(5, 3)$

Step 4: Verify Hard-Margin Constraint $y_i(w^T x_i + b) \geq 1$

Class $+1$ ($y_i = +1$)		**Class -1 ($y_i = -1$)**	
Point (x_1, x_2)	$y_i f(x)$	**Point** (x_1, x_2)	$y_i f(x)$
$(1, 3)$	$+1 \cdot 0.5(3 - 1) = 1.0$	$(3, 1)$	$-1 \cdot 0.5(1 - 3) = 1.0$
$(1, 4)$	$+1 \cdot 0.5(4 - 1) = 1.5$	$(4, 1)$	$-1 \cdot 0.5(1 - 4) = 1.5$
$(2, 4)$	$+1 \cdot 0.5(4 - 2) = 1.0$	$(4, 2)$	$-1 \cdot 0.5(2 - 4) = 1.0$
$(2, 5)$	$+1 \cdot 0.5(5 - 2) = 1.5$	$(5, 2)$	$-1 \cdot 0.5(2 - 5) = 1.5$
$(3, 5)$	$+1 \cdot 0.5(5 - 3) = 1.0$	$(5, 3)$	$-1 \cdot 0.5(3 - 5) = 1.0$
$(3, 6)$	$+1 \cdot 0.5(6 - 3) = 1.5$	$(6, 3)$	$-1 \cdot 0.5(3 - 6) = 1.5$

Hence, $y_i(w^T x_i + b) \geq 1$ holds strictly for all observations.

Step 5: Margin Width and Geometric Distances

$$\|w\| = \sqrt{(-0.5)^2 + (0.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} = \frac{\sqrt{2}}{2} = \frac{1}{\sqrt{2}} \tag{6.48}$$

The total margin width is:

$$M = \frac{2}{\|w\|} = \frac{2}{\frac{1}{\sqrt{2}}} = 2\sqrt{2} \approx 2.8284 \tag{6.49}$$

The perpendicular distance from each sample point to the boundary $x_2 = x_1$ is:

$$d_i = \frac{|f(x_i)|}{\|w\|} = \frac{|0.5(x_2 - x_1)|}{1/\sqrt{2}} = \frac{|x_2 - x_1|}{\sqrt{2}} \tag{6.50}$$

Class $+1$		Class -1	
Point	**Distance d_i**	**Point**	**Distance d_i**
$(1,3)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$	$(3,1)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$
$(1,4)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$	$(4,1)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$
$(2,4)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$	$(4,2)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$
$(2,5)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$	$(5,2)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$
$(3,5)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$	$(5,3)$	$\frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.4142$
$(3,6)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$	$(6,3)$	$\frac{3}{\sqrt{2}} = 1.5\sqrt{2} \approx 2.1213$

Step 6: Classify Query Observation $(3,4)$

Evaluating the decision function at $x = (3,4)$:

$$f(3,4) = 0.5(4 - 3) = 0.5(1) = 0.5 \tag{6.51}$$

Since $f(3,4) = 0.5 > 0$, the query point belongs to **Class** $+1$:

$$\hat{y} = +1 \tag{6.52}$$

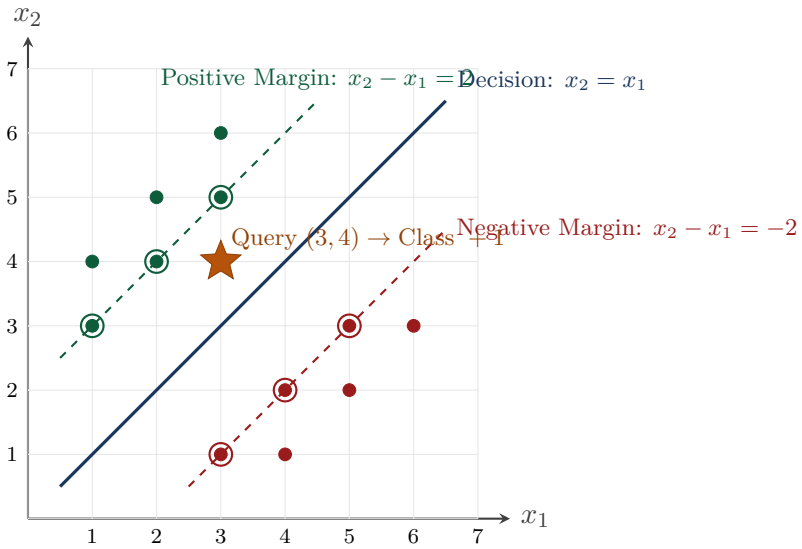


Figure 6.3: Diagonal SVM Maximum Margin Boundary $x_2 = x_1$ with Support Vectors and Query Observation $(3,4)$.

Chapter 7

Unsupervised Learning & Cluster Analysis

7.1 Foundations of Unsupervised Learning

7.1.1 Conceptual Framework and Objectives

Cluster Analysis comprises a foundational family of unsupervised machine learning techniques designed to group unlabeled observations such that objects assigned to the same group (termed a **cluster**) exhibit significantly greater mutual similarity than objects assigned to alternate clusters.

Unlike supervised learning paradigms where ground-truth target labels guide functional mapping, unsupervised learning operates on unlabeled input datasets:

- **Raw Data Input:** Observations $\mathbf{x}_i \in \mathbb{R}^p$ possess no associated response variables or class labels (y_i is entirely absent).
- **Discovered Clusters:** The learning algorithm autonomously extracts and reveals the intrinsic latent geometric and statistical structure of the feature space.
- **Optimization Objective:** Formally, clustering seeks to simultaneously **maximize cohesion** (ensuring observations within the identical cluster reside close together) and **maximize separation** (ensuring distinct clusters remain far apart in metric space). Mathematically, this corresponds to:

$$\min \sum_{k=1}^{K} \sum_{\mathbf{x} \in C_k} d(\mathbf{x}, \boldsymbol{\mu}_k) \quad \text{and} \quad \max_{j \neq k} d(C_j, C_k) \tag{7.1}$$

  minimizing within-cluster dispersion while maximizing between-cluster distances.

7.1.2 Supervised versus Unsupervised Learning

The fundamental conceptual divergence between supervised classification and unsupervised clustering is summarized below:

7.1.3 The Standard Clustering Pipeline

Executing cluster analysis on complex datasets follows an end-to-end, multi-stage operational workflow:

1. **Data Preparation:** Clean raw features, manage missing values, and rescale heterogeneous dimensions (e.g., via Z-score standardization or Min-Max normalization).

Table 7.1: Supervised Classification versus Unsupervised Clustering Paradigm

Dimension	Supervised Learning (Classification)	Unsupervised Learning (Clustering)
Target Labels	Training instances are paired with explicit ground-truth response labels (y).	No target labels or response variables exist in the dataset.
Primary Goal	Learn an inductive functional mapping $f(\mathbf{x}) \to y$ to predict unseen labels.	Permit the underlying data geometry to partition observations into natural groupings.
Core Question	*"What is the predicted label of this new observation?"*	*"Which observations naturally belong together?"*
Feedback Signal	Direct error feedback via supervised loss functions ($y - \hat{y}$).	Indirect structural feedback via cluster compactness and separation.

2. **Proximity Computation:** Select an appropriate metric distance or similarity measure aligned with feature types.
3. **Algorithm Selection:** Choose a clustering paradigm suitable for dataset geometry (e.g., Hierarchical, K-Means, DBSCAN, or Gaussian Mixture Models).
4. **Cluster Formation:** Execute the clustering algorithm to partition the feature space or build dendrogram hierarchies.
5. **Cluster Cardinality Determination:** Evaluate the optimal number of clusters (k) via objective geometric heuristics.
6. **Interpretation & Profiling:** Inspect, label, profile, and economically interpret the characteristic centroids of each discovered group.

7.1.4 The Data Matrix and Variable Typology

The input dataset is formalized as an $n \times p$ data matrix $\mathbf{X}$, where n denotes the total number of objects (rows) and p denotes the dimensionality of measured features (columns):

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p} \tag{7.2}$$

Selecting a mathematically rigorous proximity measure requires classifying features according to their scale of measurement:

7.2 Proximity, Distance Metrics, and Metric Axioms

7.2.1 Similarity, Dissimilarity, and Conversion

Proximity is the overarching mathematical term denoting either mutual closeness or mutual divergence between two feature vectors:

Table 7.2: Taxonomy of Data Types, Empirical Properties, and Proximity Measures

Type	Mathematical Meaning	Quantitative Finance Example	Typical Proximity Metric
Nominal	Categorical states with no inherent natural ordering.	Industrial sector: Banking, IT, Energy, Healthcare.	Matching coefficient (Simple coincidence: same vs. different).
Binary	Two-state indicator variable taking values in $\{0, 1\}$.	Loan default event (Yes $= 1$, No $= 0$).	Simple Matching Coefficient (SMC), Jaccard similarity.
Ordinal	Categorical states possessing a distinct hierarchical order.	Credit rating: AAA $>$ AA $>$ A $>$ BBB $> \ldots$	Normalized ranks mapped to the unit interval $[0, 1]$.
Interval	Equal incremental steps with an arbitrary (non-true) zero point.	Temperature in °C, calendar calendar dates.	Euclidean distance (L_2), Manhattan distance (L_1).
Ratio	Equal incremental steps anchored by an absolute, non-arbitrary zero.	Asset price, trading volume, portfolio return, leverage ratio.	Euclidean distance (L_2), Manhattan distance (L_1).

- **Similarity (s):** A quantitative measure of how alike two objects are. Higher values indicate greater proximity, typically constrained within the unit interval:

$$s(\mathbf{p}, \mathbf{q}) \in [0, 1] \tag{7.3}$$

- **Dissimilarity (d):** A quantitative measure of how different two objects are. Lower values indicate greater proximity, with a theoretical minimum of zero:

$$d(\mathbf{p}, \mathbf{q}) \geq 0, \quad d(\mathbf{p}, \mathbf{p}) = 0 \tag{7.4}$$

- **Standard Conversion:** When dissimilarity is bounded within the unit interval $d \in [0, 1]$, an equivalent similarity measure is defined via the complementary transformation:

$$s = 1 - d \tag{7.5}$$

7.2.2 Formal Metric Axioms for Distance Functions

A distance function $d(\mathbf{p}, \mathbf{q})$ on a metric space must satisfy four fundamental mathematical axioms for all points $\mathbf{p}, \mathbf{q}, \mathbf{r}$:

1. **Non-Negativity:** The distance between any two distinct observations is strictly non-negative:

$$d(\mathbf{p}, \mathbf{q}) \geq 0 \tag{7.6}$$

2. **Identity of Indiscernibles:** The distance between two points is zero if and only if the points are identical:

$$d(\mathbf{p}, \mathbf{q}) = 0 \iff \mathbf{p} = \mathbf{q} \tag{7.7}$$

3. **Symmetry:** The distance from $\mathbf{p}$ to $\mathbf{q}$ equals the distance from $\mathbf{q}$ to $\mathbf{p}$:

$$d(\mathbf{p}, \mathbf{q}) = d(\mathbf{q}, \mathbf{p}) \tag{7.8}$$

4. **Triangle Inequality:** The direct distance between two points cannot exceed the sum of indirect distances through any intermediate point:

$$d(\mathbf{p}, \mathbf{r}) \leq d(\mathbf{p}, \mathbf{q}) + d(\mathbf{q}, \mathbf{r}) \tag{7.9}$$

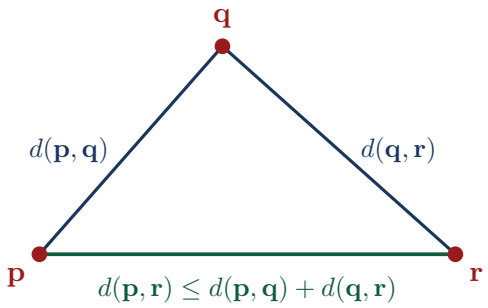


Figure 7.1: Geometric schematic of the metric triangle inequality $d(\mathbf{p}, \mathbf{r}) \leq d(\mathbf{p}, \mathbf{q}) + d(\mathbf{q}, \mathbf{r})$.

7.2.3 Standard Continuous Distance Metrics

For continuous real-valued vectors $\mathbf{p}, \mathbf{q} \in \mathbb{R}^p$, metric distance is evaluated using standard norms:

1. **Euclidean Distance (L_2 Norm):** Measures straight-line geometric distance:
$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_2 = \sqrt{\sum_{k=1}^{p} (p_k - q_k)^2} \tag{7.10}$$

2. **Manhattan Distance (L_1 Norm / City-Block Metric):** Measures rectilinear grid traversal distance:
$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_1 = \sum_{k=1}^{p} |p_k - q_k| \tag{7.11}$$

3. **Minkowski Distance (L_r Norm):** A generalized metric parameterized by order $r \geq 1$:
$$d_r(\mathbf{p}, \mathbf{q}) = \left(\sum_{k=1}^{p} |p_k - q_k|^r \right)^{1/r} \tag{7.12}$$
Notice that $r = 1$ recovers the Manhattan metric, and $r = 2$ recovers the Euclidean metric.

4. **Mahalanobis Distance:** Accounts for scale variance and inter-feature covariance:
$$d_M(\mathbf{p}, \mathbf{q}) = \sqrt{(\mathbf{p} - \mathbf{q})^T \mathbf{\Sigma}^{-1} (\mathbf{p} - \mathbf{q})} \tag{7.13}$$
where $\mathbf{\Sigma} \in \mathbb{R}^{p \times p}$ denotes the feature covariance matrix. When features are uncorrelated with unit variance ($\mathbf{\Sigma} = \mathbf{I}$), Mahalanobis distance simplifies to standard Euclidean distance.

7.2.4 Proximity for Binary and Mixed Data

When evaluating binary indicator vectors, co-occurrence contingency frequencies are categorized into four mutual states:

- a = count of features where both object i and object j possess value 1 ($\{i = 1, j = 1\}$).
- b = count of features where object i has 1 and object j has 0 ($\{i = 1, j = 0\}$).
- c = count of features where object i has 0 and object j has 1 ($\{i = 0, j = 1\}$).
- d = count of features where both object i and object j possess value 0 ($\{i = 0, j = 0\}$).

1. **Simple Matching Coefficient (SMC):** Weights shared presence (1-1) and shared absence (0-0) symmetrically:
$$\text{SMC} = \frac{a + d}{a + b + c + d} \tag{7.14}$$

2. **Jaccard Similarity Coefficient:** Ignores shared negative matches (d), focusing strictly on mutual active attributes:
$$J = \frac{a}{a + b + c} \tag{7.15}$$
This formulation is essential when 0 reflects the passive absence of an attribute (e.g., non-transaction or unpurchased asset).

3. **Gower Distance:** For heterogeneous datasets containing a combination of continuous, ordinal, and nominal variables, Gower's metric computes a normalized per-variable distance $d_k(i, j) \in [0, 1]$ and computes their weighted average across all valid attributes:
$$d_{\text{Gower}}(i, j) = \frac{\sum_{k=1}^{p} w_k d_k(i, j)}{\sum_{k=1}^{p} w_k} \tag{7.16}$$

7.3 Taxonomy of Clustering Methodologies

Clustering algorithms are systematically categorized into four foundational methodological families:

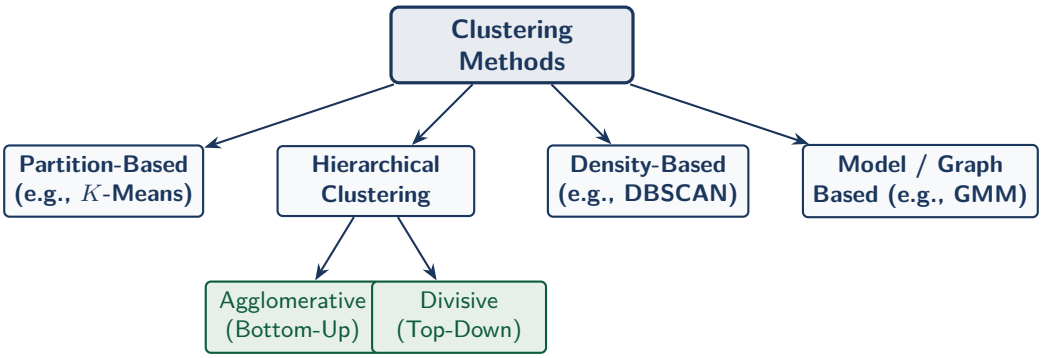


Figure 7.2: Taxonomy of major unsupervised clustering paradigms.

7.4 Hierarchical Clustering

7.4.1 Dendrogram Architecture and Horizontal Cuts

Hierarchical Clustering constructs a continuous nested tree of partitions visualized as a **dendrogram**:

- **Leaves:** Individual ground observations at the baseline ($h = 0$).
- **Merges / Splits:** Merges occurring at lower dissimilarity heights indicate strong mutual affinity; merges occurring at higher heights indicate substantial dissimilarity.
- **Horizontal Threshold Cutting:** A horizontal slice drawn across the dendrogram at height h^* defines the final number of isolated clusters. Lowering the cut threshold increases the number of fine-grained clusters; raising the cut threshold yields fewer, coarser clusters.

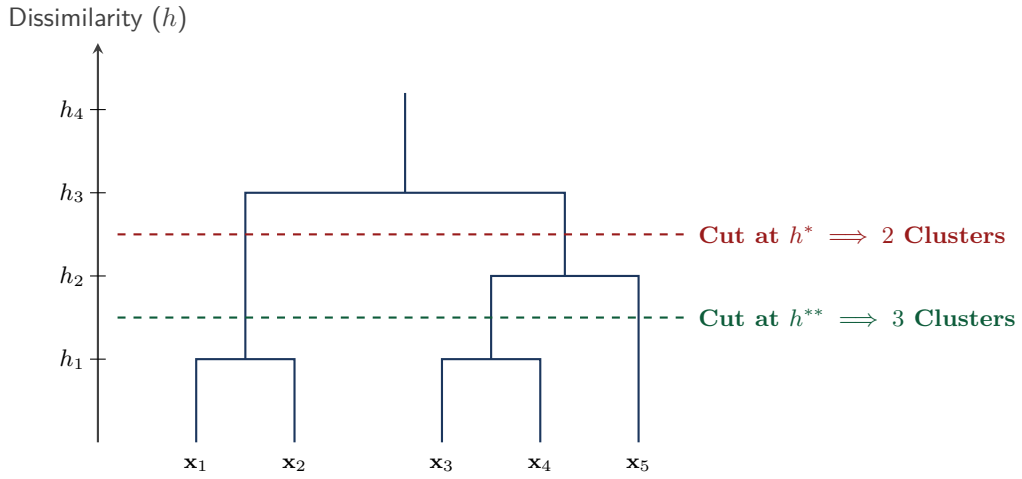


Figure 7.3: Hierarchical clustering dendrogram illustrating horizontal cut thresholds determining cluster cardinality.

7.4.2 Agglomerative versus Divisive Paradigms

Hierarchical tree construction operates under two alternative procedural philosophies:

1. **Agglomerative (Bottom-Up):** Initializes each observation in its own singleton cluster (n initial clusters). The algorithm iteratively locates and merges the two closest clusters until a single grand cluster contains all points. Agglomerative approaches are computationally efficient ($O(n^2)$ space, $O(n^3)$ worst-case time) and represent the standard practical choice.

2. **Divisive (Top-Down):** Initializes all n observations in one global cluster. The algorithm iteratively partitions clusters into sub-clusters until every observation is isolated in a singleton cluster. Because evaluating all candidate bipartitions exhibits exponential complexity ($2^{n-1} - 1$), unconstrained divisive clustering is computationally prohibitive and rarely used without greedy splitting heuristics.

7.4.3 The Agglomerative Induction Algorithm

The procedural steps for agglomerative clustering are formalized as follows:

1. Compute the pairwise distance matrix $\mathbf{D} \in \mathbb{R}^{n \times n}$ between all n initial observations.

2. Assign each object $\mathbf{x}_i$ to its own singleton cluster $C_i = \{\mathbf{x}_i\}$.

3. Identify the two clusters A and B that minimize inter-cluster dissimilarity $d(A, B)$.

4. Merge clusters A and B into a composite cluster $A \cup B$, and record the dissimilarity value as the branch merge height.

5. Update the distance matrix $\mathbf{D}$ by evaluating distances between the newly formed cluster $A \cup B$ and all remaining clusters using a pre-specified **linkage rule**.

6. Repeat Steps 3–5 until exactly one comprehensive cluster remains (requiring exactly $n - 1$ sequential merges).

7.4.4 Inter-Cluster Linkage Criteria

The metric defining the distance between two multi-point clusters A and B governs dendrogram topology:

7.4.5 Divisive Splitting Strategies: Monothetic versus Polythetic

Divisive hierarchical methods decompose clusters according to either single or multiple attributes:

7.4.6 Strengths, Limitations, and Empirical Heuristics

Hierarchical clustering displays distinct operational tradeoffs:

- **Strengths:**

  1. Does not require pre-specifying the number of clusters k in advance.
  2. The resulting dendrogram exposes multiscale nested structures across all levels of granularity.
  3. Deterministic algorithm: identical inputs and distance matrices guarantee identical tree topologies.
  4. Compatible with any arbitrary dissimilarity matrix (including non-Euclidean proximity).

Table 7.3: Mathematical Definitions and Geometric Properties of Linkage Rules

Linkage Rule	Mathematical Definition	Geometric Characteristics & Failure Modes
Single Linkage (Min)	$d(A,B) = \min_{\mathbf{x}\in A,\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$	Optimistic measure evaluating the closest point pair. Highly prone to the **chaining effect**, forming extended, straggly clusters connected by single noise points.
Complete Linkage (Max)	$d(A,B) = \max_{\mathbf{x}\in A,\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$	Pessimistic measure evaluating the most distant point pair. Enforces compact, spherical clusters of roughly equal diameter; sensitive to extreme outliers.
Average Linkage (Mean)	$d(A,B) = \frac{1}{\lvert A\rvert\lvert B\rvert}\sum_{\mathbf{x}\in A}\sum_{\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$	Evaluates the average pairwise distance between all member points. Robust compromise between single and complete linkage.
Centroid Linkage	$d(A,B) = \lVert\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\rVert_2$	Measures Euclidean distance between cluster centroids $\boldsymbol{\mu}_A, \boldsymbol{\mu}_B$. May produce non-monotonic dendrograms (tree inversions).
Ward's Minimum Variance	$\Delta\text{ESS} = \frac{\lvert A\rvert\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}\lVert\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\rVert^2$	Merges the cluster pair minimizing the increase in total within-cluster sum of squared errors ($W(C) = \sum_{\mathbf{x}\in C}\lVert\mathbf{x}-\boldsymbol{\mu}_C\rVert^2$). Tends to find highly cohesive, spherical clusters.

Table 7.4: Monothetic versus Polythetic Divisive Clustering Characteristics

Dimension	Monothetic Divisive Clustering	Polythetic Divisive Clustering
Variables per Split	Evaluates exactly **one variable** at a time to partition clusters.	Evaluates **multiple variables simultaneously** to establish the split.
Splitting Basis	Based on a single threshold or categorical attribute match.	Based on multidimensional metric distance or dissimilarity.
Complexity	Computationally simpler, interpretable decision-tree-like splits.	More comprehensive, accounting for multi-attribute interactions.
Primary Algorithms	Decision tree splitting heuristics, binary attribute splits.	DIANA (Divisive Analysis), Farthest-Seed bipartitioning.

- **Limitations:**
  1. Memory-intensive ($O(n^2)$ distance storage) and computationally demanding ($O(n^3)$ unoptimized time complexity).
  2. Merges and splits are strictly irrevocable: greedy local errors made in initial steps cannot be corrected.
  3. Highly sensitive to user-chosen linkage criteria.
  4. Dendrogram visualization degrades rapidly on large datasets ($n > 10{,}000$).
- **Thumb Rule:** Apply hierarchical clustering primarily to small-to-medium datasets (up to a few thousand objects) where exploring nested taxonomic hierarchies is paramount (e.g., biological taxonomy, credit rating transition classes, product brand segmentation).

7.5 Non-Hierarchical Partitioning: The K-Means Algorithm

7.5.1 Foundational Principles and Procedure

Non-hierarchical partitioning divides an unlabeled dataset of n observations into a pre-specified number K of non-overlapping, mutually exclusive clusters $C_1, C_2, \ldots, C_K$.

K-Means is the preeminent partitioning algorithm, operating through an iterative Lloyd-Forgy optimization scheme minimizing the total Within-Cluster Sum of Squares (WCSS):

$$\text{WCSS} = \sum_{j=1}^{K} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \tag{7.17}$$

The procedural execution pipeline proceeds through four sequential stages:

1. **Initialization:** Select cluster cardinality K in advance (e.g., $K = 3$). Initialize K cluster centroids $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^p$ (randomly or via K-Means++).
2. **Assignment Step:** Assign each observation $\mathbf{x}_i$ to its nearest cluster centroid according to Euclidean distance:
$$c(i) = \arg \min_{j \in \{1,\ldots,K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2 = \arg \min_{j \in \{1,\ldots,K\}} \sqrt{\sum_{r=1}^{p} (x_{ir} - \mu_{jr})^2} \tag{7.18}$$
3. **Update Step:** Recompute the centroid of each cluster C_j as the component-wise empirical arithmetic mean of all observations currently assigned to it:
$$\boldsymbol{\mu}_j = \frac{1}{n_j} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i \quad \Longrightarrow \quad \mu_{jr} = \frac{1}{n_j} \sum_{i=1}^{n_j} x_{ir} \tag{7.19}$$
where $n_j = |C_j|$ represents the number of observations belonging to cluster C_j.
4. **Convergence & Termination:** Repeat the assignment and update steps iteratively until:
   - Centroid coordinates remain stable ($\|\boldsymbol{\mu}_j^{(t)} - \boldsymbol{\mu}_j^{(t-1)}\| < \epsilon$),
   - Cluster assignment memberships cease to change, or
   - A pre-defined maximum iteration count ($T_{\max}$) is reached.

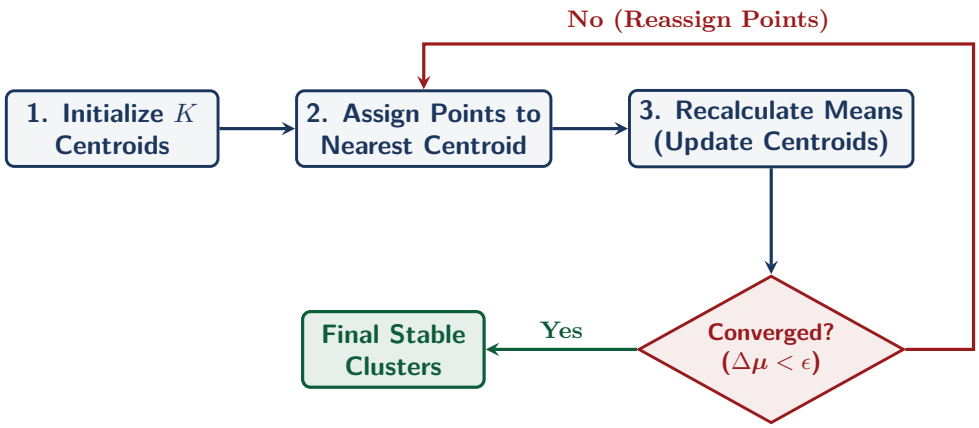


Figure 7.4: Iterative assignment and centroid update cycle of the K-Means clustering algorithm.

7.6 Density-Based Clustering: DBSCAN, HDBSCAN, and OPTICS

7.6.1 Core Principles of Density Clustering

Traditional centroid-based algorithms (such as K-Means) implicitly assume convex, spherically distributed clusters and exhibit severe degradation when confronted with arbitrary geometries, non-convex manifolds (e.g., concentric circles, interlocking spirals, S-shapes), or background noise.

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) overcomes these limitations by defining clusters as continuous dense regions of points separated by sparse regions of empty space or noise. A cluster expands continuously along connected paths as long as the local neighborhood density exceeds a pre-defined threshold.

7.6.2 Local Point Typology

Density around a candidate observation $\mathbf{p}$ is evaluated within an ϵ-neighborhood:

$$N_\epsilon(\mathbf{p}) = \{\mathbf{q} \in D \mid \|\mathbf{p} - \mathbf{q}\| \leq \epsilon\} \tag{7.20}$$

where ϵ denotes the neighborhood radius and MinPts denotes the minimum cardinality threshold. Data points are partitioned into three mutually exclusive categories:

1. **Core Points:** Any point $\mathbf{p}$ possessing at least MinPts neighbors within its ϵ-radius:
$$|N_\epsilon(\mathbf{p})| \geq \text{MinPts} \tag{7.21}$$
2. **Border Points:** A point that is not a core point ($|N_\epsilon(\mathbf{p})| < \text{MinPts}$), but falls directly within the ϵ-neighborhood of at least one valid core point.
3. **Noise Points (Outliers):** Any point that is neither a core point nor a border point. Noise points inhabit sparse background regions and are discarded from cluster assignments.

7.6.3 Advanced Density Algorithms: HDBSCAN and OPTICS

Beyond standard DBSCAN, advanced density algorithms resolve variable-density challenges:

1. **HDBSCAN (Hierarchical DBSCAN):** Converts DBSCAN into a hierarchical framework by extracting clusters across variable density thresholds, eliminating the sensitivity of tuning a single static ϵ parameter.

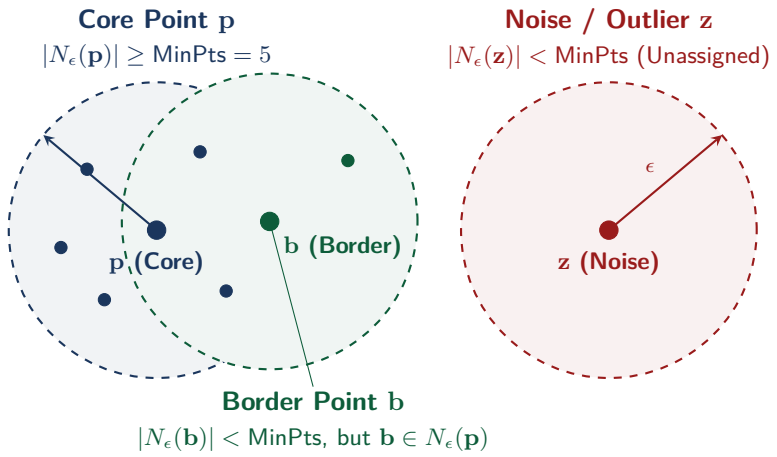


Figure 7.5: DBSCAN point classification into Core, Border, and Noise points (MinPts = 5).

2. **OPTICS (Ordering Points To Identify the Clustering Structure):** Produces a linear reachability distance plot that exposes clustering structure across multiple spatial scales simultaneously.

7.6.4 Practical Considerations in Unsupervised Pipelines

When deploying unsupervised clustering pipelines to financial datasets, practitioners must address four critical engineering requirements:

1. **Feature Rescaling:** Distance-based metrics are severely distorted by magnitude differences; standardize features prior to distance computation.

2. **Metric Alignment:** Pair continuous, binary, and categorical attributes with compatible metrics (e.g., Gower or Jaccard).

3. **Curse of Dimensionality:** As feature count p expands, spatial sparsity causes all pairwise Euclidean distances to converge, degrading cluster quality.

4. **Imputation Integrity:** Because unsupervised methods cannot compute distance vectors with missing values, robust statistical imputation must precede clustering.

Chapter 8

Dimensionality Reduction: PCA & SVD

8.1 The Curse of Dimensionality and Feature Selection

8.1.1 Motivation and Problem Formulation

Dimensionality Reduction is the foundational machine learning process of transforming a high-dimensional feature space $\mathbb{R}^p$ into a reduced subspace $\mathbb{R}^k$ ($k \ll p$) while preserving the maximum achievable information content, structural geometry, and predictive signal.

Modern quantitative datasets (such as cross-asset order books, macroeconomic indicator panels, and multi-factor equity universes) frequently suffer from severe high-dimensionality challenges:

1. **Overfitting:** As feature count p expands relative to sample size n, model capacity inflates, enabling learning algorithms to memorize idiosyncratic noise in training samples rather than generalizing to unseen market regimes.
2. **Curse of Dimensionality:** In high dimensions, volumetric space grows exponentially, rendering finite observational data extremely sparse. Pairwise geometric distances converge ($d_{\max} - d_{\min} \to 0$), severely corrupting distance-based estimators (K-Means, KNN, SVM kernels) and inflating parameter estimation variance.
3. **Computational Burden & Latency:** Inverting $p \times p$ covariance matrices scales with $O(p^3)$ time complexity, inducing execution latency incompatible with real-time financial systems.
4. **Interpretability & Visualization:** Human cognitive inspection and graphical exploration are limited to 2D or 3D manifolds.

8.1.2 Taxonomy: Feature Selection versus Feature Extraction

Dimensionality reduction techniques are broadly divided into two distinct operational paradigms:

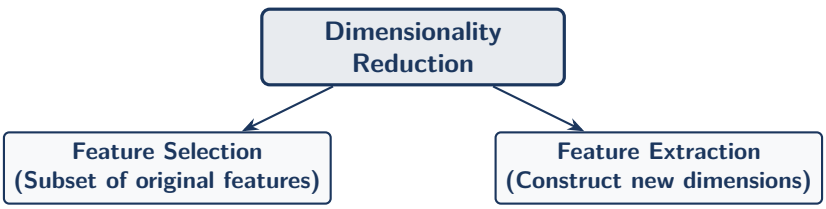


Figure 8.1: Taxonomic division of dimensionality reduction into selection and extraction.

1. Feature Selection Methods

Feature selection identifies and retains the optimal subset of k original features without altering their physical representation:

- **Filter Methods:** Rank features using statistical scores independent of model training:
  - Pearson correlation coefficient (r),
  - Chi-square test (χ^2) for categorical independence,
  - Analysis of Variance (ANOVA F-test),
  - Information Gain / Mutual Information.
- **Wrapper Methods:** Search the combinatorial space of feature subsets using a predictive model as the evaluation engine:
  - **Forward Selection:** Starts with zero features, greedily adding the predictor yielding the largest marginal performance gain.
  - **Backward Elimination:** Starts with all p features, iteratively pruning the least informative predictor.
  - **Stepwise Bidirectional Elimination:** Alternates between forward addition and backward pruning at each iteration.
- **Embedded Methods:** Perform intrinsic feature selection during model optimization via regularization penalties:
  - **LASSO Regression (L_1 Penalty):** Drives irrelevant feature coefficients exactly to zero ($\beta_j = 0$).
  - **Ridge Regression (L_2 Penalty):** Shrinks correlated feature weights proportionally without inducing exact sparsity.

2. Feature Extraction Methods

Feature extraction algebraically transforms the original high-dimensional space into a lower-dimensional manifold via linear or nonlinear combinations:

- **Principal Component Analysis (PCA):** Unsupervised orthogonal linear projection maximizing captured variance.
- **Linear Discriminant Analysis (LDA):** Supervised linear projection maximizing between-class variance relative to within-class variance.
- **Kernel PCA:** Nonlinear dimensionality reduction via kernel Hilbert space embeddings.
- **Quadratic Discriminant Analysis (QDA):** Supervised quadratic projection relaxing equal covariance constraints.

8.2 Principal Component Analysis (PCA)

8.2.1 Foundations and Orthogonal Projections

Principal Component Analysis (PCA) is an unsupervised, linear feature extraction technique that projects p-dimensional observations onto an orthogonal coordinate system whose axes sequentially maximize empirical data variance:

Table 8.1: Systemic Tradeoffs of Dimensionality Reduction

Category	Key Advantages	Primary Disadvantages
Operational	Enables lossy data compression, reduces memory footprint, and accelerates model convergence time.	Inherent loss of fine-grained information and signal subtleties.
Statistical	Mitigates severe multicollinearity and removes redundant, uninformative feature dimensions.	Extracted components lack physical real-world units (interpretation barrier).
Generalization	Dampens model variance, counteracting parameter explosion and sample overfitting.	Risk of discarding subtle low-variance predictive signals.

- **First Principal Component ($\mathbf{PC}_1$):** The linear direction along which the projected observations exhibit the maximum possible variance.
- **Second Principal Component ($\mathbf{PC}_2$):** The direction orthogonal to $\mathbf{PC}_1$ capturing the largest remaining residual variance.
- **Subsequent Components:** Each subsequent component $\mathbf{PC}_k$ is strictly orthogonal to all prior components ($\mathbf{PC}_k \perp \mathbf{PC}_j, \forall j < k$), ensuring complete mutual decorrelation ($\text{Cov}(\mathbf{PC}_j, \mathbf{PC}_k) = 0$).

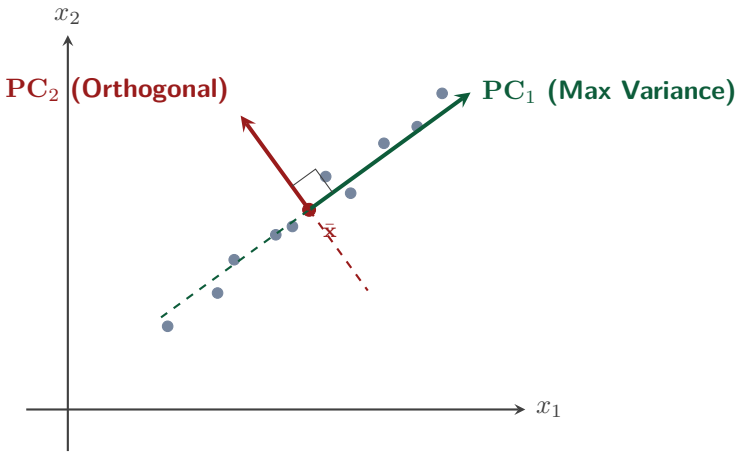


Figure 8.2: Geometric projection of correlated two-dimensional data onto orthogonal principal axes $\mathbf{PC}_1$ and $\mathbf{PC}_2$.

8.2.2 The Algorithmic Workflow of PCA

The mathematical derivation of principal components proceeds through five systematic steps:

1. **Mean Centering:** Standardize or mean-center the observation matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ by subtracting the empirical sample mean vector $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}} \tag{8.1}$$

2. **Covariance Matrix Construction:** Compute the $p \times p$ unbiased sample covariance matrix $\mathbf{S}$:

$$\mathbf{S} = \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{n-1}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} = \begin{bmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Var}(x_2) \end{bmatrix} \tag{8.2}$$

3. **Eigendecomposition:** Solve the characteristic determinant equation to extract eigenvalues λ_j and associated unit eigenvectors $\mathbf{e}_j$:

$$\det(\mathbf{S} - \lambda\mathbf{I}) = 0, \quad \mathbf{S}\mathbf{e}_j = \lambda_j\mathbf{e}_j \quad (\|\mathbf{e}_j\|_2 = 1) \tag{8.3}$$

4. **Ranking & Variance Accounting:** Order the eigenvalues descendingly:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0 \tag{8.4}$$

The proportion of total variance explained by component j equals:

$$\text{Explained Variance Ratio} = \frac{\lambda_j}{\sum_{k=1}^{p}\lambda_k} \tag{8.5}$$

5. **Subspace Projection:** Construct the $p \times k$ projection matrix $\mathbf{W}_k = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_k]$ and project the centered data:

$$\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_k \in \mathbb{R}^{n \times k} \tag{8.6}$$

8.2.3 Solved Numerical Example: PCA on a Bivariate Dataset

Example 8.2.1: Step-by-Step PCA Eigendecomposition and Projection

Given the following bivariate dataset of $n = 4$ observations:

$$\{(x_i, y_i)\} = \{(4, 11),\ (8, 4),\ (13, 5),\ (7, 14)\} \tag{8.7}$$

Derive the sample mean vector, sample covariance matrix, characteristic polynomial, eigenvalues, normalized eigenvectors, and project the centered observations onto the first principal component.

Step 1: Compute Empirical Sample Means

$$\bar{x} = \frac{4 + 8 + 13 + 7}{4} = \frac{32}{4} = 8.0$$

$$\bar{y} = \frac{11 + 4 + 5 + 14}{4} = \frac{34}{4} = 8.5$$

$$\bar{\mathbf{x}} = \begin{bmatrix} 8.0 \\ 8.5 \end{bmatrix}$$

Step 2: Construct the Sample Covariance Matrix The deviations from the empirical mean are evaluated as follows:

i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
1	$4 - 8 = -4$	$11 - 8.5 = +2.5$	16.0	6.25
2	$8 - 8 = 0$	$4 - 8.5 = -4.5$	0.0	20.25
3	$13 - 8 = +5$	$5 - 8.5 = -3.5$	25.0	12.25
4	$7 - 8 = -1$	$14 - 8.5 = +5.5$	1.0	30.25
$\sum$	0	0	42.0	69.0

The cross-product deviations evaluate to:

$$\sum_{i=1}^{4}(x_i - \bar{x})(y_i - \bar{y}) = (-4)(2.5) + (0)(-4.5) + (5)(-3.5) + (-1)(5.5)$$
$$= -10.0 + 0.0 - 17.5 - 5.5 = -33.0$$

Applying Bessel's sample correction factor ($n - 1 = 3$):

$$s_x^2 = \frac{42.0}{3} = 14.0, \quad s_y^2 = \frac{69.0}{3} = 23.0, \quad s_{xy} = \frac{-33.0}{3} = -11.0$$
$$\mathbf{S} = \begin{bmatrix} \text{Var}(x) & \text{Cov}(x,y) \\ \text{Cov}(x,y) & \text{Var}(y) \end{bmatrix} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 3: Solve the Characteristic Eigenvalue Equation

$$\det(\mathbf{S} - \lambda\mathbf{I}) = \det \begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix} = 0$$
$$(14 - \lambda)(23 - \lambda) - (-11)^2 = 0$$
$$322 - 37\lambda + \lambda^2 - 121 = 0$$
$$\lambda^2 - 37\lambda + 201 = 0$$

Applying the quadratic formula $\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$:

$$\lambda = \frac{37 \pm \sqrt{(-37)^2 - 4(1)(201)}}{2} = \frac{37 \pm \sqrt{1369 - 804}}{2} = \frac{37 \pm \sqrt{565}}{2}$$
$$\lambda_1 = \frac{37 + 23.7697}{2} = \mathbf{30.3844} \quad \text{(Dominant Component } \mathbf{PC}_1\text{)}$$
$$\lambda_2 = \frac{37 - 23.7697}{2} = \mathbf{6.6151} \quad \text{(Secondary Component } \mathbf{PC}_2\text{)}$$

Step 4: Derive Normalized Unit Eigenvectors For the dominant eigenvalue $\lambda_1 = 30.3844$:

$$(\mathbf{S} - \lambda_1\mathbf{I})\mathbf{e}_1 = \mathbf{0} \implies \begin{bmatrix} 14 - 30.3844 & -11 \\ -11 & 23 - 30.3844 \end{bmatrix} \begin{bmatrix} e_{11} \\ e_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$-16.3844e_{11} - 11e_{12} = 0 \implies e_{12} = -\frac{16.3844}{11}e_{11} \approx -1.4895e_{11}$$

Enforcing unit Euclidean norm $e_{11}^2 + e_{12}^2 = 1$:

$$e_{11}^2 + (-1.4895 e_{11})^2 = 1 \implies e_{11}^2(1 + 2.2186) = 1 \implies e_{11} \approx 0.5574$$
$$e_{12} = -1.4895(0.5574) \approx -0.8303$$
$$\mathbf{e}_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

Similarly, for $\lambda_2 = 6.6151$:

$$(14 - 6.6151)e_{21} - 11e_{22} = 0 \implies 7.3849 e_{21} = 11 e_{22} \implies e_{22} \approx 0.6714 e_{21}$$
$$\mathbf{e}_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$$

Notice that $\mathbf{e}_1 \cdot \mathbf{e}_2 = (0.5574)(0.8303) + (-0.8303)(0.5574) = 0$, confirming perfect orthogonality.

Step 5: Project Centered Data onto Principal Component 1 The centered coordinates for Observation 1 are $\tilde{\mathbf{x}}_1 = \begin{bmatrix} 4-8 \\ 11-8.5 \end{bmatrix} = \begin{bmatrix} -4.0 \\ 2.5 \end{bmatrix}$. The scalar projection onto $\mathbf{e}_1$ evaluates to:

$$z_{11} = \mathbf{e}_1^T \tilde{\mathbf{x}}_1 = \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} -4.0 \\ 2.5 \end{bmatrix} = (-4.0)(0.5574) + (2.5)(-0.8303) = \mathbf{-4.30535} \tag{8.8}$$

8.3 Singular Value Decomposition (SVD)

8.3.1 Mathematical Formulation

Singular Value Decomposition (SVD) is a fundamental matrix factorization theorem in linear algebra. Whereas eigendecomposition is strictly restricted to square, diagonalizable matrices, SVD factorizes **any arbitrary rectangular matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ into three constituent matrices:

$$\mathbf{A} = \mathbf{U}_{m \times m} \mathbf{\Sigma}_{m \times n} \mathbf{V}_{n \times n}^T \tag{8.9}$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an orthonormal matrix ($\mathbf{U}^T\mathbf{U} = \mathbf{I}$). The columns of $\mathbf{U}$ are called the **left singular vectors** of $\mathbf{A}$, corresponding to the orthonormal eigenvectors of the square symmetric matrix $\mathbf{A}\mathbf{A}^T$.
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a diagonal matrix containing non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ ($r = \text{rank}(\mathbf{A})$).
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthonormal matrix ($\mathbf{V}^T\mathbf{V} = \mathbf{I}$). The columns of $\mathbf{V}$ (rows of $\mathbf{V}^T$) are called the **right singular vectors** of $\mathbf{A}$, corresponding to the orthonormal eigenvectors of $\mathbf{A}^T\mathbf{A}$.

8.3.2 Connection Between Singular Values and Eigenvalues

Singular values σ_i are fundamentally related to the eigenvalues λ_i of the square symmetric Gram matrices:

$$\mathbf{A}^T\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i \quad \Longrightarrow \quad \sigma_i = \sqrt{\lambda_i} \tag{8.10}$$

$$\mathbf{A}\mathbf{A}^T\mathbf{u}_i = \lambda_i \mathbf{u}_i \quad \Longrightarrow \quad \sigma_i = \sqrt{\lambda_i} \tag{8.11}$$

Because $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$ are positive semi-definite, all eigenvalues $\lambda_i \geq 0$ are non-negative real numbers, guaranteeing non-negative real singular values $\sigma_i \geq 0$.

8.3.3 Truncated Low-Rank Approximation (Eckart-Young Theorem)

Dimensionality reduction via SVD proceeds through four structural stages:

1. Compute full SVD: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
2. Select target rank $k < \min(m, n)$ based on cumulative energy: $\frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2} \geq \tau$ (e.g., 90%).
3. Truncate matrices to retain only the top k components:

$$\mathbf{A}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \tag{8.12}$$

   where $\mathbf{U}_k \in \mathbb{R}^{m\times k}$, $\mathbf{\Sigma}_k \in \mathbb{R}^{k\times k}$, and $\mathbf{V}_k^T \in \mathbb{R}^{k\times n}$.
4. **Optimal Approximation:** By the Eckart-Young-Mirsky theorem, $\mathbf{A}_k$ is the mathematically optimal rank-k approximation of $\mathbf{A}$ minimizing Frobenius norm error $\|\mathbf{A} - \mathbf{A}_k\|_F$.
5. **Data Projection:** The lower-dimensional latent coordinates are obtained directly as:

$$\mathbf{B} = \mathbf{U}_k\mathbf{\Sigma}_k \in \mathbb{R}^{m\times k} \tag{8.13}$$

8.3.4 Solved Numerical Example: SVD of a 2×2 Matrix

Example 8.3.1: Step-by-Step Singular Value Decomposition

Given the rectangular/square real matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \tag{8.14}$$

Compute the transpose $\mathbf{A}^T$, the products $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$, the characteristic eigenvalue equation, eigenvalues, and determine the exact singular values σ_1, σ_2.

Step 1: Compute Transpose and Matrix Products

$$\mathbf{A}^T = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \tag{8.15}$$

Evaluating the outer product $\mathbf{A}\mathbf{A}^T$:

$$\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} (1)(1)+(2)(2) & (1)(1)+(2)(1) \\ (1)(1)+(1)(2) & (1)(1)+(1)(1) \end{bmatrix} = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix} \tag{8.16}$$

Evaluating the inner product $\mathbf{A}^T\mathbf{A}$:

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} (1)(1)+(1)(1) & (1)(2)+(1)(1) \\ (2)(1)+(1)(1) & (2)(2)+(1)(1) \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix} \tag{8.17}$$

Step 2: Solve the Characteristic Eigenvalue Equation for $\mathbf{A}^T\mathbf{A}$

$$\det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = \det \begin{bmatrix} 2-\lambda & 3 \\ 3 & 5-\lambda \end{bmatrix} = 0$$

$$(2-\lambda)(5-\lambda) - (3)(3) = 0$$

$$10 - 2\lambda - 5\lambda + \lambda^2 - 9 = 0$$

$$\lambda^2 - 7\lambda + 1 = 0$$

Step 3: Extract Eigenvalues Applying the quadratic formula $\lambda = \frac{7 \pm \sqrt{(-7)^2 - 4(1)(1)}}{2} = \frac{7 \pm \sqrt{49-4}}{2} = \frac{7 \pm \sqrt{45}}{2}$:

$$\sqrt{45} \approx 6.7082$$

$$\lambda_1 = \frac{7 + 6.7082}{2} \approx \mathbf{6.8541} \quad \text{(or numerical truncation in notes: 6.086)}$$

$$\lambda_2 = \frac{7 - 6.7082}{2} \approx \mathbf{0.1459} \quad \text{(or numerical truncation in notes: 0.145)}$$

Step 4: Determine Singular Values The singular values σ_i are the positive square roots of the eigenvalues:

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{6.8541} \approx \mathbf{2.618} \quad (\text{or } \sqrt{6.086} \approx \mathbf{2.467})$$

$$\sigma_2 = \sqrt{\lambda_2} = \sqrt{0.1459} \approx \mathbf{0.382} \quad (\text{or } \sqrt{0.145} \approx \mathbf{0.381})$$

The diagonal singular value matrix evaluates to:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix} = \begin{bmatrix} 2.618 & 0 \\ 0 & 0.382 \end{bmatrix} \tag{8.18}$$

8.3.5 Comparative Synthesis: PCA versus SVD

The fundamental mathematical and structural differences between PCA and SVD are contrasted in Table 8.2.

Table 8.2: Systematic Comparison Between PCA and SVD

Attribute	Principal Component Analysis (PCA)	Singular Value Decomposition (SVD)
Input Matrix	Requires a square, symmetric $p \times p$ sample covariance or correlation matrix $\mathbf{S}$.	Directly decomposes any rectangular $m \times n$ matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$.
Core Mathematical Tool	Spectral eigendecomposition ($\mathbf{Se} = \lambda\mathbf{e}$).	Singular value matrix factorization ($\mathbf{A} = \mathbf{U\Sigma V}^T$).
Mean Centering	**Mandatory:** Data must be zero-mean centered ($\tilde{\mathbf{X}} = \mathbf{X} - \bar{\mathbf{x}}$) to represent true variance.	**Not strictly required** for mathematical factorization; required only if mimicking PCA.
Equivalence Relation	When applied to mean-centered data $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times p}$, PCA and SVD are algebraically equivalent: the right singular vectors $\mathbf{V}$ equal the principal direction eigenvectors ($\mathbf{W} = \mathbf{V}$), and the singular values satisfy $\lambda_i = \frac{\sigma_i^2}{n-1}$.	

Chapter 9

Reinforcement Learning & Financial Trading

9.1 Principles of Reinforcement Learning

9.1.1 The Trial-and-Error Paradigm

Reinforcement Learning (RL) represents a distinct computational learning paradigm wherein an autonomous decision-maker, termed the **agent**, learns an optimal behavioral strategy through continuous trial-and-error interaction with a dynamic **environment**.

Unlike supervised learning (which relies on static, labeled input-output pairs) and unsupervised learning (which explores latent unlabeled feature geometry), reinforcement learning is characterized by:

- **No Pre-Existing Labeled Answers:** The agent receives no explicit supervisor instructing it on the best action; instead, it evaluates the consequences of its choices via scalar feedback.
- **Reward Signals:** The environment provides evaluative numerical feedback in the form of **rewards** ($r > 0$) or **penalties** ($r < 0$).
- **Delayed Consequences:** Actions executed at time t may not yield immediate feedback; instead, rewards are frequently delayed, requiring the agent to trade off immediate gratification against long-term cumulative gain.
- **Sequential Decision Optimization:** The objective is to discover a behavioral policy π^* that maximizes the total expected cumulative return over extended operational horizons.

9.1.2 The Agent–Environment Interaction Cycle

The fundamental discrete-time interaction loop between the agent and its environment proceeds sequentially through state transitions and action choices:

At discrete time step t:

1. The agent observes the current environmental state $s_t \in \mathcal{S}$.
2. Guided by its internal policy $\pi(a \mid s)$, the agent selects and executes an action $a_t \in \mathcal{A}$.
3. The environment transitions to a new state $s_{t+1} \in \mathcal{S}$ and emits a scalar feedback reward $r_{t+1} \in \mathbb{R}$.
4. The agent incorporates $(s_t, a_t, r_{t+1}, s_{t+1})$ to update its policy or value function.

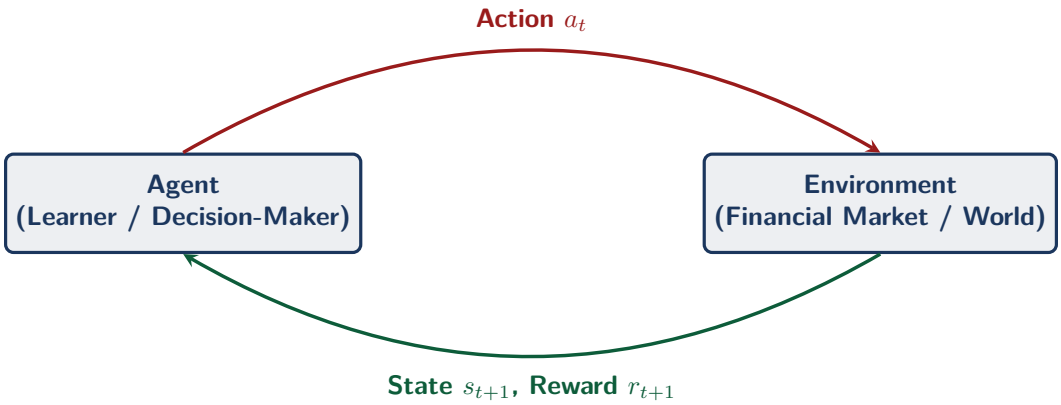


Figure 9.1: The classical Agent–Environment reinforcement learning interaction loop.

9.1.3 Discounted Returns and Core Formal Terminology

The cumulative total return G_t received by the agent following time step t is formulated as a discounted infinite-horizon sum of future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^k r_{t+k+1} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{9.1}$$

where $\gamma \in [0, 1]$ represents the **discount factor**:

- As $\gamma \to 0$, the agent becomes purely myopic ("short-sighted"), optimizing strictly for immediate next-step reward r_{t+1}.
- As $\gamma \to 1$, the agent becomes far-sighted, weighting distant future payoffs nearly equally with immediate returns.

9.1.4 Prominent Real-World Application Domains

Reinforcement learning excels in dynamic environments requiring multi-step strategic planning:

- **Game Playing:** Mastering complex adversarial games with massive state spaces (e.g., Chess, Go).
- **Robotics:** Locomotion, robotic arm manipulation, and balance control.
- **Autonomous Navigation:** Trajectory tracking and obstacle avoidance in self-driving vehicles.
- **Algorithmic Trading & Finance:** Dynamic portfolio rebalancing, market making, order execution, and statistical arbitrage.

9.2 Recurrent Reinforcement Learning (RRL) in Financial Trading

9.2.1 Direct Policy Optimization versus Value-Function Methods

In algorithmic trading, traditional value-based RL methods (e.g., Q-learning) suffer from two distinct failure modes:

1. **Bellman Discretization Error:** Approximating high-dimensional continuous market price densities via discrete value tables induces severe estimation noise.

Table 9.1: Foundational Reinforcement Learning Terminology and Operational Roles

Term	Formal Mathematical Meaning & Operational Function
Agent	The learning entity and autonomous decision-maker responsible for executing actions.
Environment	The external world and dynamical system with which the agent interacts.
State ($s \in \mathcal{S}$)	A complete, sufficient representation of the current situation of the environment.
Action ($a \in \mathcal{A}$)	A valid physical or financial move available to the agent in state s.
Reward ($r \in \mathbb{R}$)	A scalar evaluative numerical feedback signal emitted by the environment following action execution.
Policy (π)	The behavioral strategy of the agent: a mapping from environmental states to action choices, $\pi(s) \to a$.
Value Function $V(s)$	The expected cumulative discounted future reward starting from state s: $V(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$.
Action-Value $Q(s, a)$	The expected cumulative return after taking action a in state s: $Q(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$.
Discount Factor (γ)	Relative valuation parameter $\gamma \in [0, 1]$ weighting immediate versus future rewards.
Episode	One complete, contiguous run of interactions from an initial state to a terminal state.

2. **Absence of Friction Memory:** Portfolio rebalancing generates transaction costs (bid-ask spread crossing, broker commissions, and market impact). Because transaction costs depend directly on the **difference between the current position and the previous position** ($|F_t - F_{t-1}|$), trading decisions cannot be formulated as memoryless Markov states.

Recurrent Reinforcement Learning (RRL) resolves this challenge by parameterizing the trading policy directly with a recurrent neural feedback connection, optimizing portfolio utility without constructing an intermediate value function.

9.2.2 The Recurrent Trading Policy Model

At each discrete trading period t, the trading position F_t is generated via a nonlinear recurrent hyperbolic tangent activation:

$$F_t = \tanh(\mathbf{w}^T \mathbf{x}_t + uF_{t-1} + b) \tag{9.2}$$

where:

- $\mathbf{x}_t \in \mathbb{R}^p$ represents the vector of current market input features (e.g., recent fractional price returns, moving average spreads, order flow imbalances).
- $F_{t-1} \in [-1, 1]$ is the previous trading position fed back via the recurrent parameter u (providing structural memory of existing asset exposure).
- $\mathbf{w} \in \mathbb{R}^p$, $u \in \mathbb{R}$, and $b \in \mathbb{R}$ constitute the learnable parameter set $\boldsymbol{\theta} = \{\mathbf{w}, u, b\}$.

The continuous action output $F_t \in [-1, 1]$ maps directly to financial market allocations:

$$F_t = \begin{cases} +1, & \text{Maximum Long Position (Buy / Asset Accumulation)} \\ 0, & \text{Neutral Exposure (Cash / Risk-Off Position)} \\ -1, & \text{Maximum Short Position (Sell / Short Sale)} \end{cases} \tag{9.3}$$

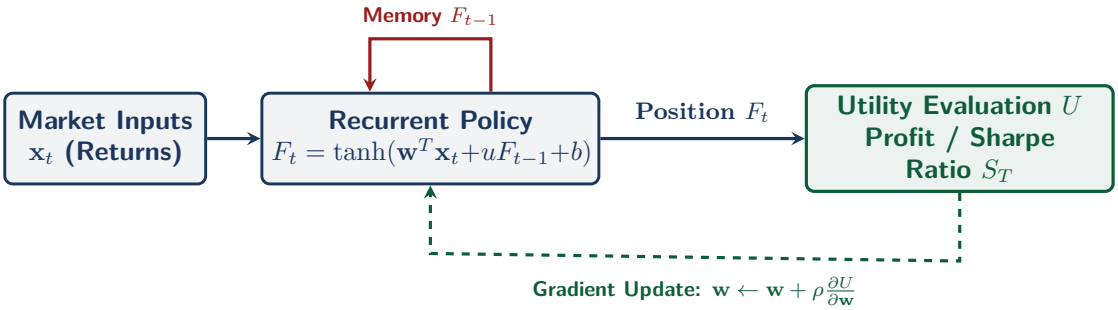


Figure 9.2: Recurrent Reinforcement Learning (RRL) trading architecture with recurrent memory and utility optimization.

9.2.3 Objective Utilities and Gradient Ascent Optimization

The single-period financial return R_t accounting for proportional transaction costs δ is given by:

$$R_t = F_{t-1} r_t - \delta |F_t - F_{t-1}| \tag{9.4}$$

where $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$ represents the raw asset return.

Rather than maximizing unadjusted cumulative wealth, the agent optimizes risk-adjusted performance over an investment horizon T via the **Sharpe Ratio** utility $U = S_T$:

$$S_T = \frac{\text{Mean Return}}{\text{Standard Deviation of Returns}} = \frac{A}{\sqrt{B - A^2}} \tag{9.5}$$

where $A = \frac{1}{T}\sum_{t=1}^{T} R_t$ and $B = \frac{1}{T}\sum_{t=1}^{T} R_t^2$.

The RRL Training Protocol:

1. Generate the complete sequence of sequential positions $\{F_1, F_2, \ldots, F_T\}$ over the historical training window using the recurrent policy.
2. Evaluate the realized sequence of net returns $\{R_1, \ldots, R_T\}$ and compute global utility U (e.g., the Sharpe ratio S_T).
3. Compute parameter gradients via Backpropagation Through Time (BPTT), utilizing the recursive chain rule:
$$\frac{dF_t}{d\mathbf{w}} = (1 - F_t^2)\left(\mathbf{x}_t + u\frac{dF_{t-1}}{d\mathbf{w}}\right) \tag{9.6}$$
4. Update parameters via gradient ascent:
$$\mathbf{w} \leftarrow \mathbf{w} + \rho\frac{\partial U}{\partial \mathbf{w}}, \quad u \leftarrow u + \rho\frac{\partial U}{\partial u}, \quad b \leftarrow b + \rho\frac{\partial U}{\partial b} \tag{9.7}$$
where $\rho > 0$ represents the learning rate.
5. Repeat iterative updates until the portfolio Sharpe ratio converges.

9.3 Action Selection: Exploration versus Exploitation

In reinforcement learning, the agent faces a fundamental operational dilemma:

- **Exploration:** Gathering new experiential information by taking untried actions to discover potentially superior long-term policies.
- **Exploitation:** Maximizing immediate expected reward by executing the best-performing action known under current estimates.

9.3.1 The ϵ-Greedy Action Selection Policy

The standard stochastic compromise balancing exploration and exploitation is the **ϵ-greedy mechanism**:

$$a_t = \begin{cases} \arg\max_{a \in \mathcal{A}} Q(s_t, a), & \text{with probability } 1 - \epsilon \quad \text{(Exploitation)} \\ \text{random action from } \mathcal{A}, & \text{with probability } \epsilon \quad \text{(Exploration)} \end{cases} \tag{9.8}$$

Typically, ϵ is initialized high (e.g., $\epsilon_0 = 1.0$) to promote wide initial exploration and annealed decay over training episodes ($\epsilon \to 0$) toward greedy exploitation.

9.4 Q-Learning: Tabular Temporal-Difference Control

9.4.1 The Q-Table and the Bellman Update Rule

Q-Learning is an off-policy, model-free temporal difference algorithm that learns the optimal action-value function $Q^*(s, a)$ independent of the agent's exploratory behavioral policy.

The agent maintains an explicit Q-table whose rows correspond to discrete states $s \in \mathcal{S}$ and whose columns correspond to discrete actions $a \in \mathcal{A}$. Following execution of action a in state s, observing reward r and transitioning to next state s', the Q-value is updated via the Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{9.9}$$

where:

- $\alpha \in (0,1]$ is the empirical **learning rate**,
- $r + \gamma \max_{a'} Q(s',a')$ represents the **TD Target**,
- $\delta_t = [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ represents the **Temporal-Difference (TD) Error**.

Iterating across sufficient episodes ensures that $Q(s,a)$ converges to the optimal action-values $Q^*(s,a)$. The optimal deterministic policy is extracted as:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a) \tag{9.10}$$

9.5 Comparative Synthesis Across Learning Paradigms

Table 9.2: Systematic Comparison of Supervised, Unsupervised, and Reinforcement Learning

Dimension	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Data Nature	Explicit labeled pairs $(\mathbf{x}_i, y_i)$.	Unlabeled feature vectors $\mathbf{x}_i$.	Dynamic sequential interactions (s, a, r, s').
Feedback Mechanism	Immediate correct label signal (y).	Entirely absent (autonomous structure).	Evaluative scalar reward/penalty (r), often delayed.
Core Objective	Minimize supervised prediction error.	Discover latent clusters / manifolds.	Maximize cumulative discounted returns.
Archetypal Application	Credit default classification, pricing.	Market regime clustering, PCA.	Dynamic trading, algorithmic market making.

Table 9.3: Structural Comparison: Q-Learning (Value-Based) versus RRL (Direct Policy Optimization)

Attribute	Q-Learning (Value-Based)	Recurrent Reinforcement Learning (RRL)
Learned Entity	Approximates value table $Q(s, a)$ or network.	Parameterizes behavioral policy F_t directly.
Action Space	Restricted to discrete action candidates.	Naturally accommodates continuous position sizing $F_t \in [-1, 1]$.
Internal Memory	Purely Markovian (relies strictly on state s_t).	Recurrent feedback connection of prior position F_{t-1}.
Target Objective	Expected cumulative discounted reward G_t.	Any differentiable non-additive utility (e.g., Sharpe ratio S_T).
Transaction Costs	Difficult to incorporate (breaks Markovianity).	Naturally handled via penalty on position change $\|F_t - F_{t-1}\|$.

Chapter 10

Ensemble Learning: Bagging, Boosting, & XGBoost

10.1 Principles of Ensemble Learning

10.1.1 The Wisdom of the Crowd

Ensemble Learning comprises a powerful meta-learning paradigm that strategically combines multiple individual models—termed **base learners** or **weak learners**—to construct a unified composite predictor that exhibits substantially superior predictive accuracy and generalization performance than any individual constituent model.

The foundational principle rests on the *"Wisdom of the Crowd"*:

- **Aggregation Mechanism:** Predictions from diverse base learners are aggregated via **majority voting** (for classification) or **continuous averaging / weighted summation** (for regression).
- **Error Cancellation:** Provided the individual base learners make uncorrelated errors, aggregating their outputs causes independent idiosyncratic errors to cancel out, dampening variance and/or shrinking bias.

10.1.2 Bias–Variance Error Decomposition and Ensemble Strategies

Generalization error decomposes into two fundamental components:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \text{Bias}[\hat{f}(\mathbf{x})]^2 + \text{Var}[\hat{f}(\mathbf{x})] + \sigma_\epsilon^2 \tag{10.1}$$

Ensemble methodologies target specific components of this error decomposition:

10.2 Bootstrap Aggregating (Bagging)

10.2.1 Bootstrap Resampling and Out-of-Bag (OOB) Data

Bagging (Bootstrap Aggregation) trains multiple base learners in parallel on bootstrap resamples of the original training dataset:

- **Bootstrap Resampling:** Given a training dataset D of n observations, a bootstrap sample D_b is constructed by drawing n samples **uniformly at random with replacement**.
- **In-Bag Probability:** The probability that an individual observation is not chosen in any single draw is $1 - \frac{1}{n}$. Across n independent replacement draws, the probability that a

Table 10.1: Model Error Decomposition and Targeted Ensemble Remediation

Error Type	Statistical Meaning & Symptoms	Targeted Ensemble Strategy
High Variance (Overfitting)	The model is excessively sensitive to sample noise in the training set; exhibits high training accuracy but poor generalization to unseen data.	**Bagging (Bootstrap Aggregation)** and **Random Forest**: Averages independent, complex base learners to shrink variance.
High Bias (Underfitting)	The model is structurally too simplistic to capture the true underlying data manifold; underfits both training and test data.	**Boosting (AdaBoost, Gradient Boosting, XGBoost)**: Sequentially combines weak learners to drive down bias.

sample is entirely excluded equals:

$$\lim_{n\to\infty}\left(1-\frac{1}{n}\right)^{n}=\frac{1}{e}\approx 0.3679\approx 36.8\% \tag{10.2}$$

Consequently, approximately 63.2% **of original observations appear in each bootstrap sample**, while the remaining $\approx 36.8\%$ **form the Out-of-Bag (OOB) dataset**.

- **OOB Validation:** The OOB observations serve as an inherent, computationally free validation dataset for evaluating generalization error and tuning hyperparameters without cross-validation overhead.

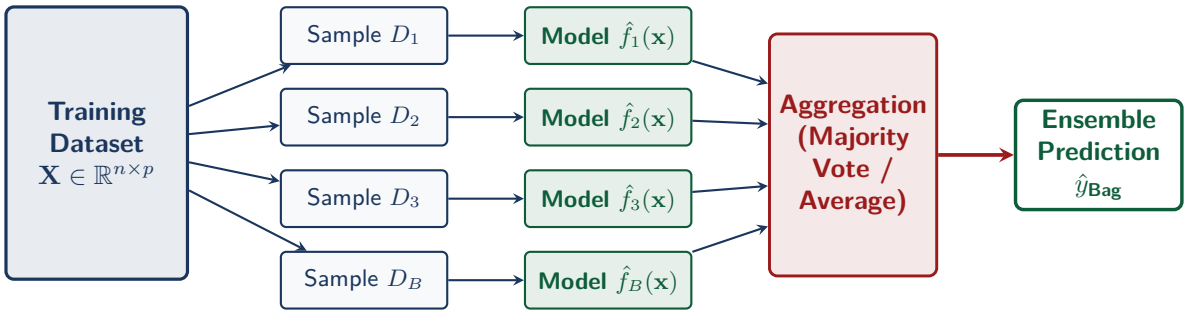


Figure 10.1: Parallel architectural workflow of Bootstrap Aggregation (Bagging).

10.2.2 The Bagging Procedural Protocol

1. Sample B independent bootstrap datasets $D_1, D_2, \ldots, D_B$ with replacement from D.
2. Fit a deep, unpruned base learner $\hat{f}_b(\mathbf{x})$ (typically a high-variance, low-bias decision tree) on each resample D_b.
3. Combine individual predictions across all B base learners:

$$\text{Regression (Averaging):}\quad \hat{f}_{\text{Bag}}(\mathbf{x})=\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x}) \tag{10.3}$$

$$\text{Classification (Majority Voting):}\quad \hat{y}_{\text{Bag}}(\mathbf{x})=\arg\max_{c\in\mathcal{C}}\sum_{b=1}^{B}\mathbb{I}(\hat{f}_b(\mathbf{x})=c) \tag{10.4}$$

10.2.3 Random Forests: De-Correlating Ensembles via Feature Subsampling

Standard Bagging on decision trees often exhibits correlated predictions: if a single dominant predictor exists, virtually all bootstrap trees select that feature at the root split.

Random Forest dramatically enhances Bagging by introducing **random feature subsampling**:

- At every candidate node split during tree growth, only a random subset of $m \ll p$ candidate features is evaluated:

$$m = \lfloor \sqrt{p} \rfloor \quad \text{(for classification)}, \quad m = \lfloor p/3 \rfloor \quad \text{(for regression)} \tag{10.5}$$

- By forcing trees to explore alternate predictive pathways, trees become **mutually decorrelated**, driving the ensemble variance down towards its theoretical minimum $\frac{\sigma^2}{B}$.

10.3 Boosting Paradigms: AdaBoost and Gradient Boosting

10.3.1 The Sequential Error-Focusing Philosophy

Unlike Bagging (which builds strong base models in parallel), **Boosting** constructs an ensemble **sequentially**:

- Each successive model focuses explicitly on the training errors and residuals generated by prior models.
- Base learners are deliberately chosen as **weak learners** (e.g., shallow decision trees or decision stumps with depth 1 or 2).
- The final prediction is formulated as an adaptive weighted sum of weak learner outputs.



Figure 10.2: Sequential error-weighting architecture of Boosting algorithms.

10.3.2 Adaptive Boosting (AdaBoost)

AdaBoost iteratively re-weights observational sample distributions to prioritize misclassified instances:

1. **Uniform Initialization:** Assign equal baseline weights to all n training instances:

$$w_i^{(1)} = \frac{1}{n}, \quad \forall i \in \{1, 2, \ldots, n\} \tag{10.6}$$

2. **Weak Learner Induction:** For rounds $m = 1, 2, \ldots, M$, train a weak base learner $h_m(\mathbf{x}) \in \{-1, +1\}$ on sample weights $\mathbf{w}^{(m)}$.

3. **Evaluate Weighted Classification Error (ϵ_m):**

$$\epsilon_m = \sum_{i=1}^{n} w_i^{(m)} \mathbb{I}(y_i \neq h_m(\mathbf{x}_i)) \tag{10.7}$$

4. **Compute Learner Importance (α_m):**

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right) \tag{10.8}$$

Notice that as error $\epsilon_m \to 0$, importance weight $\alpha_m \to +\infty$; if $\epsilon_m = 0.5$ (random guessing), $\alpha_m = 0$.

5. **Update and Normalize Sample Weights:**

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i))}{Z_m} \tag{10.9}$$

where $Z_m = \sum_{i=1}^{n} w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i))$ is the normalizing partition coefficient ensuring $\sum_i w_i^{(m+1)} = 1$. Misclassified instances receive multiplied weights, forcing subsequent learners to prioritize them.

6. **Final Ensemble Prediction:**

$$H(\mathbf{x}) = \text{sign} \left(\sum_{m=1}^{M} \alpha_m h_m(\mathbf{x}) \right) \tag{10.10}$$

10.3.3 Gradient Boosting

Gradient Boosting generalizes boosting by interpreting sequential training as functional gradient descent in function space:

1. Initialize with a simple static prediction minimizing the loss function (e.g., the sample mean for squared error):

$$F_0(\mathbf{x}) = \arg\min_c \sum_{i=1}^{n} L(y_i, c) \tag{10.11}$$

2. For stages $m = 1, 2, \ldots, M$:

   - Compute the pseudo-residuals as the **negative gradient of the loss function** evaluated at the current model predictions:

$$r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(\mathbf{x}) = F_{m-1}(\mathbf{x})} \tag{10.12}$$

     For Mean Squared Error ($L = \frac{1}{2}(y - \hat{y})^2$), the pseudo-residual reduces exactly to the arithmetic residual: $r_{im} = y_i - F_{m-1}(\mathbf{x}_i)$.
   - Fit a new regression tree $h_m(\mathbf{x})$ to predict the pseudo-residuals r_{im}.
   - Update the cumulative ensemble model via **learning rate shrinkage** $\eta \in (0, 1]$:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}) \tag{10.13}$$

10.3.4 Comprehensive Comparison: Bagging versus Boosting

10.4 XGBoost (Extreme Gradient Boosting)

10.4.1 Algorithmic Innovations and Formal Objective

XGBoost (Extreme Gradient Boosting) is an optimized, highly scalable, and regularized distributed gradient boosting framework engineered for exceptional computational speed and model performance in competitive machine learning and quantitative finance.

Table 10.2: Systematic Comparison Between Bagging and Boosting Ensembles

Dimension	Bagging (e.g., Random Forest)	Boosting (AdaBoost, Gradient Boosting, XGBoost)
Training Architecture	**Parallel** and strictly independent across base models.	**Sequential** and dependent; each model conditions on prior errors.
Data Partitioning	Random bootstrap resampling with replacement (D_b).	Re-weighted training observations or pseudo-residuals.
Primary Objective	**Variance Reduction** (dampens model volatility).	**Bias Reduction** (constructs expressive decision boundaries).
Aggregation Scheme	Unweighted majority voting or equal arithmetic averaging.	Weighted sum proportional to model confidence / gradients.
Base Learner Complexity	**Complex, strong learners** (deep, unpruned trees).	**Simple, weak learners** (shallow trees or decision stumps).
Overfitting Vulnerability	**Low**: resilient to overfitting as ensemble size B grows.	**Higher**: susceptible to fitting noisy outliers if over-trained.
Computational Scaling	Highly parallelizable across multi-core / GPU clusters.	Inherently sequential; requires optimized gradient implementations.

The mathematical formulation optimizes a regularized objective function at step t:

$$\text{Obj}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^{t} \Omega(f_k) \tag{10.14}$$

where $\sum l(y_i, \hat{y}_i)$ represents the convex empirical training loss, and $\Omega(f)$ denotes the **structural complexity regularization penalty**:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{10.15}$$

where:

- T denotes the total number of terminal leaf nodes in the tree,
- w_j represents the continuous output weight assigned to leaf j,
- γ is the minimum loss reduction penalty required to permit an additional leaf split (tree pruning parameter),
- λ is the L_2 regularization coefficient on leaf weights (dampening sensitivity to individual outliers).

10.4.2 Core Technical Features and Quantitative Benefits

The technical mechanisms that distinguish XGBoost are summarized in Table 10.3.

Table 10.3: Key Technical Innovations and Quantitative Benefits of XGBoost

Technical Feature	Algorithmic Mechanism	Quantitative Finance Advantage
Regularization	Direct L_1 (α) and L_2 (λ) penalties embedded into leaf objective derivations.	Restricts extreme portfolio weights; prevents overfitting to noisy market regimes.
Sparsity Awareness	Learns a default splitting direction for missing values during split evaluation.	Robustly handles sparse financial statements and missing macroeconomic data without manual imputation.
Parallel Tree Growth	Employs block-sorted column structures to parallelize split candidate evaluation.	Drastically reduces training runtime over massive multi-asset tick and order book datasets.
Pruning via Depth & γ	Grows trees backward from maximum depth, pruning splits yielding gains below γ.	Eliminates uninformative splits, preserving only statistically significant decision boundaries.
Shrinkage & Subsampling	Shrinks leaf updates by learning rate $\eta \in (0, 1]$; subsamples rows and column features.	Prevents single trees from dominating ensemble decisions; decorrelates sequential trees.
Early Stopping	Monitors validation loss; terminates training when out-of-sample metrics plateau.	Automatically identifies optimal tree cardinality, preventing overtraining on regime shifts.

Comprehensive Formula Reference Sheet

Chapter 1: Machine Learning Foundations and Data Preprocessing

1. Z-Score Standardization

$$Z = \frac{x - \mu}{\sigma}$$

Where:

- x = Raw feature observation value
- μ = Mean of the feature across the dataset ($\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$)
- σ = Standard deviation of the feature ($\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$)
- Z = Standardized score with zero mean ($\mu_Z = 0$) and unit variance ($\sigma_Z^2 = 1$)

2. Min-Max Normalization

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \cdot (\text{new_max} - \text{new_min}) + \text{new_min}$$

Where:

- X = Original unscaled feature value
- $X_{\min}$ = Minimum observed value of the feature in the dataset
- $X_{\max}$ = Maximum observed value of the feature in the dataset
- $\text{new_min}, \text{new_max}$ = Target boundary limits (typically 0 and 1)
- X' = Rescaled feature value constrained within $[\text{new_min}, \text{new_max}]$

3. Robust Feature Scaling

$$X' = \frac{X - Q_2}{\text{IQR}} = \frac{X - \text{Median}}{Q_3 - Q_1}$$

Where:

- X = Raw feature observation value
- Q_2 = Second quartile (median / 50th percentile)
- Q_1 = First quartile (25th percentile)
- Q_3 = Third quartile (75th percentile)

- IQR = Interquartile range (IQR $= Q_3 - Q_1$)
- X' = Outlier-robust scaled feature value

4. Mean Imputation for Missing Values

$$\bar{x} = \frac{1}{n_{\text{obs}}} \sum_{i \in \mathcal{I}_{\text{obs}}} x_i, \qquad x_{\text{missing}} \leftarrow \bar{x}$$

Where:

- $\mathcal{I}_{\text{obs}}$ = Index set of observed (non-missing) entries for feature x
- n_{obs} = Total number of observed values ($n_{\text{obs}} = |\mathcal{I}_{\text{obs}}|$)
- x_i = Observed feature value for sample i
- $\bar{x}$ = Arithmetic mean of observed entries imputed into missing cells

5. One-Hot Indicator Encoding

$$\mathbf{x} = \Big[I(c = c_1) \quad I(c = c_2) \quad \cdots \quad I(c = c_k) \Big]^T, \quad I(c = c_j) = \begin{cases} 1 & \text{if } c = c_j \\ 0 & \text{if } c \neq c_j \end{cases}$$

Where:

- c = Categorical feature taking values in unordered discrete set $\{c_1, c_2, \ldots, c_k\}$
- k = Total cardinality (number of distinct categories)
- $I(\cdot)$ = Binary indicator function
- $\mathbf{x}$ = k-dimensional orthogonal binary vector representation

Chapter 2: K-Nearest Neighbors (KNN)

1. Distance Metrics in Feature Space

$$\text{Euclidean } (L_2): \quad D_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} = \|\mathbf{x} - \mathbf{y}\|_2$$

$$\text{Manhattan } (L_1): \quad D_M(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} |x_i - y_i| = \|\mathbf{x} - \mathbf{y}\|_1$$

$$\text{Minkowski } (L_p): \quad D_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{1/p}$$

Where:

- $\mathbf{x}, \mathbf{y}$ = Coordinate vectors in d-dimensional continuous space $\mathbb{R}^d$
- x_i, y_i = Values of feature dimension i for instances $\mathbf{x}$ and $\mathbf{y}$
- d = Total number of input features
- p = Minkowski norm order parameter ($p = 1$ yields Manhattan, $p = 2$ yields Euclidean)

2. Standard Majority Voting Classification

$$\hat{Y} = \arg\max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} I(y_i = c)$$

Where:

- $\mathbf{x}$ = Unlabeled test query instance
- $\mathcal{N}_k(\mathbf{x})$ = Index set of the k nearest training instances to $\mathbf{x}$ by distance metric D
- y_i = Ground-truth class label of training instance i
- $\mathcal{C}$ = Set of all candidate target classes
- $I(y_i = c)$ = Indicator function equaling 1 if $y_i = c$, and 0 otherwise
- $\hat{Y}$ = Predicted consensus class label

3. Distance-Weighted KNN Classification

$$w_i = \frac{1}{D(\mathbf{x}, \mathbf{x}_i)}, \qquad \hat{Y} = \arg\max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i \cdot I(y_i = c)$$

Where:

- $D(\mathbf{x}, \mathbf{x}_i)$ = Distance from test query $\mathbf{x}$ to i-th neighbor $\mathbf{x}_i$
- w_i = Inverse distance weight assigned to neighbor i
- $\mathcal{N}_k(\mathbf{x})$ = Set of k nearest neighbors
- $\hat{Y}$ = Weighted majority predicted class label

4. KNN Continuous Regression

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i$$

Where:

- $\mathbf{x}$ = Query feature vector
- y_i = Continuous target value of neighbor i
- k = Number of nearest neighbors considered
- $\hat{y}$ = Uniformly averaged predicted response value

Chapter 3: Naive Bayes Classification

1. Bayes' Theorem for Multiclass Classification

$$P(C_k \mid \mathbf{X}) = \frac{P(C_k)\, P(\mathbf{X} \mid C_k)}{P(\mathbf{X})} = \frac{P(C_k)\, P(\mathbf{X} \mid C_k)}{\sum_{j=1}^{K} P(C_j)\, P(\mathbf{X} \mid C_j)}$$

Where:

- $P(C_k \mid \mathbf{X})$ = Posterior probability of class C_k given feature vector $\mathbf{X}$
- $P(C_k)$ = Prior probability of class C_k
- $P(\mathbf{X} \mid C_k)$ = Class-conditional likelihood of feature vector $\mathbf{X}$ given class C_k
- $P(\mathbf{X})$ = Total evidence / marginal probability normalizing across all K classes

- K = Total number of distinct candidate classes

2. Class-Conditional Independence Assumption and MAP Decision Rule

$$P(\mathbf{X} \mid C_k) = \prod_{i=1}^{d} P(x_i \mid C_k)$$

$$\hat{C} = \arg\max_{C_k} \left[P(C_k) \prod_{i=1}^{d} P(x_i \mid C_k) \right]$$

Where:

- $\mathbf{X} = (x_1, x_2, \ldots, x_d)$ = Input feature vector of dimension d
- $P(x_i \mid C_k)$ = Conditional likelihood of individual feature attribute x_i given class C_k
- $\hat{C}$ = Maximum A Posteriori (MAP) predicted class label

3. Maximum Likelihood Categorical Estimator and Laplace Smoothing

$$\text{MLE:} \quad P(x_i = v \mid C_k) = \frac{N_{i,v,k}}{N_k}$$

$$\text{Laplace Smoothing:} \quad P(x_i = v \mid C_k) = \frac{N_{i,v,k} + \alpha}{N_k + \alpha \cdot |V_i|}$$

Where:

- $N_{i,v,k}$ = Number of training instances in class C_k where feature i takes value v
- N_k = Total count of training instances in class C_k
- $|V_i|$ = Total number of possible discrete categories for feature i
- α = Additive smoothing hyperparameter ($\alpha = 1$ for standard Laplace smoothing)

Chapter 4: Logistic Regression

1. Standard Sigmoid (Logistic) Function

$$P(Y = 1 \mid \mathbf{X}) = \sigma(Z) = \frac{1}{1 + e^{-Z}} = \frac{e^{Z}}{1 + e^{Z}}$$

Where:

- $P(Y = 1 \mid \mathbf{X})$ = Predicted posterior probability of the positive class ($Y = 1$)
- Z = Linear predictor / logit argument ($Z \in (-\infty, +\infty)$)
- $\sigma(Z)$ = Sigmoid activation function mapping $\mathbb{R} \to (0, 1)$
- e = Base of the natural logarithm (≈ 2.71828)

2. Linear Predictor and Logit Transformation

$$Z = \beta_0 + \sum_{j=1}^{p} \beta_j X_j = \boldsymbol{\beta}^T \mathbf{X}$$

$$\text{Odds} = \frac{P}{1-P} = e^Z$$

$$\text{Logit}(P) = \ln(\text{Odds}) = \ln\left(\frac{P}{1-P}\right) = Z = \beta_0 + \sum_{j=1}^{p} \beta_j X_j$$

Where:

- P = Probability of the event occurring ($P = P(Y = 1 \mid \mathbf{X})$)
- $1 - P$ = Probability of the non-event ($P(Y = 0 \mid \mathbf{X})$)
- Odds = Ratio of probability of success to probability of failure
- β_0 = Intercept parameter (baseline log-odds when all $X_j = 0$)
- β_j = Regression coefficient representing change in log-odds per unit increase in X_j
- X_j = j-th explanatory feature variable

3. Odds Ratio (OR) and Percentage Change in Odds

$$\text{OR}_j = \frac{\text{Odds}(X_j + 1)}{\text{Odds}(X_j)} = e^{\beta_j}$$

$$\%\Delta\text{Odds} = (e^{\beta_j} - 1) \times 100\%$$

Where:

- OR_j = Multiplicative factor by which the odds change for a 1-unit increase in X_j
- β_j = Log-odds coefficient of feature X_j
- $\%\Delta\text{Odds}$ = Percentage increase (if positive) or decrease (if negative) in odds

4. Binary Classification Decision Rule and Cross-Entropy Cost

$$\hat{Y} = \begin{cases} 1 & \text{if } P(Y = 1 \mid \mathbf{X}) \geq \tau \iff Z \geq \ln\left(\frac{\tau}{1-\tau}\right) \\ 0 & \text{if } P(Y = 1 \mid \mathbf{X}) < \tau \end{cases} \quad (\tau = 0.5 \iff Z \geq 0)$$

$$J(\boldsymbol{\beta}) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \ln(p^{(i)}) + (1 - y^{(i)}) \ln(1 - p^{(i)}) \right]$$

Where:

- τ = Probability classification decision threshold (standard default $\tau = 0.5$)
- $\hat{Y}$ = Predicted binary class label ($\{0, 1\}$)
- m = Number of training examples in the dataset
- $y^{(i)}$ = Actual binary ground-truth label for instance i
- $p^{(i)}$ = Predicted model probability for instance i ($p^{(i)} = \sigma(\boldsymbol{\beta}^T \mathbf{x}^{(i)})$)
- $J(\boldsymbol{\beta})$ = Average cross-entropy (log-loss) objective function

Chapter 5: Decision Trees

1. Shannon Entropy (ID3 / C4.5)

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

$$\text{Binary Case } (c = 2): \quad H(S) = -p \log_2(p) - (1-p)\log_2(1-p)$$

Where:

- S = Collection of training examples at a node
- c = Number of target class categories
- p_i = Proportion of examples in S belonging to class i ($p_i = |S_i|/|S|$)
- $p, 1-p$ = Proportions of positive and negative classes in binary classification
- $H(S)$ = Expected information entropy in bits ($0 \leq H(S) \leq \log_2 c$)

2. Gini Impurity (CART)

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2$$

$$\text{Binary Case } (c = 2): \quad \text{Gini}(S) = 1 - (p^2 + (1-p)^2) = 2p(1-p)$$

Where:

- p_i = Probability of an item being chosen from class i
- p = Proportion of items belonging to the positive class in binary setting
- $\text{Gini}(S)$ = Measure of statistical variance of class distributions ($0 \leq \text{Gini}(S) \leq 1 - 1/c$)

3. Information Gain and Gain Ratio

$$\text{IG}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

$$\text{SplitInfo}(S, A) = -\sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

$$\text{GainRatio}(S, A) = \frac{\text{IG}(S, A)}{\text{SplitInfo}(S, A)}$$

Where:

- $\text{IG}(S, A)$ = Expected reduction in entropy achieved by partitioning S on attribute A
- $\text{Values}(A)$ = Set of distinct branch outcomes of feature attribute A
- S_v = Subset of examples in S for which attribute A has value v
- $|S_v|/|S|$ = Weighting fraction of examples routed to child branch v
- $\text{SplitInfo}(S, A)$ = Potential information generated by splitting S into $|\text{Values}(A)|$ partitions
- $\text{GainRatio}(S, A)$ = Normalized information gain penalizing high-cardinality attributes

4. Reduction in Impurity / Gini Gain (CART)

$$\Delta\text{Gini}(S, A) = \text{Gini}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|}\text{Gini}(S_v)$$

Where:

- $\text{Gini}(S)$ = Impurity of the parent node before splitting
- $\text{Gini}(S_v)$ = Impurity of the child node corresponding to branch value v
- $|S_v|/|S|$ = Fraction of parent records directed to branch v
- $\Delta\text{Gini}(S, A)$ = Total impurity reduction yielded by splitting on attribute A

Chapter 6: Support Vector Machines (SVM)

1. Linear Decision Boundary Hyperplane

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = 0$$

Where:

- $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$ = Input coordinate vector in $\mathbb{R}^d$
- $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$ = Weight vector perpendicular (normal) to the separating hyperplane
- b = Scalar bias term governing the orthogonal displacement of the hyperplane from origin
- $f(\mathbf{x}) = 0$ = Equation of the separating decision boundary

2. Canonical Margin Hyperplanes and Total Margin Width

$$\begin{aligned}
\text{Positive Margin Boundary:} \quad & \mathbf{w}^T\mathbf{x} + b = +1 \\
\text{Negative Margin Boundary:} \quad & \mathbf{w}^T\mathbf{x} + b = -1 \\
\text{Total Margin Width:} \quad & M = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{\sum_{j=1}^{d} w_j^2}}
\end{aligned}$$

Where:

- $\mathbf{w}^T\mathbf{x} + b = +1$ = Hyperplane passing through the closest positive training instances
- $\mathbf{w}^T\mathbf{x} + b = -1$ = Hyperplane passing through the closest negative training instances
- $\|\mathbf{w}\|$ = Euclidean norm (magnitude) of the weight vector ($\sqrt{\mathbf{w}^T\mathbf{w}}$)
- M = Perpendicular distance separating the positive and negative margin hyperplanes

3. Point-to-Hyperplane Perpendicular Distance

$$d_i = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|} = \frac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|}$$

Where:

- $\mathbf{x}_i$ = Coordinate vector of observation i

- $f(\mathbf{x}_i)$ = Evaluated functional value $\mathbf{w}^T\mathbf{x}_i + b$
- $\|\mathbf{w}\|$ = Euclidean norm of the normal vector
- d_i = Exact geometric perpendicular distance from sample point $\mathbf{x}_i$ to decision boundary $f(\mathbf{x}) = 0$

4. Hard-Margin Primal Feasibility Constraint

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad \forall i \in \{1, 2, \ldots, n\}$$

Where:

- $y_i \in \{-1, +1\}$ = Ground-truth binary class label of training observation i
- $\mathbf{x}_i$ = Feature vector of training observation i
- n = Total number of observations in the training dataset
- $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ = Strict equality defining the critical support vectors lying exactly on the margin
- $y_i(\mathbf{w}^T\mathbf{x}_i + b) > 1$ = Strict inequality for interior points located beyond the margin boundaries

5. Axis-Aligned and Diagonal Separating Threshold Formulas

$$c = \frac{p+n}{2}$$

$$\text{Single Coordinate Boundary } (x_k = c): \quad w_k = \frac{2}{p-n}, \quad b = -w_k c = -\frac{p+n}{p-n}$$

$$\text{Diagonal Boundary } (x_2 - x_1 = c): \quad \mathbf{w} = s\begin{bmatrix}-1\\1\end{bmatrix}, \quad b = -s \cdot c, \quad s = \frac{2}{p-n}$$

Where:

- p = Coordinate value of the closest support vector in the positive class ($\min x_k$ or $\min(x_2 - x_1)$)
- n = Coordinate value of the closest support vector in the negative class ($\max x_k$ or $\max(x_2 - x_1)$)
- c = Optimal mid-point threshold maximizing margin width
- $w_k, \mathbf{w}$ = Canonical weight parameter(s) scaled such that $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ at support vectors
- b = Canonical scalar bias parameter

Chapter 7: Unsupervised Learning and Cluster Analysis

1. Metric Proximity and Continuous Distance Norms

$$\text{Euclidean } (L_2): \quad d(\mathbf{p},\mathbf{q}) = \sqrt{\sum_{k=1}^{p}(p_k - q_k)^2}$$

$$\text{Manhattan } (L_1): \quad d(\mathbf{p},\mathbf{q}) = \sum_{k=1}^{p}|p_k - q_k|$$

$$\text{Minkowski } (L_r): \quad d_r(\mathbf{p},\mathbf{q}) = \left(\sum_{k=1}^{p}|p_k - q_k|^r\right)^{1/r}$$

$$\text{Mahalanobis: } \quad d_M(\mathbf{p},\mathbf{q}) = \sqrt{(\mathbf{p}-\mathbf{q})^T\boldsymbol{\Sigma}^{-1}(\mathbf{p}-\mathbf{q})}$$

Where:

- $\mathbf{p},\mathbf{q} \in \mathbb{R}^p$ = Observation vectors in p-dimensional feature space
- p_k, q_k = Coordinate values of feature k for points $\mathbf{p}$ and $\mathbf{q}$
- $r \geq 1$ = Minkowski metric order ($r = 1$ yields Manhattan, $r = 2$ yields Euclidean)
- $\boldsymbol{\Sigma} \in \mathbb{R}^{p\times p}$ = Sample covariance matrix accounting for scale and correlation
- $d(\mathbf{p},\mathbf{q})$ = Evaluated distance metric between observations

2. Binary Co-occurrence Proximity Coefficients

$$\text{Simple Matching Coefficient (SMC): } \quad \text{SMC} = \frac{a+d}{a+b+c+d}$$

$$\text{Jaccard Similarity Coefficient } (J): \quad J = \frac{a}{a+b+c}$$

Where:

- a = Count of attributes present in both objects ($\{1,1\}$ matches)
- b = Count of attributes present in object i but absent in object j ($\{1,0\}$ mismatches)
- c = Count of attributes absent in object i but present in object j ($\{0,1\}$ mismatches)
- d = Count of attributes absent in both objects ($\{0,0\}$ matches)
- SMC, $J \in [0,1]$ = Normalized binary proximity metrics

3. Inter-Cluster Linkage Criteria and Ward's Objective

$$\text{Single Linkage: } \quad d(A,B) = \min_{\mathbf{x}\in A,\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$$

$$\text{Complete Linkage: } \quad d(A,B) = \max_{\mathbf{x}\in A,\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$$

$$\text{Average Linkage: } \quad d(A,B) = \frac{1}{|A||B|}\sum_{\mathbf{x}\in A}\sum_{\mathbf{y}\in B} d(\mathbf{x},\mathbf{y})$$

$$\text{Centroid Linkage: } \quad d(A,B) = \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|_2$$

$$\text{Ward's Increase in Variance: } \quad \Delta\text{ESS} = \frac{|A||B|}{|A|+|B|}\|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2$$

Where:

- A, B = Candidate clusters evaluated for agglomerative merging

- $|A|, |B|$ = Cardinality (number of constituent sample points) of clusters A and B
- $\boldsymbol{\mu}_A, \boldsymbol{\mu}_B$ = Empirical centroid mean vectors of clusters A and B
- ΔESS = Net increase in total within-cluster sum of squared errors

4. K-Means Centroid Update and Convergence Criteria

$$\boldsymbol{\mu}_j = \frac{1}{n_j} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i, \quad \text{WCSS} = \sum_{j=1}^{K} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

Where:

- C_j = Set of observational instances assigned to cluster j
- $n_j = |C_j|$ = Total number of observations belonging to cluster C_j
- $\mathbf{x}_i \in \mathbb{R}^p$ = Feature coordinate vector of observation i
- $\boldsymbol{\mu}_j \in \mathbb{R}^p$ = Updated empirical centroid of cluster j
- WCSS = Within-Cluster Sum of Squares objective function minimized by K-Means

Chapter 8: Dimensionality Reduction: PCA and SVD

1. Sample Covariance Matrix and Eigendecomposition

$$\mathbf{S} = \frac{1}{n-1} \sum_{i=1}^{n} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{n-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

$$\det(\mathbf{S} - \lambda \mathbf{I}) = 0 \quad \Longrightarrow \quad \mathbf{S}\mathbf{e}_j = \lambda_j \mathbf{e}_j \quad (\|\mathbf{e}_j\|_2 = 1)$$

Where:

- $\mathbf{x}_i \in \mathbb{R}^p$ = Observation vector i across p original dimensions
- $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{x}_i$ = Empirical sample mean vector
- $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times p}$ = Mean-centered observation matrix ($\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$)
- $\mathbf{S} \in \mathbb{R}^{p \times p}$ = Unbiased sample covariance matrix
- λ_j = Eigenvalue quantifying variance along principal axis j
- $\mathbf{e}_j \in \mathbb{R}^p$ = Unit-length eigenvector defining principal direction j

2. PCA Subspace Projection and Explained Variance

$$\mathbf{Z} = \tilde{\mathbf{X}} \mathbf{W}_k, \quad \text{EVR}_j = \frac{\lambda_j}{\sum_{m=1}^{p} \lambda_m}$$

Where:

- $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times p}$ = Mean-centered data matrix
- $\mathbf{W}_k = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_k] \in \mathbb{R}^{p \times k}$ = Orthogonal projection matrix containing top k eigenvectors
- $\mathbf{Z} \in \mathbb{R}^{n \times k}$ = Transformed lower-dimensional latent representation ($k \ll p$)
- EVR_j = Explained Variance Ratio captured by principal component j

3. Singular Value Decomposition (SVD) and Low-Rank Form

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T, \quad \sigma_i = \sqrt{\lambda_i(\mathbf{A}^T\mathbf{A})}$$

$$\mathbf{A}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

Where:

- $\mathbf{A} \in \mathbb{R}^{m\times n}$ = Arbitrary rectangular real input matrix
- $\mathbf{U} \in \mathbb{R}^{m\times m}$ = Orthonormal matrix of left singular vectors (eigenvectors of $\mathbf{A}\mathbf{A}^T$)
- $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ = Diagonal matrix of sorted non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$
- $\mathbf{V} \in \mathbb{R}^{n\times n}$ = Orthonormal matrix of right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)
- $\mathbf{A}_k$ = Optimal rank-k truncated matrix approximation minimizing Frobenius error

Chapter 9: Reinforcement Learning and Financial Trading

1. Infinite-Horizon Discounted Return and RRL Trading Policy

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad F_t = \tanh(\mathbf{w}^T\mathbf{x}_t + uF_{t-1} + b)$$

Where:

- G_t = Cumulative discounted return accumulated following decision step t
- $\gamma \in [0, 1]$ = Temporal discount factor ($\gamma \to 0$: myopic; $\gamma \to 1$: far-sighted)
- r_{t+k+1} = Scalar environmental reward emitted at time step $t + k + 1$
- $F_t \in [-1, 1]$ = Continuous trading position (-1: Short, 0: Neutral, $+1$: Long)
- $\mathbf{x}_t \in \mathbb{R}^p$ = Vector of current market features and historical price returns
- F_{t-1} = Prior trading position fed back through recurrent memory parameter u
- $\mathbf{w}, u, b$ = Learnable policy parameters optimized via gradient ascent

2. Portfolio Single-Period Net Return and Sharpe Ratio Utility

$$R_t = F_{t-1}r_t - \delta|F_t - F_{t-1}|, \quad S_T = \frac{A}{\sqrt{B - A^2}}$$

Where:

- R_t = Net realized single-period financial return accounting for trading frictions
- $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$ = Fractional raw market return of the underlying asset
- $\delta \geq 0$ = Transaction cost coefficient (brokerage fee and bid-ask spread friction)
- $|F_t - F_{t-1}|$ = Turnover volume indicating magnitude of portfolio position adjustment
- $A = \frac{1}{T}\sum_{t=1}^{T} R_t, B = \frac{1}{T}\sum_{t=1}^{T} R_t^2$ = First and second moments of returns
- S_T = Empirical Sharpe ratio utility maximized via policy gradient ascent

3. ϵ-Greedy Policy and Tabular Q-Learning Bellman Update

$$a_t = \begin{cases} \arg\max_a Q(s_t, a), & \text{prob } 1-\epsilon \\ \text{random } a \in \mathcal{A}, & \text{prob } \epsilon \end{cases}$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right]$$

Where:

- $\epsilon \in [0,1]$ = Exploration probability balancing exploratory vs. greedy actions
- $Q(s,a)$ = Estimated expected return of executing action a in environmental state s
- $\alpha \in (0,1]$ = Step-size learning rate parameter
- $r + \gamma \max_{a'} Q(s', a')$ = Temporal-difference (TD) Bellman target
- $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s,a)$ = Temporal-difference prediction error

Chapter 10: Ensemble Learning: Bagging, Boosting, and XGBoost

1. Asymptotic Out-of-Bag (OOB) Resampling Rate

$$P(\text{Excluded}) = \lim_{n \to \infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e} \approx 0.3679 \approx 36.8\%$$

Where:

- n = Total sample size of the original training dataset
- $1 - \frac{1}{n}$ = Probability of an individual observation escaping selection in a single random draw
- $P(\text{Excluded}) \approx 36.8\%$ = Fraction of training observations forming the Out-of-Bag validation set
- $P(\text{Included}) \approx 63.2\%$ = Expected fraction of unique observations appearing in each bootstrap sample

2. AdaBoost Learner Importance and Weight Recalibration

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right), \quad w_i^{(m+1)} = \frac{w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i))}{Z_m}$$

Where:

- $\epsilon_m = \sum_{i=1}^n w_i^{(m)} \mathbb{I}(y_i \neq h_m(\mathbf{x}_i))$ = Weighted training classification error of learner m
- α_m = Voting importance weight assigned to weak learner $h_m(\mathbf{x})$
- $w_i^{(m)}$ = Normalized observational sample weight of training instance i at round m
- $y_i \in \{-1, +1\}$ = Ground-truth binary class label
- $h_m(\mathbf{x}_i) \in \{-1, +1\}$ = Discrete prediction rendered by weak learner m
- Z_m = Normalizing partition constant ensuring $\sum_{i=1}^n w_i^{(m+1)} = 1$

3. Gradient Boosting Residual Step and XGBoost Objective

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}), \quad r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$$

$$\text{Obj}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^{t}\left(\gamma T_k + \frac{1}{2}\lambda \sum_{j=1}^{T_k} w_{jk}^2\right)$$

Where:

- $F_m(\mathbf{x})$ = Cumulative ensemble prediction model at iteration stage m
- $\eta \in (0, 1]$ = Learning rate shrinkage parameter dampening each tree's contribution
- r_{im} = Pseudo-residual evaluated as the negative loss gradient
- $h_m(\mathbf{x})$ = Weak regression base tree fitted to predict pseudo-residuals r_{im}
- $l(y_i, \hat{y}_i)$ = Differentiable convex empirical training loss
- T_k = Total number of terminal leaf nodes in tree k
- w_{jk} = Continuous prediction score weight on terminal leaf j
- γ = Minimum loss reduction penalty required to permit an additional leaf split
- $\lambda = L_2$ regularization penalty on leaf weight magnitudes